

NEXUS ソース全文（印刷用）

日付: 2026-08-25

版: 0.1.0+1

ファイル数: 42

合計行数: 11567

この書類は、アプリ本体の現行ソースをファイル単位でそのまま写したものです。印刷・PDF化・紙での保管を想定しています。

含めていないもの: Android / iOS / Windows / Linux / macOS のプラットフォーム雛形、`build/`、`.dart_tool/`、`pubspec.lock`、画像アセットのバイナリ。

収録一覧

1. `README.md` — 13 行
2. `pubspec.yaml` — 27 行
3. `analysis_options.yaml` — 29 行
4. `web/index.html` — 57 行
5. `web/manifest.json` — 36 行
6. `lib/main.dart` — 26 行
7. `lib/app/app.dart` — 91 行
8. `lib/app/theme.dart` — 169 行
9. `lib/core/format.dart` — 74 行
10. `lib/data/app_store.dart` — 1054 行
11. `lib/data/models.dart` — 764 行
12. `lib/data/nexus_prefs.dart` — 51 行
13. `lib/data/nexus_icons.dart` — 47 行
14. `lib/domain/money_calc.dart` — 67 行
15. `lib/domain/money_catalog.dart` — 117 行
16. `lib/domain/review_scheduler.dart` — 24 行
17. `lib/domain/daily_quotes.dart` — 448 行
18. `lib/domain/day_occasions.dart` — 1653 行
19. `lib/screens/app_shell.dart` — 51 行
20. `lib/screens/home/home_screen.dart` — 357 行
21. `lib/screens/home/home_header.dart` — 145 行
22. `lib/screens/home/home_widgets.dart` — 331 行
23. `lib/screens/study/study_screen.dart` — 1121 行
24. `lib/screens/study/focus_timer_page.dart` — 205 行
25. `lib/screens/life/life_screen.dart` — 684 行
26. `lib/screens/life/sleep_sheet.dart` — 171 行
27. `lib/screens/money/money_screen.dart` — 398 行
28. `lib/screens/money/money_forms.dart` — 822 行
29. `lib/screens/money/box_detail_page.dart` — 237 行

- 30. `lib/screens/settings/settings_screen.dart` — 220 行
 - 31. `lib/screens/ai/ai_screen.dart` — 255 行
 - 32. `lib/screens/ai/negumo_mascot.dart` — 72 行
 - 33. `lib/widgets/ui_bits.dart` — 480 行
 - 34. `lib/widgets/glass_card.dart` — 81 行
 - 35. `lib/widgets/nexus_nav_bar.dart` — 127 行
 - 36. `lib/widgets/nexus_logo.dart` — 135 行
 - 37. `lib/widgets/progress_ring.dart` — 89 行
 - 38. `lib/widgets/schedule_sheet.dart` — 140 行
 - 39. `lib/widgets/duration_picker.dart` — 228 行
 - 40. `lib/widgets/count_up_yen.dart` — 46 行
 - 41. `test/widget_test.dart` — 113 行
 - 42. `test/home_content_test.dart` — 312 行
-

ファイル: README.md

行数: 13

```
# NEXUS

Study / Life / Money をひとつにまとめるスマホ向けパーソナルハブ (Flutter)。

## 開発
```

flutter pub get

flutter run

```
実機またはエミュレータ (Android / iOS) での起動を想定しています。PC 上で動かした場合も、スマホ幅の画面として表示します。
```

ファイル: pubspec.yaml

行数: 27

```
name: nexus
description: "NEXUS – personal hub for study, life, and money."
publish_to: "none"

version: 0.1.0+1

environment:
  sdk: ^3.12.2

dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8
  google_fonts: ^6.3.2
  image_picker: ^1.2.3
  shared_preferences: ^2.5.5

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^6.0.0

flutter:
  uses-material-design: true
  assets:
    - assets/branding/nexus_mark.png
```

ファイル: analysis_options.yaml

行数: 29

```
# This file configures the analyzer, which statically analyzes Dart code to
# check for errors, warnings, and lints.
#
# The issues identified by the analyzer are surfaced in the UI of Dart-enabled
# IDEs (https://dart.dev/tools#ides-and-editors). The analyzer can also be
# invoked from the command line by running `flutter analyze`.
#
# The following line activates a set of recommended lints for Flutter apps,
# packages, and plugins designed to encourage good coding practices.
include: package:flutter_lints/flutter.yaml

linter:
  # The lint rules applied to this project can be customized in the
  # section below to disable rules from the `package:flutter_lints/flutter.yaml`
  # included above or to enable additional rules. A list of all available lints
  # and their documentation is published at https://dart.dev/lints.
  #
  # Instead of disabling a lint rule for the entire project in the
  # section below, it can also be suppressed for a single line of code
  # or a specific dart file by using the `// ignore: name_of_lint` and
  # `// ignore_for_file: name_of_lint` syntax on the line or in the file
  # producing the lint.
  rules:
    # avoid_print: false # Uncomment to disable the `avoid_print` rule
    # prefer_single_quotes: true # Uncomment to enable the `prefer_single_quotes` rule

# Additional information about this file can be found at
# https://dart.dev/guides/language/analysis-options
```

ファイル: web/index.html

行数: 57

```
<!DOCTYPE html>
<html>
<head>
  <base href="$FLUTTER_BASE_HREF">

  <meta charset="UTF-8">
  <meta content="IE=edge" http-equiv="X-UA-Compatible">
  <meta name="description" content="NEXUS - Study / Money / Life / Health をひとつにまとめるパーソナルハブ">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="theme-color" content="#050916">

  <meta property="og:title" content="NEXUS">
  <meta property="og:description" content="Study / Money / Life / Health をひとつにまとめるパーソナルハブ">
  <meta property="og:type" content="website">

  <meta name="mobile-web-app-capable" content="yes">
  <meta name="apple-mobile-web-app-status-bar-style" content="black">
  <meta name="apple-mobile-web-app-title" content="NEXUS">
  <link rel="apple-touch-icon" href="icons/Icon-192.png">

  <link rel="icon" type="image/png" href="favicon.png"/>

  <title>NEXUS</title>
  <link rel="manifest" href="manifest.json">

<style>
  html, body {
    margin: 0;
    padding: 0;
    height: 100%;
    background: #050916;
    color: #ffffff;
    font-family: system-ui, -apple-system, Segoe UI, sans-serif;
  }
  .loading {
    display: flex;
    align-items: center;
    justify-content: center;
    height: 100%;
    letter-spacing: 0.08em;
    font-weight: 700;
    opacity: 0.7;
  }
</style>
</head>
<body>
  <div class="loading" id="loading">NEXUS</div>
  <script>
    window.addEventListener('flutter-first-frame', function () {
      var el = document.getElementById('loading');
      if (el) el.remove();
    });
  </script>
  <script src="flutter_bootstrap.js" async></script>
</body>
</html>
```

ファイル: web/manifest.json

行数: 36

```
{
  "name": "NEXUS",
  "short_name": "NEXUS",
  "start_url": ".",
  "display": "standalone",
  "background_color": "#050916",
  "theme_color": "#050916",
  "description": "Study / Money / Life / Health をひとつにまとめるパーソナルハブ",
  "orientation": "any",
  "prefer_related_applications": false,
  "icons": [
    {
      "src": "icons/Icon-192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "icons/Icon-512.png",
      "sizes": "512x512",
      "type": "image/png"
    },
    {
      "src": "icons/Icon-maskable-192.png",
      "sizes": "192x192",
      "type": "image/png",
      "purpose": "maskable"
    },
    {
      "src": "icons/Icon-maskable-512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "maskable"
    }
  ]
}
```

ファイル: lib/main.dart

行数: 26

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

import 'app/app.dart';
import 'app/theme.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await SystemChrome.setPreferredOrientations([
    DeviceOrientation.portraitUp,
  ]);

  SystemChrome.setSystemUIOverlayStyle(
    const SystemUiOverlayStyle(
      statusBarColor: Colors.transparent,
      statusBarIconBrightness: Brightness.dark,
      statusBarBrightness: Brightness.light,
      systemNavigationBarColor: NexusColors.background,
      systemNavigationBarIconBrightness: Brightness.dark,
    ),
  );

  runApp(const NexusApp());
}
```

ファイル: lib/app/app.dart

行数: 91

```
import 'package:flutter/material.dart';

import '../app/theme.dart';
import '../data/app_store.dart';
import '../screens/app_shell.dart';

class NexusApp extends StatefulWidget {
  const NexusApp({super.key});

  @override
  State<NexusApp> createState() => _NexusAppState();
}

class _NexusAppState extends State<NexusApp> {
  final AppStore _store = AppStore.seed();

  @override
  void initState() {
    super.initState();
    _store.hydrate();
  }

  @override
  void dispose() {
    _store.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return AppScope(
      store: _store,
      child: MaterialApp(
        title: 'NEXUS',
        debugShowCheckedModeBanner: false,
        theme: NexusTheme.light,
        themeMode: ThemeMode.light,
        builder: (context, child) {
          return PhoneScope(child: child ?? const SizedBox.shrink());
        },
        home: const AppShell(),
      ),
    );
  }
}

class PhoneScope extends StatelessWidget {
  const PhoneScope({super.key, required this.child});

  final Widget child;

  static const double phoneWidth = 390;

  @override
  Widget build(BuildContext context) {
    final media = MediaQuery.of(context);
    if (media.size.width <= 500) return child;

    final height = media.size.height < phoneWidth * 19.5 / 9
      ? media.size.height
      : phoneWidth * 19.5 / 9;

    return ColoredBox(
      color: const Color(0xFFE8E2D8),
      child: Center(
        child: Container(
          width: phoneWidth,
          height: height,
          decoration: BoxDecoration(
```

```
borderRadius: BorderRadius.circular(28),
boxShadow: [
  BoxShadow(
    color: const Color(0x33000000),
    blurRadius: 42,
    spreadRadius: 2,
  ),
],
),
child: ClipRRect(
  borderRadius: BorderRadius.circular(28),
  child: MediaQuery(
    data: media.copyWith(size: Size(phoneWidth, height)),
    child: child,
  ),
),
),
),
);
}
```

ファイル: lib/app/theme.dart

行数: 169

```
import 'package:flutter/material.dart';
import 'package:google_fonts/google_fonts.dart';

class NexusColors {
  NexusColors._();

  static const Color background = Color(0xFFFFF4F1EB);
  static const Color surface = Color(0xFFFFF8CF8);
  static const Color card = Color(0xFFFFFFFF);
  static const Color cardTop = Color(0xFFFFFDFB);
  static const Color navBar = Color(0xF2FFFFFF);

  static const Color cyan = Color(0xFF4C8DFF);
  static const Color cyanMuted = Color(0xFF6F90C8);
  static const Color purple = Color(0xFF8B8FD9);
  static const Color green = Color(0xFF6FBF8A);
  static const Color gold = Color(0xFFC4A574);
  static const Color income = Color(0xFF5AAA78);
  static const Color expense = Color(0xFFE06B7A);

  static const Color peach = Color(0xFFFFFE4D1);
  static const Color sage = Color(0xFFD8EDDA);
  static const Color sky = Color(0xFFD7E8FB);
  static const Color lilac = Color(0xFFFEDE4F8);
  static const Color cream = Color(0xFFFFF0D6);

  static const Color text = Color(0xFF2C2A28);
  static const Color textSecondary = Color(0xFF6F6B66);
  static const Color textMuted = Color(0xFF9A958E);
  static const Color border = Color(0xFFE6E1D8);
  static const Color hairline = Color(0x1A2C2A28);

  static const double cardRadius = 20;
}

class NexusTheme {
  NexusTheme._();

  static ThemeData get dark => light;

  static ThemeData get light {
    const scheme = ColorScheme.light(
      primary: NexusColors.cyan,
      secondary: NexusColors.purple,
      surface: NexusColors.surface,
      error: NexusColors.expense,
      onPrimary: Colors.white,
      onSecondary: Colors.white,
      onSurface: NexusColors.text,
      onError: Colors.white,
    );

    final radius = BorderRadius.circular(14);
    final base = ThemeData(
      useMaterial3: true,
      brightness: Brightness.light,
      colorScheme: scheme,
      scaffoldBackgroundColor: NexusColors.background,
      canvasColor: NexusColors.surface,
      dividerColor: NexusColors.border,
      appBarTheme: const AppBarTheme(
        backgroundColor: Colors.transparent,
        elevation: 0,
        scrolledUnderElevation: 0,
        foregroundColor: NexusColors.text,
      ),
      filledButtonTheme: FilledButtonThemeData(
        style: FilledButton.styleFrom(
          backgroundColor: NexusColors.cyan,
```

```

        foregroundColor: Colors.white,
        elevation: 0,
        padding: const EdgeInsets.symmetric(horizontal: 18, vertical: 12),
        shape: RoundedRectangleBorder(borderRadius: radius),
        textStyle: const TextStyle(fontWeight: FontWeight.w700, letterSpacing: 0.2),
    ),
),
outlinedButtonTheme: OutlinedButtonThemeData(
    style: OutlinedButton.styleFrom(
        foregroundColor: NexusColors.textSecondary,
        side: const BorderSide(color: NexusColors.border),
        padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
        shape: RoundedRectangleBorder(borderRadius: radius),
    ),
),
textButtonTheme: TextButtonThemeData(
    style: TextButton.styleFrom(foregroundColor: NexusColors.cyan),
),
inputDecorationTheme: InputDecorationTheme(
    filled: true,
    fillColor: NexusColors.surface,
    hintStyle: const TextStyle(color: NexusColors.textMuted, fontSize: 13),
    labelStyle: const TextStyle(color: NexusColors.textSecondary, fontSize: 13),
    contentPadding: const EdgeInsets.symmetric(horizontal: 14, vertical: 12),
    enabledBorder: OutlineInputBorder(
        borderRadius: radius,
        borderSide: const BorderSide(color: NexusColors.border),
    ),
    focusedBorder: OutlineInputBorder(
        borderRadius: radius,
        borderSide: const BorderSide(color: NexusColors.cyan, width: 1.4),
    ),
),
chipTheme: ChipThemeData(
    backgroundColor: NexusColors.sky.withValues(alpha: 0.7),
    selectedColor: NexusColors.cyan.withValues(alpha: 0.18),
    disabledColor: NexusColors.surface,
    labelStyle: const TextStyle(color: NexusColors.text, fontSize: 12),
    secondaryLabelStyle: const TextStyle(color: NexusColors.cyan, fontSize: 12),
    padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 2),
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(20)),
    side: BorderSide.none,
),
sliderTheme: SliderThemeData(
    activeTrackColor: NexusColors.cyan,
    inactiveTrackColor: NexusColors.border,
    thumbColor: NexusColors.cyan,
    overlayColor: NexusColors.cyan.withValues(alpha: 0.16),
),
switchTheme: SwitchThemeData(
    thumbColor: WidgetStateProperty.resolveWith(
        (states) => states.contains(WidgetState.selected) ? Colors.white : NexusColors.textMuted,
    ),
    trackColor: WidgetStateProperty.resolveWith(
        (states) => states.contains(WidgetState.selected)
            ? NexusColors.cyan
            : NexusColors.border,
    ),
),
dialogTheme: DialogThemeData(
    backgroundColor: NexusColors.card,
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(NexusColors.cardRadius)),
    titleTextStyle: const TextStyle(color: NexusColors.text, fontSize: 18, fontWeight: FontWeight.w700),
    contentTextStyle: const TextStyle(color: NexusColors.textSecondary, height: 1.45),
),
datePickerTheme: DatePickerThemeData(
    backgroundColor: NexusColors.card,
    headerForegroundColor: NexusColors.text,
    dayForegroundColor: WidgetStateProperty.all(NexusColors.text),
    todayForegroundColor: WidgetStateProperty.all(NexusColors.cyan),
    todayBackgroundColor: WidgetStateProperty.all(NexusColors.cyan.withValues(alpha: 0.16)),
),
snackBarTheme: SnackBarThemeData(
    backgroundColor: NexusColors.text,
    contentTextStyle: const TextStyle(color: Colors.white),
    behavior: SnackBarBehavior.floating,
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(14)),

```

```
    ),
    dividerTheme: const DividerThemeData(color: NexusColors.border, space: 1, thickness: 1),
    progressIndicatorTheme: const ProgressIndicatorThemeData(
      color: NexusColors.cyan,
      linearTrackColor: NexusColors.border,
    ),
    iconButtonTheme: IconButtonThemeData(
      style: IconButton.styleFrom(foregroundColor: NexusColors.textSecondary),
    ),
  );

  final textTheme = GoogleFonts.notoSansJpTextTheme(base.textTheme).apply(
    bodyColor: NexusColors.textSecondary,
    displayColor: NexusColors.text,
  );

  return base.copyWith(
    textTheme: textTheme,
    primaryTextTheme: textTheme,
  );
}
```

ファイル: lib/core/format.dart

行数: 74

```
String yen(int amount) {
  final negative = amount < 0;
  final digits = amount.abs().toString();
  final buffer = StringBuffer();
  for (var i = 0; i < digits.length; i++) {
    if (i > 0 && (digits.length - i) % 3 == 0) buffer.write(',');
    buffer.write(digits[i]);
  }
  return '${negative ? '-' : ''}¥$buffer';
}

String two(int n) => n.toString().padLeft(2, '0');

String jpDate(DateTime d) => '${d.year}年 ${d.month}月 ${d.day}日';

String jpMonth(DateTime d) => '${d.year}年${d.month}月';

String hm(DateTime d) => '${two(d.hour)}:${two(d.minute)}';

String mmss(int seconds) {
  final s = seconds < 0 ? 0 : seconds;
  final hours = s ~/ 3600;
  final minutes = (s % 3600) ~/ 60;
  final rest = s % 60;
  if (hours > 0) return '$hours:${two(minutes)}:${two(rest)}';
  return '${two(minutes)}:${two(rest)}';
}

/// 残りの勉強時間。60分以上は時間、未満は分数で出す。
String remainingStudyLabel(int remainingMinutes) {
  if (remainingMinutes <= 0) return '達成';
  if (remainingMinutes >= 60) {
    final hours = remainingMinutes ~/ 60;
    final minutes = remainingMinutes % 60;
    if (minutes == 0) return '$hours時間';
    return '$hours時間$minutes分';
  }
  return '$remainingMinutes分';
}

String studyGoalLabel(int minutes) {
  if (minutes >= 60) {
    final hours = minutes ~/ 60;
    final rest = minutes % 60;
    if (rest == 0) return '$hours時間';
    return '$hours時間$rest分';
  }
  return '$minutes分';
}

const weekLabels = ['月', '火', '水', '木', '金', '土', '日'];

int mondayIndex(DateTime d) => (d.weekday + 6) % 7;

String weekdayLabelOf(DateTime d) => weekLabels[mondayIndex(d)];

String daysLeftLabel(DateTime due, DateTime today) {
  final days = dateOnly(due).difference(dateOnly(today)).inDays;
  if (days < 0) return '期限切れ ${-days}日';
  if (days == 0) return '今日まで';
  if (days == 1) return 'あと1日';
  return 'あと$days日';
}

DateTime dateOnly(DateTime d) => DateTime(d.year, d.month, d.day);

String dateKey(DateTime d) {
  final day = dateOnly(d);
  return '${day.year}-${two(day.month)}-${two(day.day)}';
}
```

```
}  
  
bool sameDay(DateTime a, DateTime b) =>  
    a.year == b.year && a.month == b.month && a.day == b.day;
```

ファイル: lib/data/app_store.dart

行数: 1054

```
import 'dart:typed_data';

import 'package:flutter/material.dart';

import '../app/theme.dart';
import '../core/format.dart';
import '../domain/daily_quotes.dart';
import '../domain/day_occasions.dart';
import '../domain/money_calc.dart';
import '../domain/review_scheduler.dart';
import '../widgets/nexus_nav_bar.dart';
import 'models.dart';
import 'nexus_prefs.dart';

class AppStore extends ChangeNotifier {
  AppStore.seed() {
    _seed();
  }

  int _seq = 0;
  String _id() => 'n${_seq++}';

  int tabIndex = NexusTab.home;
  DateTime focusedDate = dateOnly(DateTime.now());
  DateTime lifeDate = dateOnly(DateTime.now());

  String userName = '蒼井 ユウ';
  int level = 1;
  double levelProgress = 0;

  int dailyStudyGoalMinutes = 120;
  DateTime todayStudyDate = dateOnly(DateTime.now());
  int todayStudyLoggedSeconds = 0;

  final List<ScheduleItem> schedules = [];
  final List<StudySubject> subjects = [];
  final List<StudySession> sessions = [];
  final List<StudyGoal> goals = [];
  final List<Assignment> assignments = [];
  final List<Exam> exams = [];
  final List<LearningEntry> diaryPosts = [];
  final List<ProblemRecord> problems = [];
  final List<ReviewCard> reviewCards = [];
  final List<Habit> habits = [];
  final List<BudgetBox> boxes = [];
  final List<MoneyCard> cards = [];
  final List<IncomeEntry> incomes = [];
  final List<PaymentPlan> payments = [];
  final List<ChatMessage> messages = [];

  bool _canSave = false;

  int income = 0;
  double weekStudyHours = 0;
  double totalStudyHours = 0;
  List<double> weekBars = const [0, 0, 0, 0, 0, 0, 0];
  String nextTaskTitle = '';
  String nextTaskDuration = '';
  String nextStudySubjectId = '';
  String nextStudyPlace = '';
  DateTime nextStudyAt = DateTime(
    DateTime.now().year,
    DateTime.now().month,
    DateTime.now().day,
    16,
    30,
  );

  int timerTotalSeconds = 30 * 60;
```



```

int timerAccumulatedSeconds = 0;
DateTime? timerStartedAt;
bool timerRunning = false;
String? timerSubjectId;

final List<SleepLog> sleepLogs = [];
DateTime? sleepStartedAt;

bool get isSleeping => sleepStartedAt != null;
int mood = 0;
int energy = 0;
int steps = 0;
int stepGoal = 10000;
String diary = '';

UserSettings settings = const UserSettings();
AiProposal? proposal;
String lastToast = '';

SleepLog? sleepLogOn(DateTime day) {
  final key = dateKey(day);
  for (final log in sleepLogs) {
    if (dateKey(log.wakeDate) == key) return log;
  }
  return null;
}

SleepLog? get latestSleepLog {
  if (sleepLogs.isEmpty) return null;
  final sorted = [...sleepLogs]..sort((a, b) => b.wakeAt.compareTo(a.wakeAt));
  return sorted.first;
}

double get sleepHours => latestSleepLog?.hours ?? 0;

List<double> sleepWeekHours([DateTime? day]) {
  final anchor = dateOnly(day ?? focusedDate);
  final monday = anchor.subtract(Duration(days: mondayIndex(anchor)));
  return [
    for (var i = 0; i < 7; i++)
      sleepLogOn(monday.add(Duration(days: i)))?.hours ?? 0,
  ];
}

MoneySnapshot get money {
  final day = focusedDate;
  final budgetBoxes = [
    for (final box in boxes.where((b) => !b.isSavings))
      box.copyWith(spent: spentOfBox(box.id, periodOf: box, day: day)),
  ];
  return MoneyCalc.compute(
    income: incomeForMonth(day),
    boxes: budgetBoxes,
    payments: payments,
    today: day,
  );
}

int incomeForMonth(DateTime day) {
  return incomes
    .where((e) => e.useYear == day.year && e.useMonth == day.month)
    .fold<int>(0, (sum, e) => sum + e.amount);
}

int spentOfBox(
  String boxId, {
  DateTime? month,
  BudgetBox? periodOf,
  DateTime? day,
}) {
  DateTime? start;
  DateTime? end;
  if (periodOf != null) {
    final period = budgetPeriod(periodOf, day ?? focusedDate);
    start = period.start;
    end = period.end;
  }
}

```

```

return cards.where((c) {
    if (c.boxId != boxId || c.kind != MoneyCardKind.spend) return false;
    if (start != null && end != null) {
        final at = dateOnly(c.at);
        return !at.isBefore(dateOnly(start)) && !at.isAfter(dateOnly(end));
    }
    if (month != null) return c.at.year == month.year && c.at.month == month.month;
    return true;
}).fold<int>(0, (sum, c) => sum + c.amount);
}

(DateTime start, DateTime end) budgetPeriod(BudgetBox box, DateTime today) {
    final d = dateOnly(today);
    int clampDay(DateTime month, int day) {
        final last = DateTime(month.year, month.month + 1, 0).day;
        return day.clamp(1, last);
    }

    late DateTime start;
    if (d.day >= clampDay(d, box.renewalDay)) {
        start = DateTime(d.year, d.month, clampDay(d, box.renewalDay));
    } else {
        final prev = DateTime(d.year, d.month - 1, 1);
        start = DateTime(prev.year, prev.month, clampDay(prev, box.renewalDay));
    }
    final next = DateTime(start.year, start.month + 1, 1);
    final nextStart = DateTime(next.year, next.month, clampDay(next, box.renewalDay));
    return (start: start, end: nextStart.subtract(const Duration(days: 1)));
}

int savingsBalance(BudgetBox box) {
    var value = box.openingAmount;
    for (final card in cards.where((c) => c.boxId == box.id)) {
        if (card.kind == MoneyCardKind.saveIn) value += card.amount;
        if (card.kind == MoneyCardKind.saveOut) value -= card.amount;
    }
    return value;
}

List<MoneyCard> cardsForBox(String boxId, {DateTime? month}) {
    final list = cards.where((c) {
        if (c.boxId != boxId) return false;
        if (month == null) return true;
        return c.at.year == month.year && c.at.month == month.month;
    }).toList();
    list.sort((a, b) => b.at.compareTo(a.at));
    return list;
}

BudgetBox? boxById(String id) {
    for (final b in boxes) {
        if (b.id == id) return b;
    }
    return null;
}

DateTime nextPaymentDue(PaymentPlan plan, DateTime today) {
    var due = dateOnly(plan.dueAt);
    final now = dateOnly(today);
    if (plan.repeat == PaymentRepeat.none || !due.isBefore(now)) return due;
    while (due.isBefore(now)) {
        due = plan.repeat == PaymentRepeat.yearly
            ? DateTime(due.year + 1, due.month, due.day)
            : DateTime(due.year, due.month + 1, due.day);
    }
    return due;
}

DayOccasion get todayOccasion => occasionFor(focusedDate);

DailyQuote get todayQuote => quoteFor(focusedDate);

double get goalProgress {
    final goal = dailyStudyGoalMinutes * 60;
    if (goal <= 0) return 0;
    return (todayStudySeconds() / goal).clamp(0.0, 1.0);
}

```

```

int todayStudySeconds() {
    _rollTodayStudyIfNeeded();
    var seconds = todayStudyLoggedSeconds;
    if (timerRunning) seconds += timerElapsedSeconds();
    return seconds;
}

int remainingStudyMinutes() {
    final remain = dailyStudyGoalMinutes * 60 - todayStudySeconds();
    if (remain <= 0) return 0;
    return (remain / 60).ceil();
}

void _rollTodayStudyIfNeeded() {
    final today = dateOnly(DateTime.now());
    if (sameDay(todayStudyDate, today)) return;
    todayStudyDate = today;
    todayStudyLoggedSeconds = 0;
}

List<ScheduleItem> schedulesOn(DateTime day) {
    final items = schedules.where((s) => sameDay(s.startAt, day)).toList()
        ..sort((a, b) => a.startAt.compareTo(b.startAt));
    return items;
}

StudySubject? subjectById(String id) {
    for (final s in subjects) {
        if (s.id == id) return s;
    }
    return null;
}

List<List<double>> weekStackedHours() {
    final monday = dateOnly(focusedDate).subtract(Duration(days: mondayIndex(focusedDate)));
    final stacks = List.generate(subjects.isEmpty ? 0 : 7, (_) {
        return List<double>.filled(subjects.length, 0);
    });
    if (subjects.isEmpty) return stacks;
    for (final session in sessions) {
        final idx = dateOnly(session.at).difference(monday).inDays;
        if (idx < 0 || idx > 6) continue;
        final si = subjects.indexWhere((s) => s.id == session.subjectId);
        if (si < 0) continue;
        stacks[idx][si] += session.minutes / 60.0;
    }
    return stacks;
}

void _refreshWeekBars() {
    final stacks = weekStackedHours();
    if (stacks.isEmpty) {
        weekBars = List.filled(7, 0);
        weekStudyHours = 0;
        return;
    }
    final totals = [
        for (final day in stacks) day.fold<double>(0, (a, b) => a + b),
    ];
    final max = totals.fold<double>(0, (a, b) => a > b ? a : b);
    weekBars = [
        for (final total in totals) max <= 0 ? 0.2 : (total / max).clamp(0.08, 1.0),
    ];
    weekStudyHours = double.parse(
        totals.fold<double>(0, (a, b) => a + b).toStringAsFixed(1),
    );
    for (var i = 0; i < subjects.length; i++) {
        final hours = stacks.fold<double>(0, (sum, day) => sum + day[i]);
        subjects[i] = subjects[i].copyWith(weekHours: double.parse(hours.toStringAsFixed(1)));
    }
}

int reviewDueCount() {
    return reviewCards
        .where((c) => c.status == 'pending' && !c.dueAt.isAfter(focusedDate))
        .length;
}

```

```

}

int timerElapsedSeconds() {
    var elapsed = timerAccumulatedSeconds;
    if (timerRunning && timerStartedAt != null) {
        elapsed += DateTime.now().difference(timerStartedAt!).inSeconds;
    }
    return elapsed.clamp(0, timerTotalSeconds);
}

int timerRemainingSeconds() =>
    (timerTotalSeconds - timerElapsedSeconds()).clamp(0, timerTotalSeconds);

void goTo(int tab) {
    tabIndex = tab.clamp(NexusTab.home, NexusTab.settings);
    notifyListeners();
}

void setFocusedDate(DateTime day) {
    focusedDate = dateOnly(day);
    notifyListeners();
}

void setLifeDate(DateTime day) {
    lifeDate = dateOnly(day);
    notifyListeners();
}

void addSchedule({
    required String title,
    required DateTime startAt,
    String category = 'life',
}) {
    schedules.add(
        ScheduleItem(id: _id(), title: title, startAt: startAt, category: category),
    );
    lastToast = '予定を追加しました';
    notifyListeners();
}

void updateSchedule(ScheduleItem item) {
    final i = schedules.indexWhere((s) => s.id == item.id);
    if (i < 0) return;
    schedules[i] = item;
    notifyListeners();
}

void deleteSchedule(String id) {
    schedules.removeWhere((s) => s.id == id);
    lastToast = '予定を削除しました';
    notifyListeners();
}

void toggleHabit(String habitId, DateTime day) {
    final i = habits.indexWhere((h) => h.id == habitId);
    if (i < 0) return;
    final key = dateKey(day);
    final next = {...habits[i].doneDays};
    if (next.contains(key)) {
        next.remove(key);
    } else {
        next.add(key);
    }
    habits[i] = habits[i].copyWith(doneDays: next);
    lastToast = next.contains(key) ? '習慣を記録しました' : '習慣を取り消しました';
    notifyListeners();
    _saveUserData();
}

Habit addHabit({
    required String name,
    required IconData icon,
    required Color color,
}) {
    final habit = Habit(
        id: _id(),
        name: name.trim(),
    );
}

```

```

        icon: icon,
        color: color,
    );
    habits.add(habit);
    lastToast = '習慣「${habit.name}」を追加しました';
    notifyListeners();
    _saveUserData();
    return habit;
}

void startSleep({DateTime? at}) {
    sleepStartedAt = at ?? DateTime.now();
    lastToast = '就寝を記録しました';
    notifyListeners();
    _saveUserData();
}

void cancelSleep() {
    sleepStartedAt = null;
    notifyListeners();
    _saveUserData();
}

SleepLog? wakeUp({int quality = 3, DateTime? at}) {
    final started = sleepStartedAt;
    if (started == null) return null;
    final wake = at ?? DateTime.now();
    sleepStartedAt = null;
    return logSleep(bedAt: started, wakeAt: wake, quality: quality);
}

SleepLog logSleep({
    required DateTime bedAt,
    required DateTime wakeAt,
    int quality = 3,
}) {
    var wake = wakeAt;
    var bed = bedAt;
    if (!wake.isAfter(bed)) {
        wake = bed.add(const Duration(hours: 7));
    }
    sleepLogs.removeWhere((l) => sameDay(l.wakeDate, wake));
    final log = SleepLog(
        id: _id(),
        bedAt: bed,
        wakeAt: wake,
        quality: quality.clamp(1, 5),
    );
    sleepLogs.add(log);
    sleepLogs.sort((a, b) => b.wakeAt.compareTo(a.wakeAt));
    lastToast = '睡眠 ${log.hours.toStringAsFixed(1)}時間を記録しました';
    notifyListeners();
    _saveUserData();
    return log;
}

void setMood(int value) {
    mood = value;
    notifyListeners();
}

void setEnergy(int value) {
    energy = value;
    notifyListeners();
}

void setDiary(String value) {
    diary = value;
    notifyListeners();
}

void startTimer() {
    if (timerRunning) return;
    timerSubjectId ??= nextStudySubjectId;
    timerStartedAt = DateTime.now();
    timerRunning = true;
    notifyListeners();
}

```

```

}

void pauseTimer() {
    if (!timerRunning) return;
    timerAccumulatedSeconds = timerElapsedSeconds();
    timerRunning = false;
    timerStartedAt = null;
    notifyListeners();
}

void finishTimer({StudyFocus focus = StudyFocus.high}) {
    final elapsed = timerElapsedSeconds();
    final sid = timerSubjectId ?? nextStudySubjectId;
    timerRunning = false;
    timerStartedAt = null;
    timerAccumulatedSeconds = 0;
    if (elapsed > 0) {
        addStudySession(
            subjectId: sid,
            minutes: (elapsed / 60).ceil().clamp(1, 24 * 60),
            focus: focus,
            loggedSeconds: elapsed,
        );
    } else {
        notifyListeners();
    }
}

void setTimerMinutes(int minutes) {
    if (timerRunning) return;
    timerTotalSeconds = minutes.clamp(1, 12 * 60) * 60;
    timerAccumulatedSeconds = 0;
    notifyListeners();
}

void setDailyStudyGoalMinutes(int minutes) {
    dailyStudyGoalMinutes = minutes.clamp(10, 12 * 60);
    lastToast = '今日の勉強目標を${studyGoalLabel(dailyStudyGoalMinutes)}にしました';
    notifyListeners();
}

void addStudySession({
    required String subjectId,
    required int minutes,
    required StudyFocus focus,
    DateTime? at,
    int? loggedSeconds,
}) {
    final when = at ?? DateTime.now();
    sessions.insert(
        0,
        StudySession(
            id: _id(),
            subjectId: subjectId,
            minutes: minutes,
            focus: focus,
            at: when,
        ),
    );
    final seconds = loggedSeconds ?? minutes * 60;
    totalStudyHours = double.parse((totalStudyHours + seconds / 3600).toStringAsFixed(1));
    if (sameDay(when, DateTime.now())) {
        _rollTodayStudyIfNeeded();
        todayStudyLoggedSeconds += seconds;
    }
    _refreshWeekBars();
    lastToast = '学習を記録しました';
    notifyListeners();
}

void addAssignment({
    required String subjectId,
    required String title,
    required DateTime dueAt,
}) {
    assignments.add(
        Assignment(id: _id(), subjectId: subjectId, title: title, dueAt: dateOnly(dueAt)),
    );
}

```

```

    );
    assignments.sort((a, b) => a.dueAt.compareTo(b.dueAt));
    lastToast = '提出物を追加しました';
    notifyListeners();
}

void addExam({
  required String title,
  required DateTime examAt,
}) {
  const palette = [
    NexusColors.purple,
    NexusColors.cyan,
    NexusColors.green,
    Color(0xFFFFC857),
  ];
  exams.add(
    Exam(
      id: _id(),
      title: title,
      examAt: dateOnly(examAt),
      color: palette[exams.length % palette.length],
      weekdayLabel: weekdayLabelOf(examAt),
    ),
  );
  exams.sort((a, b) => a.examAt.compareTo(b.examAt));
  lastToast = '試験日を追加しました';
  notifyListeners();
}

void addGoal({
  required String title,
  required int current,
  required int target,
  required DateTime dueAt,
  required List<String> subGoalTitles,
}) {
  final subs = List<SubGoal>.generate(4, (i) {
    final text = i < subGoalTitles.length ? subGoalTitles[i].trim() : '';
    return SubGoal(title: text);
  });
  goals.add(
    StudyGoal(
      id: _id(),
      title: title,
      current: current,
      target: target <= 0 ? 1 : target,
      dueAt: dateOnly(dueAt),
      subGoals: subs,
    ),
  );
  lastToast = '目標を追加しました';
  notifyListeners();
}

void toggleSubGoal(String goalId, int index) {
  final i = goals.indexWhere((g) => g.id == goalId);
  if (i < 0 || index < 0 || index >= goals[i].subGoals.length) return;
  final next = [...goals[i].subGoals];
  next[index] = next[index].copyWith(done: !next[index].done);
  goals[i] = goals[i].copyWith(subGoals: next);
  notifyListeners();
}

ProblemRecord addProblem({
  required String subjectId,
  required String title,
  Uint8List? photoBytes,
}) {
  final learnedAt = focusedDate;
  final problem = ProblemRecord(
    id: _id(),
    subjectId: subjectId,
    title: title,
    learnedAt: learnedAt,
    photoBytes: photoBytes,
  );
}

```

```

problems.insert(0, problem);
reviewCards.addAll(
    ReviewScheduler.cardsFor(
        problemId: problem.id,
        learnedAt: learnedAt,
        nextId: _id,
    ),
);
lastToast = '問題を記録し、1日後と5日後の復習カードを作りました';
notifyListeners();
return problem;
}

void rateReview(String cardId, ReviewRating rating) {
    final i = reviewCards.indexWhere((c) => c.id == cardId);
    if (i < 0) return;
    reviewCards[i] = reviewCards[i].copyWith(
        status: 'done',
        lastRating: rating,
    );
    notifyListeners();
}

void addExpense({required String boxId, required int amount, required String note}) {
    addMoneyCard(
        boxId: boxId,
        title: note.isEmpty ? '支出' : note,
        amount: amount,
        at: focusedDate,
        kind: MoneyCardKind.spend,
    );
}

StudySubject addSubject({required String name}) {
    const palette = [
        NexusColors.purple,
        NexusColors.cyan,
        NexusColors.green,
        Color(0xFF65EDFF),
        Color(0xFFFFC857),
        Color(0xFFFF8AD2),
    ];
    const icons = [
        Icons.menu_book_rounded,
        Icons.science_rounded,
        Icons.history_edu_rounded,
        Icons.biotech_rounded,
        Icons.language_rounded,
    ];
    final subject = StudySubject(
        id: _id(),
        name: name.trim(),
        color: palette[subjects.length % palette.length],
        weekHours: 0,
        icon: icons[subjects.length % icons.length],
    );
    subjects.add(subject);
    lastToast = '教科「${subject.name}」を追加しました';
    notifyListeners();
    _saveUserData();
    return subject;
}

void addIncome({
    required String name,
    required int amount,
    required DateTime depositedAt,
    required int useYear,
    required int useMonth,
    String memo = '',
}) {
    incomes.insert(
        0,
        IncomeEntry(
            id: _id(),
            name: name,
            amount: amount,

```



```

        depositedAt: depositedAt,
        useYear: useYear,
        useMonth: useMonth,
        memo: memo,
    ),
);
lastToast = '収入を追加しました';
notifyListeners();
_saveUserData();
}

BudgetBox addBudgetBox({
  required String name,
  required IconData icon,
  required Color color,
  required int monthlyBudget,
  required int renewalDay,
  required List<String> tags,
  String memo = '',
}) {
  final box = BudgetBox(
    id: _id(),
    name: name,
    kind: BoxKind.budget,
    monthlyBudget: monthlyBudget,
    color: color,
    icon: icon,
    tags: tags,
    renewalDay: renewalDay.clamp(1, 31),
    memo: memo,
  );
  boxes.add(box);
  lastToast = 'ボックス「$name」を追加しました';
  notifyListeners();
  _saveUserData();
  return box;
}

BudgetBox addSavingsBox({
  required String name,
  required IconData icon,
  required Color color,
  required int targetAmount,
  required int openingAmount,
  DateTime? targetDate,
  required List<String> tags,
  String memo = '',
}) {
  final box = BudgetBox(
    id: _id(),
    name: name,
    kind: BoxKind.savings,
    monthlyBudget: 0,
    color: color,
    icon: icon,
    tags: tags,
    targetAmount: targetAmount,
    openingAmount: openingAmount,
    targetDate: targetDate,
    memo: memo,
  );
  boxes.add(box);
  lastToast = '貯蓄ボックス「$name」を追加しました';
  notifyListeners();
  _saveUserData();
  return box;
}

void addBoxTag(String boxId, String tag) {
  final i = boxes.indexWhere((b) => b.id == boxId);
  if (i < 0) return;
  final name = tag.trim();
  if (name.isEmpty || boxes[i].tags.contains(name)) return;
  boxes[i] = boxes[i].copyWith(tags: [...boxes[i].tags, name]);
  notifyListeners();
  _saveUserData();
}

```

```

MoneyCard addMoneyCard({
    required String boxId,
    required String title,
    required int amount,
    required DateTime at,
    String tag = '',
    String memo = '',
    MoneyCardKind kind = MoneyCardKind.spend,
}) {
    final card = MoneyCard(
        id: _id(),
        boxId: boxId,
        title: title,
        amount: amount,
        at: at,
        tag: tag,
        memo: memo,
        kind: kind,
    );
    cards.insert(0, card);
    lastToast = 'カードを追加しました';
    notifyListeners();
    _saveUserData();
    return card;
}

void updateMoneyCard(MoneyCard card) {
    final i = cards.indexWhere((c) => c.id == card.id);
    if (i < 0) return;
    cards[i] = card;
    lastToast = 'カードを更新しました';
    notifyListeners();
    _saveUserData();
}

void deleteBox(String id) {
    if (boxes.every((b) => b.id != id)) return;
    if (boxes.length <= 1) {
        lastToast = 'ボックスは1つ以上残してください';
        notifyListeners();
        return;
    }
    final fallback = boxes.firstWhere(
        (b) => b.id != id && b.id == 'box-unassigned',
        orElse: () => boxes.firstWhere((b) => b.id != id),
    );
    for (var i = 0; i < cards.length; i++) {
        if (cards[i].boxId == id) cards[i] = cards[i].copyWith(boxId: fallback.id);
    }
    for (var i = 0; i < payments.length; i++) {
        if (payments[i].boxId == id) payments[i] = payments[i].copyWith(boxId: fallback.id);
    }
    boxes.removeWhere((b) => b.id == id);
    lastToast = 'ボックスを削除しました';
    notifyListeners();
    _saveUserData();
}

void deletePayment(String id) {
    final before = payments.length;
    payments.removeWhere((p) => p.id == id);
    if (payments.length == before) return;
    lastToast = '支払予定を削除しました';
    notifyListeners();
    _saveUserData();
}

void addPaymentPlan({
    required String title,
    required int amount,
    required DateTime dueAt,
    String? boxId,
    PaymentRepeat repeat = PaymentRepeat.none,
    String memo = '',
}) {
    payments.add(

```

```

        PaymentPlan(
            id: _id(),
            title: title,
            amount: amount,
            dueAt: dateOnly(dueAt),
            boxId: boxId,
            repeat: repeat,
            memo: memo,
        ),
    ),
);
payments.sort((a, b) => a.dueAt.compareTo(b.dueAt));
lastToast = '支払予定を追加しました';
notifyListeners();
_saveUserData();
}

Future<void> hydrate() async {
    final data = await NexusPrefs.load();
    if (data != null) {
        final savedSubjects = [
            for (final item in (data['subjects'] as List? ?? const []))
                StudySubject.fromJson(Map<String, dynamic>.from(item as Map)),
        ];
        if (savedSubjects.isNotEmpty) {
            subjects
                ..clear()
                ..addAll(savedSubjects);
            _refreshWeekBars();
        }
        final savedBoxes = [
            for (final item in (data['boxes'] as List? ?? const []))
                BudgetBox.fromJson(Map<String, dynamic>.from(item as Map)),
        ];
        final savedCards = [
            for (final item in (data['cards'] as List? ?? const []))
                MoneyCard.fromJson(Map<String, dynamic>.from(item as Map)),
        ];
        final savedIncomes = [
            for (final item in (data['incomes'] as List? ?? const []))
                IncomeEntry.fromJson(Map<String, dynamic>.from(item as Map)),
        ];
        final savedPayments = [
            for (final item in (data['payments'] as List? ?? const []))
                PaymentPlan.fromJson(Map<String, dynamic>.from(item as Map)),
        ];
        if (savedBoxes.isNotEmpty) {
            boxes
                ..clear()
                ..addAll(savedBoxes);
        }
        if (savedCards.isNotEmpty || savedBoxes.isNotEmpty) {
            cards
                ..clear()
                ..addAll(savedCards);
        }
        if (savedIncomes.isNotEmpty) {
            incomes
                ..clear()
                ..addAll(savedIncomes);
        }
        if (savedPayments.isNotEmpty) {
            payments
                ..clear()
                ..addAll(savedPayments);
        }
        final savedHabits = [
            for (final item in (data['habits'] as List? ?? const []))
                Habit.fromJson(Map<String, dynamic>.from(item as Map)),
        ];
        if (savedHabits.isNotEmpty) {
            habits
                ..clear()
                ..addAll(savedHabits);
        }
        final savedSleep = [
            for (final item in (data['sleepLogs'] as List? ?? const []))
                SleepLog.fromJson(Map<String, dynamic>.from(item as Map)),
        ];
    }
}

```

```

    };
    if (savedSleep.isNotEmpty) {
        sleepLogs
            ..clear()
            ..addAll(savedSleep);
    }
    final started = data['sleepStartedAt'] as String?;
    sleepStartedAt = started == null || started.isEmpty ? null : DateTime.tryParse(started);
    notifyListeners();
}
_canSave = true;
}

```

```

void _saveUserData() {
    if (!_canSave) return;
    NexusPrefs.save(
        subjects: subjects,
        boxes: boxes,
        cards: cards,
        incomes: incomes,
        payments: payments,
        habits: habits,
        sleepLogs: sleepLogs,
        sleepStartedAt: sleepStartedAt,
    );
}

```

```

void sendUserMessage(String text) {
    messages.add(
        ChatMessage(id: _id(), fromUser: true, text: text, at: DateTime.now()),
    );
    messages.add(
        ChatMessage(
            id: _id(),
            fromUser: false,
            text: _replyFor(text),
            at: DateTime.now(),
        ),
    );
    notifyListeners();
}

```

```

String _replyFor(String text) {
    if (text.contains('学習')) {
        return '数学を30分だけ先に確保するのはどう？プランとして出すネグ。';
    }
    if (text.contains('予定') || text.contains('整理')) {
        return '今日は学校・カフェ・テレビが入ってるネグ。詰まりそうならテレビを短くできるよ。';
    }
    if (text.contains('相談')) {
        return 'いまの予定と学習だけを見て考えるネグ。お金の詳細は許可がないから見てないよ。';
    }
    return '今日の予定、少し詰まってるネグ。数学を30分だけ先にやる?';
}

```

```

void createDefaultProposal() {
    final tv = schedules.cast<ScheduleItem?>().firstWhere(
        (s) => s?.title == 'テレビ',
        orElse: () => null,
    );
    if (tv == null) return;
    proposal = AiProposal(
        id: _id(),
        rationale: '集中しやすい時間を先に確保するため',
        summary: '数学を30分前倒しし、20:00のテレビを30分短縮する',
        scheduleId: tv.id,
        newStartAt: tv.startAt.add(const Duration(minutes: 30)),
    );
    notifyListeners();
}

```

```

bool approveProposal() {
    final p = proposal;
    if (p == null || p.status != ProposalStatus.pending) return false;
    final i = schedules.indexWhere((s) => s.id == p.scheduleId);
    if (i < 0) return false;
    schedules[i] = schedules[i].copyWith(startAt: p.newStartAt);
}

```

```

        proposal = p.copyWith(status: ProposalStatus.approved);
        lastToast = 'プランを反映しました';
        notifyListeners();
        return true;
    }

    void rejectProposal() {
        final p = proposal;
        if (p == null) return;
        proposal = p.copyWith(status: ProposalStatus.rejected);
        lastToast = 'プランは反映していません';
        notifyListeners();
    }

    void setUserName(String value) {
        userName = value;
        notifyListeners();
    }

    void updateSettings(UserSettings next) {
        settings = next;
        notifyListeners();
    }

    void _seed() {
        final today = dateOnly(DateTime.now());
        focusedDate = today;
        lifeDate = today;
        todayStudyDate = today;
        income = 0;
        weekStudyHours = 0;
        totalStudyHours = 0;
        weekBars = List.filled(7, 0);
        nextTaskTitle = '';
        nextTaskDuration = '';
        nextStudySubjectId = '';
        nextStudyPlace = '';
        mood = 0;
        energy = 0;
        steps = 0;
        diary = '';
        proposal = null;
        level = 1;
        levelProgress = 0;
    }
}

class AppScope extends InheritedNotifier<AppStore> {
    const AppScope({
        super.key,
        required AppStore store,
        required super.child,
    }) : super(notifier: store);

    static AppStore of(BuildContext context) {
        final scope = context.dependOnInheritedWidgetOfExactType<AppScope>();
        assert(scope != null, 'AppScope が見つかりません');
        return scope!.notifier!;
    }
}

```

ファイル: lib/data/models.dart

行数: 764

```
import 'dart:typed_data';

import 'package:flutter/material.dart';

import '../core/format.dart';
import 'nexus_icons.dart';

enum StudyFocus {
  low,
  mid,
  high,
  peak;

  String get label => switch (this) {
    low => 'いまいち',
    mid => 'ふつう',
    high => '集中',
    peak => '最高',
  };
}

enum ReviewRating { again, hard, normal, easy }

enum ProposalStatus { draft, pending, approved, rejected }

class ScheduleItem {
  const ScheduleItem({
    required this.id,
    required this.title,
    required this.startAt,
    this.endAt,
    this.allDay = false,
    this.location,
    this.category = 'life',
    this.source = 'user',
  });

  final String id;
  final String title;
  final DateTime startAt;
  final DateTime? endAt;
  final bool allDay;
  final String? location;
  final String category;
  final String source;

  ScheduleItem copyWith({
    String? title,
    DateTime? startAt,
    DateTime? endAt,
    String? location,
  }) {
    return ScheduleItem(
      id: id,
      title: title ?? this.title,
      startAt: startAt ?? this.startAt,
      endAt: endAt ?? this.endAt,
      allDay: allDay,
      location: location ?? this.location,
      category: category,
      source: source,
    );
  }
}

class StudySubject {
  const StudySubject({
    required this.id,
    required this.name,
```

```

        required this.color,
        required this.weekHours,
        required this.icon,
    });

    final String id;
    final String name;
    final Color color;
    final double weekHours;
    final IconData icon;

    StudySubject copyWith({double? weekHours, String? name, Color? color, IconData? icon}) {
        return StudySubject(
            id: id,
            name: name ?? this.name,
            color: color ?? this.color,
            weekHours: weekHours ?? this.weekHours,
            icon: icon ?? this.icon,
        );
    }
}

Map<String, dynamic> toJson() => {
    'id': id,
    'name': name,
    'color': color.toARGB32(),
    'weekHours': weekHours,
    'icon': icon.codePoint,
};

factory StudySubject.fromJson(Map<String, dynamic> json) {
    return StudySubject(
        id: json['id'] as String,
        name: json['name'] as String,
        color: Color(json['color'] as int),
        weekHours: (json['weekHours'] as num).toDouble(),
        icon: nexusIconFromCode(json['icon'] as int),
    );
}

class StudySession {
    const StudySession({
        required this.id,
        required this.subjectId,
        required this.minutes,
        required this.focus,
        required this.at,
    });

    final String id;
    final String subjectId;
    final int minutes;
    final StudyFocus focus;
    final DateTime at;
}

class SubGoal {
    const SubGoal({required this.title, this.done = false});

    final String title;
    final bool done;

    SubGoal copyWith({String? title, bool? done}) {
        return SubGoal(title: title ?? this.title, done: done ?? this.done);
    }
}

class StudyGoal {
    const StudyGoal({
        required this.id,
        required this.title,
        required this.current,
        required this.target,
        required this.dueAt,
        required this.subGoals,
    });
}

```

```

final String id;
final String title;
final int current;
final int target;
final DateTime dueAt;
final List<SubGoal> subGoals;

List<SubGoal> get filledSubGoals =>
    subGoals.where((s) => s.title.trim().isEmpty()).toList();

double get progress {
    final filled = filledSubGoals;
    if (filled.isNotEmpty) {
        return filled.where((s) => s.done).length / filled.length;
    }
    if (target <= 0) return 0;
    return (current / target).clamp(0, 1);
}

StudyGoal copyWith({List<SubGoal>? subGoals, int? current}) {
    return StudyGoal(
        id: id,
        title: title,
        current: current ?? this.current,
        target: target,
        dueAt: dueAt,
        subGoals: subGoals ?? this.subGoals,
    );
}

}

class Assignment {
    const Assignment({
        required this.id,
        required this.subjectId,
        required this.title,
        required this.dueAt,
        this.done = false,
    });

    final String id;
    final String subjectId;
    final String title;
    final DateTime dueAt;
    final bool done;
}

class Exam {
    const Exam({
        required this.id,
        required this.title,
        required this.examAt,
        required this.color,
        required this.weekdayLabel,
    });

    final String id;
    final String title;
    final DateTime examAt;
    final Color color;
    final String weekdayLabel;
}

class LearningEntry {
    const LearningEntry({
        required this.id,
        required this.author,
        required this.subject,
        required this.body,
        required this.visibility,
        required this.learnedAt,
        this.helpful = 0,
    });

    final String id;
    final String author;
    final String subject;

```



```

    final String body;
    final String visibility;
    final DateTime learnedAt;
    final int helpful;
}

class ProblemRecord {
    const ProblemRecord({
        required this.id,
        required this.subjectId,
        required this.title,
        required this.learnedAt,
        this.photoBytes,
    });

    final String id;
    final String subjectId;
    final String title;
    final DateTime learnedAt;
    final Uint8List? photoBytes;
}

class ReviewCard {
    const ReviewCard({
        required this.id,
        required this.problemId,
        required this.dueAt,
        required this.intervalStep,
        this.status = 'pending',
        this.lastRating,
    });

    final String id;
    final String problemId;
    final DateTime dueAt;
    final int intervalStep;
    final String status;
    final ReviewRating? lastRating;

    ReviewCard copyWith({String? status, ReviewRating? lastRating}) {
        return ReviewCard(
            id: id,
            problemId: problemId,
            dueAt: dueAt,
            intervalStep: intervalStep,
            status: status ?? this.status,
            lastRating: lastRating ?? this.lastRating,
        );
    }
}

class Habit {
    const Habit({
        required this.id,
        required this.name,
        required this.icon,
        required this.color,
        this.doneDays = const {},
    });

    final String id;
    final String name;
    final IconData icon;
    final Color color;
    final Set<String> doneDays;

    bool doneOn(DateTime day) => doneDays.contains(dateKey(day));

    int streakEndingOn(DateTime day) {
        if (!doneOn(day)) return 0;
        var count = 0;
        var cursor = dateOnly(day);
        while (doneOn(cursor)) {
            count++;
            cursor = cursor.subtract(const Duration(days: 1));
        }
        return count;
    }
}

```

```

}

int currentStreak(DateTime today) {
  final day = dateOnly(today);
  if (doneOn(day)) return streakEndingOn(day);
  return streakEndingOn(day.subtract(const Duration(days: 1)));
}

Habit copyWith({Set<String>? doneDays}) {
  return Habit(
    id: id,
    name: name,
    icon: icon,
    color: color,
    doneDays: doneDays ?? this.doneDays,
  );
}

Map<String, dynamic> toJson() => {
  'id': id,
  'name': name,
  'icon': icon.codePoint,
  'color': color.toARGB32(),
  'doneDays': doneDays.toList(),
};

factory Habit.fromJson(Map<String, dynamic> json) {
  return Habit(
    id: json['id'] as String,
    name: json['name'] as String,
    icon: nexusIconFromCode(json['icon'] as int? ?? 0),
    color: Color(json['color'] as int? ?? 0xFF00D4FF),
    doneDays: {
      for (final d in (json['doneDays'] as List? ?? const [])) d.toString(),
    },
  );
}

}

class SleepLog {
  const SleepLog({
    required this.id,
    required this.bedAt,
    required this.wakeAt,
    this.quality = 3,
  });

  final String id;
  final DateTime bedAt;
  final DateTime wakeAt;
  final int quality;

  DateTime get wakeDate => dateOnly(wakeAt);

  double get hours {
    final minutes = wakeAt.difference(bedAt).inMinutes;
    if (minutes <= 0) return 0;
    return minutes / 60.0;
  }

  String get qualityLabel => switch (quality) {
    1 => '浅い',
    2 => 'いまいち',
    3 => '普通',
    4 => 'よい',
    _ => 'とてもよい',
  };

  Map<String, dynamic> toJson() => {
    'id': id,
    'bedAt': bedAt.toIso8601String(),
    'wakeAt': wakeAt.toIso8601String(),
    'quality': quality,
  };

  factory SleepLog.fromJson(Map<String, dynamic> json) {
    return SleepLog(

```

```

        id: json['id'] as String,
        bedAt: DateTime.parse(json['bedAt'] as String),
        wakeAt: DateTime.parse(json['wakeAt'] as String),
        quality: json['quality'] as int? ?? 3,
    );
}
}

enum BoxKind { budget, savings }

enum MoneyCardKind { spend, saveIn, saveOut }

enum PaymentRepeat { none, monthly, yearly }

class BudgetBox {
    const BudgetBox({
        required this.id,
        required this.name,
        required this.monthlyBudget,
        required this.color,
        required this.icon,
        this.kind = BoxKind.budget,
        this.spent = 0,
        this.tags = const [],
        this.renewalDay = 1,
        this.memo = '',
        this.targetAmount = 0,
        this.openingAmount = 0,
        this.targetDate,
    });

    final String id;
    final String name;
    final BoxKind kind;
    final int monthlyBudget;
    final int spent;
    final Color color;
    final IconData icon;
    final List<String> tags;
    final int renewalDay;
    final String memo;
    final int targetAmount;
    final int openingAmount;
    final DateTime? targetDate;

    bool get isSavings => kind == BoxKind.savings;

    int get remaining => monthlyBudget - spent;
    double get usedRatio => monthlyBudget == 0 ? 0 : spent / monthlyBudget;
    double get savingsProgress =>
        targetAmount == 0 ? 0 : (openingAmount / targetAmount).clamp(0, 1);

    BudgetBox copyWith({
        String? name,
        int? monthlyBudget,
        int? spent,
        List<String>? tags,
        int? renewalDay,
        String? memo,
        int? targetAmount,
        int? openingAmount,
        DateTime? targetDate,
        IconData? icon,
        Color? color,
    }) {
        return BudgetBox(
            id: id,
            name: name ?? this.name,
            kind: kind,
            monthlyBudget: monthlyBudget ?? this.monthlyBudget,
            spent: spent ?? this.spent,
            color: color ?? this.color,
            icon: icon ?? this.icon,
            tags: tags ?? this.tags,
            renewalDay: renewalDay ?? this.renewalDay,
            memo: memo ?? this.memo,
            targetAmount: targetAmount ?? this.targetAmount,

```

```

        openingAmount: openingAmount ?? this.openingAmount,
        targetDate: targetDate ?? this.targetDate,
    );
}

Map<String, dynamic> toJson() => {
    'id': id,
    'name': name,
    'kind': kind.name,
    'monthlyBudget': monthlyBudget,
    'color': color.toARGB32(),
    'icon': icon.codePoint,
    'tags': tags,
    'renewalDay': renewalDay,
    'memo': memo,
    'targetAmount': targetAmount,
    'openingAmount': openingAmount,
    'targetDate': targetDate?.toIso8601String(),
};

factory BudgetBox.fromJson(Map<String, dynamic> json) {
    return BudgetBox(
        id: json['id'] as String,
        name: json['name'] as String,
        kind: json['kind'] == 'savings' ? BoxKind.savings : BoxKind.budget,
        monthlyBudget: json['monthlyBudget'] as int? ?? 0,
        color: Color(json['color'] as int),
        icon: nexusIconFromCode(json['icon'] as int),
        tags: [for (final t in (json['tags'] as List? ?? const [])) t.toString()],
        renewalDay: json['renewalDay'] as int? ?? 1,
        memo: json['memo'] as String? ?? '',
        targetAmount: json['targetAmount'] as int? ?? 0,
        openingAmount: json['openingAmount'] as int? ?? 0,
        targetDate: json['targetDate'] == null ? null : DateTime.parse(json['targetDate'] as String),
    );
}
}

class MoneyCard {
    const MoneyCard({
        required this.id,
        required this.boxId,
        required this.title,
        required this.amount,
        required this.at,
        this.tag = '',
        this.memo = '',
        this.kind = MoneyCardKind.spend,
    });

    final String id;
    final String boxId;
    final String title;
    final int amount;
    final DateTime at;
    final String tag;
    final String memo;
    final MoneyCardKind kind;

    MoneyCard copyWith({
        String? boxId,
        String? title,
        int? amount,
        DateTime? at,
        String? tag,
        String? memo,
        MoneyCardKind? kind,
    }) {
        return MoneyCard(
            id: id,
            boxId: boxId ?? this.boxId,
            title: title ?? this.title,
            amount: amount ?? this.amount,
            at: at ?? this.at,
            tag: tag ?? this.tag,
            memo: memo ?? this.memo,
            kind: kind ?? this.kind,
        );
    }
}

```

```

    );
}

Map<String, dynamic> toJson() => {
  'id': id,
  'boxId': boxId,
  'title': title,
  'amount': amount,
  'at': at.toIso8601String(),
  'tag': tag,
  'memo': memo,
  'kind': kind.name,
};

factory MoneyCard.fromJson(Map<String, dynamic> json) {
  return MoneyCard(
    id: json['id'] as String,
    boxId: json['boxId'] as String,
    title: json['title'] as String,
    amount: json['amount'] as int,
    at: DateTime.parse(json['at'] as String),
    tag: json['tag'] as String? ?? '',
    memo: json['memo'] as String? ?? '',
    kind: MoneyCardKind.values.firstWhere(
      (k) => k.name == json['kind'],
      orElse: () => MoneyCardKind.spend,
    ),
  );
}
}

class IncomeEntry {
  const IncomeEntry({
    required this.id,
    required this.name,
    required this.amount,
    required this.depositedAt,
    required this.useYear,
    required this.useMonth,
    this.memo = '',
  });

  final String id;
  final String name;
  final int amount;
  final DateTime depositedAt;
  final int useYear;
  final int useMonth;
  final String memo;

  Map<String, dynamic> toJson() => {
    'id': id,
    'name': name,
    'amount': amount,
    'depositedAt': depositedAt.toIso8601String(),
    'useYear': useYear,
    'useMonth': useMonth,
    'memo': memo,
  };

  factory IncomeEntry.fromJson(Map<String, dynamic> json) {
    return IncomeEntry(
      id: json['id'] as String,
      name: json['name'] as String,
      amount: json['amount'] as int,
      depositedAt: DateTime.parse(json['depositedAt'] as String),
      useYear: json['useYear'] as int,
      useMonth: json['useMonth'] as int,
      memo: json['memo'] as String? ?? '',
    );
  }
}

class PaymentPlan {
  const PaymentPlan({
    required this.id,
    required this.title,

```

```

        required this.amount,
        required this.dueAt,
        this.boxId,
        this.repeat = PaymentRepeat.none,
        this.memo = '',
    });

    final String id;
    final String title;
    final int amount;
    final DateTime dueAt;
    final String? boxId;
    final PaymentRepeat repeat;
    final String memo;

    PaymentPlan copyWith({String? boxId, bool clearBoxId = false}) {
        return PaymentPlan(
            id: id,
            title: title,
            amount: amount,
            dueAt: dueAt,
            boxId: clearBoxId ? null : (boxId ?? this.boxId),
            repeat: repeat,
            memo: memo,
        );
    }
}

Map<String, dynamic> toJson() => {
    'id': id,
    'title': title,
    'amount': amount,
    'dueAt': dueAt.toIso8601String(),
    'boxId': boxId,
    'repeat': repeat.name,
    'memo': memo,
};

factory PaymentPlan.fromJson(Map<String, dynamic> json) {
    return PaymentPlan(
        id: json['id'] as String,
        title: json['title'] as String,
        amount: json['amount'] as int,
        dueAt: DateTime.parse(json['dueAt'] as String),
        boxId: json['boxId'] as String?,
        repeat: PaymentRepeat.values.firstWhere(
            (r) => r.name == json['repeat'],
            orElse: () => PaymentRepeat.none,
        ),
        memo: json['memo'] as String? ?? '',
    );
}

class ChatMessage {
    const ChatMessage({
        required this.id,
        required this.fromUser,
        required this.text,
        required this.at,
    });

    final String id;
    final bool fromUser;
    final String text;
    final DateTime at;
}

class AiProposal {
    const AiProposal({
        required this.id,
        required this.rationale,
        required this.summary,
        required this.scheduleId,
        required this.newStartAt,
        this.status = ProposalStatus.pending,
    });
}

```

```

final String id;
final String rationale;
final String summary;
final String scheduleId;
final DateTime newStartAt;
final ProposalStatus status;

AiProposal copyWith({ProposalStatus? status}) {
  return AiProposal(
    id: id,
    rationale: rationale,
    summary: summary,
    scheduleId: scheduleId,
    newStartAt: newStartAt,
    status: status ?? this.status,
  );
}
}

class UserSettings {
  const UserSettings({
    this.notifyTasks = true,
    this.notifySchedule = true,
    this.notifyReview = true,
    this.notifyNegumo = true,
    this.memoryStudy = true,
    this.memorySchedule = true,
    this.memoryMoney = false,
    this.memoryLife = false,
    this.reduceMotion = false,
    this.diaryDefaultPrivate = true,
    this.negumoStrength = 0.5,
    this.proposalFrequency = 0.5,
  });

  final bool notifyTasks;
  final bool notifySchedule;
  final bool notifyReview;
  final bool notifyNegumo;
  final bool memoryStudy;
  final bool memorySchedule;
  final bool memoryMoney;
  final bool memoryLife;
  final bool reduceMotion;
  final bool diaryDefaultPrivate;
  final double negumoStrength;
  final double proposalFrequency;

  UserSettings copyWith({
    bool? notifyTasks,
    bool? notifySchedule,
    bool? notifyReview,
    bool? notifyNegumo,
    bool? memoryStudy,
    bool? memorySchedule,
    bool? memoryMoney,
    bool? memoryLife,
    bool? reduceMotion,
    bool? diaryDefaultPrivate,
    double? negumoStrength,
    double? proposalFrequency,
  }) {
    return UserSettings(
      notifyTasks: notifyTasks ?? this.notifyTasks,
      notifySchedule: notifySchedule ?? this.notifySchedule,
      notifyReview: notifyReview ?? this.notifyReview,
      notifyNegumo: notifyNegumo ?? this.notifyNegumo,
      memoryStudy: memoryStudy ?? this.memoryStudy,
      memorySchedule: memorySchedule ?? this.memorySchedule,
      memoryMoney: memoryMoney ?? this.memoryMoney,
      memoryLife: memoryLife ?? this.memoryLife,
      reduceMotion: reduceMotion ?? this.reduceMotion,
      diaryDefaultPrivate: diaryDefaultPrivate ?? this.diaryDefaultPrivate,
      negumoStrength: negumoStrength ?? this.negumoStrength,
      proposalFrequency: proposalFrequency ?? this.proposalFrequency,
    );
  }
}

```


ファイル: lib/data/nexus_prefs.dart

行数: 51

```
import 'dart:convert';

import 'package:shared_preferences/shared_preferences.dart';

import 'models.dart';

class NexusPrefs {
  NexusPrefs._();

  static const _key = 'nexus_user_data_v2';

  static Future<void> save({
    required List<StudySubject> subjects,
    required List<BudgetBox> boxes,
    required List<MoneyCard> cards,
    required List<IncomeEntry> incomes,
    required List<PaymentPlan> payments,
    required List<Habit> habits,
    required List<SleepLog> sleepLogs,
    DateTime? sleepStartedAt,
  }) async {
    try {
      final prefs = await SharedPreferences.getInstance();
      await prefs.setString(
        _key,
        jsonEncode({
          'subjects': [for (final s in subjects) s.toJson()],
          'boxes': [for (final b in boxes) b.toJson()],
          'cards': [for (final c in cards) c.toJson()],
          'incomes': [for (final i in incomes) i.toJson()],
          'payments': [for (final p in payments) p.toJson()],
          'habits': [for (final h in habits) h.toJson()],
          'sleepLogs': [for (final s in sleepLogs) s.toJson()],
          'sleepStartedAt': sleepStartedAt?.toIso8601String(),
        }),
      );
    } catch (_) {}
  }

  static Future<Map<String, dynamic>?> load() async {
    try {
      final prefs = await SharedPreferences.getInstance();
      final raw = prefs.getString(_key);
      if (raw == null || raw.isEmpty) return null;
      return jsonDecode(raw) as Map<String, dynamic>;
    } catch (_) {
      return null;
    }
  }
}
```

ファイル: lib/data/nexus_icons.dart

行数: 47

```
import 'package:flutter/material.dart';

const kNexusIcons = <IconData>[
  Icons.restaurant_rounded,
  Icons.train_rounded,
  Icons.menu_book_rounded,
  Icons.people_alt_rounded,
  Icons.spa_rounded,
  Icons.shopping_bag_rounded,
  Icons.flight_rounded,
  Icons.savings_rounded,
  Icons.sports_esports_rounded,
  Icons.checkroom_rounded,
  Icons.school_rounded,
  Icons.home_rounded,
  Icons.phone_iphone_rounded,
  Icons.category_rounded,
  Icons.inbox_rounded,
  Icons.functions,
  Icons.code_rounded,
  Icons.calculate_outlined,
  Icons.science_rounded,
  Icons.history_edu_rounded,
  Icons.biotech_rounded,
  Icons.language_rounded,
  Icons.directions_car_rounded,
  Icons.trending_up_rounded,
  Icons.laptop_rounded,
  Icons.celebration_rounded,
  Icons.subscriptions_rounded,
  Icons.wb_sunny_rounded,
  Icons.directions_run_rounded,
  Icons.nightlight_round,
  Icons.bedtime_rounded,
  Icons.water_drop_rounded,
  Icons.self_improvement_rounded,
  Icons.fitness_center_rounded,
  Icons.edit_note_rounded,
];

IconData nexusIconFromCode(int codePoint) {
  for (final icon in kNexusIcons) {
    if (icon.codePoint == codePoint) return icon;
  }
  return Icons.category_rounded;
}
```

ファイル: lib/domain/money_calc.dart

行数: 67

```
import '../data/models.dart';

class MoneySnapshot {
  const MoneySnapshot({
    required this.income,
    required this.expense,
    required this.balance,
    required this.spendableToday,
    required this.todayBudget,
    required this.spendableWeek,
  });

  final int income;
  final int expense;
  final int balance;
  final int spendableToday;
  final int todayBudget;
  final int spendableWeek;
}

class MoneyCalc {
  MoneyCalc._();

  static MoneySnapshot compute({
    required int income,
    required List<BudgetBox> boxes,
    required List<PaymentPlan> payments,
    required DateTime today,
  }) {
    final expense = boxes.fold<int>(0, (sum, b) => sum + b.spent);
    final variableRemaining = boxes.fold<int>(0, (sum, b) => sum + b.remaining);
    final monthEnd = DateTime(today.year, today.month + 1, 0);
    final remainingDays = monthEnd.day - today.day + 1;
    final upcoming = payments
      .where((p) => !p.dueAt.isBefore(DateTime(today.year, today.month, today.day)))
      .where((p) => p.dueAt.month == today.month && p.dueAt.year == today.year)
      .fold<int>(0, (sum, p) => sum + p.amount);

    final pool = (variableRemaining - upcoming).clamp(0, 1 << 31);
    final spendableToday = remainingDays <= 0 ? 0 : (pool / remainingDays).floor();
    final weekDays = remainingDays < 7 ? remainingDays : 7;
    final spendableWeek = weekDays <= 0 ? 0 : spendableToday * weekDays;
    final todayBudget = spendableToday == 0
      ? 0
      : (spendableToday / 0.62).round().clamp(spendableToday, spendableToday * 3);

    return MoneySnapshot(
      income: income,
      expense: expense,
      balance: income - expense,
      spendableToday: spendableToday,
      todayBudget: todayBudget < spendableToday ? spendableToday : todayBudget,
      spendableWeek: spendableWeek,
    );
  }
}

String assignmentRisk({required DateTime dueAt, required DateTime today, required bool done}) {
  if (done) return '余裕あり';
  final days = DateTime(dueAt.year, dueAt.month, dueAt.day)
    .difference(DateTime(today.year, today.month, today.day))
    .inDays;
  if (days <= 2) return '要注意';
  if (days <= 7) return '注意';
  return '余裕あり';
}
```

ファイル: lib/domain/money_catalog.dart

行数: 117

```
import 'package:flutter/material.dart';

import '../app/theme.dart';
import '../data/nexus_icons.dart';

class BoxTemplate {
  const BoxTemplate({
    required this.name,
    required this.icon,
    required this.color,
    required this.tags,
  });

  final String name;
  final IconData icon;
  final Color color;
  final List<String> tags;
}

const budgetBoxTemplates = <BoxTemplate>[
  BoxTemplate(
    name: '食費',
    icon: Icons.restaurant_rounded,
    color: Color(0xFF3DA9FC),
    tags: ['外食', 'コンビニ', 'カフェ', 'スーパー', '昼食', '夕食', '飲み物', 'お菓子', 'その他'],
  ),
  BoxTemplate(
    name: '友達',
    icon: Icons.people_alt_rounded,
    color: Color(0xFFFF8AD2),
    tags: ['ご飯', 'プレゼント', 'イベント', 'その他'],
  ),
  BoxTemplate(
    name: '趣味',
    icon: Icons.sports_esports_rounded,
    color: NexusColors.purple,
    tags: ['本', 'ゲーム', '映画', 'その他'],
  ),
  BoxTemplate(
    name: '美容',
    icon: Icons.spa_rounded,
    color: Color(0xFFFFC857),
    tags: ['化粧品', 'サロン', 'その他'],
  ),
  BoxTemplate(
    name: '交通費',
    icon: Icons.train_rounded,
    color: NexusColors.purple,
    tags: ['電車', 'バス', 'タクシー', 'その他'],
  ),
  BoxTemplate(
    name: '日用品',
    icon: Icons.shopping_bag_rounded,
    color: Color(0xFF65EDFF),
    tags: ['ドラッグストア', 'スーパー', 'その他'],
  ),
  BoxTemplate(
    name: 'サブスク',
    icon: Icons.subscriptions_rounded,
    color: Color(0xFF2EE6C7),
    tags: ['動画', '音楽', 'ツール', 'その他'],
  ),
  BoxTemplate(
    name: '服',
    icon: Icons.checkroom_rounded,
    color: Color(0xFF9B6BFF),
    tags: ['服', '靴', 'その他'],
  ),
  BoxTemplate(
```

```

        name: '娯楽',
        icon: Icons.celebration_rounded,
        color: Color(0xFFFFF5B7A),
        tags: ['外出', 'イベント', 'その他'],
    ),
    BoxTemplate(
        name: '学費',
        icon: Icons.school_rounded,
        color: NexusColors.cyan,
        tags: ['授業料', '教材', 'その他'],
    ),
    BoxTemplate(
        name: 'その他',
        icon: Icons.category_rounded,
        color: NexusColors.textSecondary,
        tags: ['その他'],
    ),
];

const savingsBoxTemplates = <BoxTemplate>[
    BoxTemplate(name: '車の免許', icon: Icons.directions_car_rounded, color: NexusColors.cyan, tags: ['教習', '検定', 'その他']),
    BoxTemplate(name: '韓国旅行', icon: Icons.flight_rounded, color: Color(0xFFFFF8AD2), tags: ['ホテル', '飛行機', '食事', 'お土産', '現地交通']),
    BoxTemplate(name: '投資', icon: Icons.trending_up_rounded, color: NexusColors.green, tags: ['入金', '出金']),
    BoxTemplate(name: '学費', icon: Icons.school_rounded, color: NexusColors.purple, tags: ['積立', 'その他']),
    BoxTemplate(name: 'PC', icon: Icons.laptop_rounded, color: Color(0xFF65EDFF), tags: ['本体', '周辺機器']),
    BoxTemplate(name: '将来資金', icon: Icons.savings_rounded, color: Color(0xFFFFFC857), tags: ['積立', 'その他']),
];

const boxIconChoices = kNexusIcons;

const boxColorChoices = <Color>[
    Color(0xFF3DA9FC),
    NexusColors.purple,
    Color(0xFF2EE6C7),
    NexusColors.green,
    Color(0xFFFFF5B7A),
    Color(0xFFFFFC857),
    Color(0xFFFFF8AD2),
    Color(0xFF65EDFF),
];

List<String> suggestedTagsFor(String name) {
    for (final t in [...budgetBoxTemplates, ...savingsBoxTemplates]) {
        if (name.contains(t.name) || t.name.contains(name)) return List<String>.from(t.tags);
    }
    return ['その他'];
}

```

ファイル: `lib/domain/review_scheduler.dart`

行数: 24

```
import '../data/models.dart';

class ReviewScheduler {
  ReviewScheduler._();

  static const intervals = [1, 5];

  static List<ReviewCard> cardsFor({
    required String problemId,
    required DateTime learnedAt,
    required String Function() nextId,
  }) {
    return [
      for (final day in intervals)
        ReviewCard(
          id: nextId(),
          problemId: problemId,
          dueAt: DateTime(learnedAt.year, learnedAt.month, learnedAt.day + day),
          intervalStep: day,
        ),
    ];
  }
}
```

ファイル: lib/domain/daily_quotes.dart

行数: 448

```
class DailyQuote {
  const DailyQuote({
    required this.text,
    required this.author,
    required this.note,
  });

  final String text;
  final String author;
  final String note;
}

DailyQuote quoteFor(DateTime date) {
  final index = DateTime(date.year, date.month, date.day)
    .difference(DateTime(date.year))
    .inDays;
  return dailyQuotes[index % dailyQuotes.length];
}

/// 実在の人物が残した言葉。日付の通し番号で毎日入れ替える。
const dailyQuotes = <DailyQuote>[
  DailyQuote(
    text: '学びて時にこれを習う、また説ばしからずや。',
    author: '孔子',
    note: '中国の思想家。『論語』学而篇の一節で、学んだことを折に触れて復習する喜びを説いた。',
  ),
  DailyQuote(
    text: '千里の道も一歩から。',
    author: '老子',
    note: '中国の思想家。『道德経』の一節で、大きなことも最初の一步から始まるという意味。',
  ),
  DailyQuote(
    text: '己の欲せざる所は、人に施すことなかれ。',
    author: '孔子',
    note: '『論語』衛霊公篇。自分がされたくないことを、他人にしてはならないという教え。',
  ),
  DailyQuote(
    text: '知らざるを知らずとなす、これ知るなり。',
    author: '孔子',
    note: '知らないことを知らないと認めることこそ、真の知であるという『論語』の言葉。',
  ),
  DailyQuote(
    text: '我思う、ゆえに我あり。',
    author: 'ルネ・デカルト',
    note: 'フランスの哲学者。すべてを疑っても、疑っている自分の存在だけは疑い得ない、と述べた。',
  ),
  DailyQuote(
    text: '人間は考える葦である。',
    author: 'ブレーズ・パスカル',
    note: 'フランスの数学者・哲学者。『パンセ』で、人間は弱いが思考によって尊厳を持つと書いた。',
  ),
  DailyQuote(
    text: '運命は勇敢な者を助ける。',
    author: 'ウェルギリウス',
    note: '古代ローマの詩人。『アエネイス』に残る一節で、勇気ある行動が道を開くことを示す。',
  ),
  DailyQuote(
    text: '知識よりも想像力のほうが大切だ。',
    author: 'アルベルト・アインシュタイン',
    note: '物理学者。知識には限界があるが、想像力は世界を広く捉える、と語った。',
  ),
  DailyQuote(
    text: '天才とは、1パーセントのひらめきと99パーセントの努力である。',
    author: 'トーマス・エジソン',
    note: '発明家。成功の大半は地道な実験と継続にある、という意味で繰り返し語った。',
  ),
  DailyQuote(
    text: '失敗したときこそ、ほんとうの出発点だ。',
    author: '松下幸之助',
  ),
];
```

```
    note: 'パナソニック創業者。失敗を責めるより、そこから学び直すことを重視した。',
  ),
  DailyQuote(
    text: '天は人の上に人を造らず、人の下に人を造らず。',
    author: '福沢諭吉',
    note: '『学問のすゝめ』の冒頭。人は生まれながら平等で、学ぶことで差がつくと説いた。',
  ),
  DailyQuote(
    text: '夢なき者に成功なし。',
    author: '吉田松陰',
    note: '幕末の思想家。志を持つことが行動の前提だと、松下村塾で繰り返し教えた。',
  ),
  DailyQuote(
    text: '努力だ、勉強だ。それが天才だ。',
    author: '野口英世',
    note: '細菌学者。才能より継続した努力こそが成果を生む、と自らの人生を振り返って述べた。',
  ),
  DailyQuote(
    text: '雨ニモマケズ、風ニモマケズ。',
    author: '宮沢賢治',
    note: '詩人・童話作家。手帳に残した詩で、目立たず人の役に立つ生き方を願った。',
  ),
  DailyQuote(
    text: 'いま、なすべきことをせよ。',
    author: '新渡戸稲造',
    note: '教育者・国際人。『武士道』の著者として、その日の務めを果たすことを重んじた。',
  ),
  DailyQuote(
    text: '成功は、失敗の回数に比例する。',
    author: '本田宗一郎',
    note: '本田技研工業の創業者。挑戦の回数だけ学びが増える、という信念を語った。',
  ),
  DailyQuote(
    text: '自分に忠実であれ。',
    author: 'ウィリアム・シェイクスピア',
    note: '『ハムレット』のポロニアスの台詞。他人より先に、自分の良心に正直であれという意味。',
  ),
  DailyQuote(
    text: '学ぶことに終わりはない。',
    author: 'レオナルド・ダ・ヴィンチ',
    note: 'ルネサンスの芸術家・科学者。晩年まで観察と記録をやめなかった人物の言葉。',
  ),
  DailyQuote(
    text: 'まだ学んでいる。',
    author: 'ミケランジェロ',
    note: '彫刻家・画家。87歳近くになっても「まだ学んでいる」と語ったと伝えられる。',
  ),
  DailyQuote(
    text: '人生に恐れるものはない。理解すべきものがあるだけだ。',
    author: 'マリ・キュリー',
    note: '物理学者・化学者。未知を恐れず調べることが科学の態度だと述べた。',
  ),
  DailyQuote(
    text: '明日死ぬかのように生きよ。永遠に生きるかのように学べ。',
    author: 'マハトマ・ガンディー',
    note: 'インド独立運動の指導者。今日を大切にし、学びは生涯続けるべきだと説いた。',
  ),
  DailyQuote(
    text: '暗闇を呪うより、一本のろうそくを灯せ。',
    author: 'マーティン・ルーサー・キング・ジュニア',
    note: '公民権運動の指導者。不満より、小さな実行が状況を変えると語った。',
  ),
  DailyQuote(
    text: '何事も、成功するまでは不可能に見える。',
    author: 'ネルソン・マンデラ',
    note: '南アフリカの元大統領。長い投獄のあと、不可能に見えた変化を実現した。',
  ),
  DailyQuote(
    text: '希望を失ったとき、すべてを失う。',
    author: 'ヘレン・ケラー',
    note: '視覚と聴覚を失いながら著述と講演を続けた。希望が行動を支えると書いた。',
  ),
  DailyQuote(
    text: 'ハングリーであれ。愚か者であれ。',
    author: 'スティーブ・ジョブズ',
    note: 'Apple創業者。2005年のスタンフォード大学卒業式で、満足せず学び続けよと述べた。',
  ),
```



```
DailyQuote(  
  text: '大切なものは、目に見えない。',  
  author: 'アントワーン・ド・サン＝テグジュペリ',  
  note: '『星の王子さま』の狐の言葉。本質は数字や外見より、関係のなかにあると書いた。',  
)  
DailyQuote(  
  text: 'それでも、人は善良だと信じていたい。',  
  author: 'アンネ・フランク',  
  note: '日記を残した少女。迫害のなかでも、人間の善良さを信じたいと記した。',  
)  
DailyQuote(  
  text: '時は金なり。',  
  author: 'ベンジャミン・フランクリン',  
  note: 'アメリカの政治家・科学者。時間を浪費せず、仕事と学びに使えと勧めた。',  
)  
DailyQuote(  
  text: 'ほとんどすべての人は、決意しただけ幸福になれる。',  
  author: 'エイブラハム・リンカーン',  
  note: 'アメリカ第16代大統領。境遇より、心の持ちようが幸福を左右すると述べた。',  
)  
DailyQuote(  
  text: '不可能という文字は、愚者の辞書にしかない。',  
  author: 'ナポレオン・ボナパルト',  
  note: 'フランス皇帝。困難を最初から不可能と決めつけるな、という趣旨で伝えられる。',  
)  
DailyQuote(  
  text: '我々は、繰り返し行うことの産物である。優秀さは行為ではなく習慣である。',  
  author: 'アリストテレス',  
  note: '古代ギリシアの哲学者。徳は一度の行動ではなく、日々の習慣で身につくと説いた。',  
)  
DailyQuote(  
  text: '朝起きたら、今日出会う人のことを思いなさい。',  
  author: 'マルクス・アウレリウス',  
  note: 'ローマ皇帝。『自省録』で、人の欠点に怒らず自分の態度を整えよと書いた。',  
)  
DailyQuote(  
  text: '汝自身を知れ。',  
  author: 'ソクラテス',  
  note: '古代ギリシアの哲学者。デルフォイの神託に連なる言葉として、自己理解を重んじた。',  
)  
DailyQuote(  
  text: '敢えて知れ。',  
  author: 'イマヌエル・カント',  
  note: 'ドイツの哲学者。啓蒙とは、自分の理性を使う勇気を持つことだと述べた。',  
)  
DailyQuote(  
  text: '今日という日は、残りの人生の最初の日である。',  
  author: 'ヨハン・ヴォルフガング・フォン・ゲーテ',  
  note: 'ドイツの詩人。過ぎた日より、いま始まる一日を大切にせよという趣旨の言葉。',  
)  
DailyQuote(  
  text: '我を殺さぬものは、我を強くする。',  
  author: 'フリードリヒ・ニーチェ',  
  note: 'ドイツの哲学者。『偶像の黄昏』などで、苦難が力に変わることを述べた。',  
)  
DailyQuote(  
  text: '幸福とは、求めるものではなく与えるものである。',  
  author: 'レフ・トルストイ',  
  note: 'ロシアの文豪。幸福は所有より、他者への働きかけのなかにあると書いた。',  
)  
DailyQuote(  
  text: '美は世界を救う。',  
  author: 'フョードル・ドストエフスキー',  
  note: '『白痴』に登場する言葉。真実と美が、荒れた世界を立て直す力になると書いた。',  
)  
DailyQuote(  
  text: '巨人の肩の上に乗っている。',  
  author: 'アイザック・ニュートン',  
  note: '物理学者。自分の発見は先人の学問の上にある、と謙虚に述べた手紙の一節。',  
)  
DailyQuote(  
  text: 'それでも地球は動く。',  
  author: 'ガリレオ・ガリレイ',  
  note: '天文学者。地動説を撤回させられたあと、真実は変わらないという意味で伝えられる。',  
)  
DailyQuote(  
  text: '測定できるものは測定し、測定できないものは測定できるようにせよ。',
```

```
    author: 'ガリレオ・ガリレイ',
    note: '自然を数学で捉える姿勢を示した言葉として、科学の方法を象徴する。',
  ),
  DailyQuote(
    text: '始めることが、仕事の半ばである。',
    author: 'ホラティウス',
    note: '古代ローマの詩人。着手しない限り進まない、という実務的な助言。',
  ),
  DailyQuote(
    text: '計画のない目標は、ただの願い事にすぎない。',
    author: 'アントワーヌ・ド・サン＝テグジュペリ',
    note: '目標は気持ちだけでなく、手順に落とさなければ実現しない、という趣旨で引用される。',
  ),
  DailyQuote(
    text: '明日をつくるために、今日を捨てよ。',
    author: 'ピーター・ドラッカー',
    note: '経営学者。成果を出すには、過去のやり方を見直し続ける必要があると説いた。',
  ),
  DailyQuote(
    text: '心を高めることが、人生を伸ばす。',
    author: '稻盛和夫',
    note: '京セラ創業者。技術や戦略より先に、日々の心のあり方を整えることを重んじた。',
  ),
  DailyQuote(
    text: '智に働けば角が立つ。情に棹させば流される。',
    author: '夏目漱石',
    note: '『草枕』の冒頭。理屈だけでは生きにくく、感情だけでも流される、という人間観察。',
  ),
  DailyQuote(
    text: '天災は忘れた頃にやってくる。',
    author: '寺田寅彦',
    note: '物理学者・随筆家。災害への備えを怠るな、という警告として広く知られる。',
  ),
  DailyQuote(
    text: '世の中は、自分の思いどおりにならない。それでよい。',
    author: '勝海舟',
    note: '幕末の政治家。自分の都合より、現実を見て動くことを重んじた。',
  ),
  DailyQuote(
    text: '一日一生。',
    author: '吉田兼好',
    note: '『徒然草』の精神を短く言い表した言葉として、今日を疎かにするなという意味で用いられる。',
  ),
  DailyQuote(
    text: '石の上にも三年。忍耐は学びの母である。',
    author: '貝原益軒',
    note: '江戸の儒者・本草学者。学問は性急に成果を求めず、続けるべきだと勧めた。',
  ),
  DailyQuote(
    text: '読書は、過去の最も優れた人々との会話である。',
    author: 'ルネ・デカルト',
    note: '本を読むことは、時代を超えて先人と話することだ、と学問の意味を述べた。',
  ),
  DailyQuote(
    text: '教育とは、学校で習ったことをすべて忘れたあとに残るものである。',
    author: 'アルベルト・アインシュタイン',
    note: '暗記より、考え方や好奇心が残ることが学びの本質だ、という意味で語った。',
  ),
  DailyQuote(
    text: '小さなことを積み重ねることが、とんでもないところへ行くただひとつの方法だ。',
    author: 'イチロー',
    note: '野球選手。毎日の準備と反復が、大きな結果につながるという持論。',
  ),
  DailyQuote(
    text: '継続は力なり。毎日の積み重ねが、やがて大きな力になる。',
    author: '中村天風',
    note: '実業家・思想家。心身統一法を説き、日々の継続こそが力になると指導した。',
  ),
  DailyQuote(
    text: '為せば成る、為さねば成らぬ何事も。',
    author: '上杉鷹山',
    note: '米沢藩主。改革を進めるなかで、やらなければ何も始まらないと家臣に示した。',
  ),
  DailyQuote(
    text: '人、学びて後に足らざるを知る。',
    author: '荀子',
    note: '中国の思想家。学んではじめて自分の不足に気づく、と学習の意味を述べた。',
```

```
),
DailyQuote(
    text: '少年よ、大志を抱け。',
    author: 'ウィリアム・スミス・クラーク',
    note: '札幌農学校教頭。1877年の別れの言葉として日本に残った。',
),
DailyQuote(
    text: '世の中に満てる宝は、学問より貴きはなし。',
    author: '福沢諭吉',
    note: '学問こそが独立と実利を支える、という『学問のすゝめ』の主張。',
),
DailyQuote(
    text: '観察せよ、観察せよ、観察せよ。',
    author: 'シャルコー（ジャン＝マルタン・シャルコー）',
    note: 'フランスの神経学者。事実をよく見ることから研究が始まる、と弟子に教えた。',
),
DailyQuote(
    text: '真実は、必ず簡単なほうにある。',
    author: 'アイザック・ニュートン',
    note: '自然は無駄のない法則に従う、という考えを簡潔に述べた言葉として伝わる。',
),
DailyQuote(
    text: '与えられた才能を、眠らせてはならない。',
    author: 'レオナルド・ダ・ヴィンチ',
    note: '能力は使わなければ衰える。手と目を働かせ続けよ、という制作態度。',
),
DailyQuote(
    text: '一日の計は朝にあり。',
    author: '朱用純（朱子家訓の系譜）',
    note: '中国の家訓に由来し、日本でも朝の計画が一日を決める、という意味で広まった。',
),
DailyQuote(
    text: '健康は、最上の富である。',
    author: 'ラルフ・ワルド・エマーソン',
    note: 'アメリカの思想家。財産より、体と心が整っていることが土台だと述べた。',
),
DailyQuote(
    text: 'ゆっくり急げ。',
    author: 'アウグストゥス',
    note: 'ローマ皇帝の座右の銘「Festina lente」。急ぐほど、手順を丁寧にせよという意味。',
),
DailyQuote(
    text: '自己本位という言葉や、自分の手に入れるまでには大変な時間がかかった。',
    author: '夏目漱石',
    note: '講演『私の個人主義』。他人の評価ではなく、自分の立ち位置を持つことの難しさを語った。',
),
DailyQuote(
    text: '困難は、分割せよ。',
    author: 'ルネ・デカルト',
    note: '『方法序説』の規則。大きな問題は、解ける大きさに分けて考えるべきだとした。',
),
DailyQuote(
    text: '最も強い者が生き残るのではない。変化できる者が生き残る。',
    author: 'チャールズ・ダーウィン',
    note: '進化論の趣旨を短く言い表した言葉として広く引用される。環境に合わせて学び直せ、という意味。',
),
DailyQuote(
    text: '人は、自分が考えているような者になる。',
    author: 'マルクス・アウレリウス',
    note: '『自省録』。心の使い方が、その人の人生を形づくるというストア派の考え。',
),
DailyQuote(
    text: '今日できることを、明日に延ばすな。',
    author: 'ベンジャミン・フランクリン',
    note: '13の徳目のひとつ。先延ばしを戒め、その日のうちに片づけることを勧めた。',
),
DailyQuote(
    text: '学問に王道なし。',
    author: 'エウクレイデス（ユークリッド）',
    note: '古代の数学者。王に幾何を教えるとき、近道はないと答えたと伝えられる。',
),
DailyQuote(
    text: '心が変われば、行動が変わる。',
    author: 'ウィリアム・ジェームズ',
    note: 'アメリカの心理学者。習慣と注意の向け方が、人生を変えると述べた。',
),
DailyQuote(
```

```
    text: '同じ過ちを繰り返す者を、愚者と呼ぶ。',
    author: 'キケロ',
    note: 'ローマの弁論家。失敗から学ばないこそが真の失敗だ、という指摘。',
  ),
  DailyQuote(
    text: '平和は、戦争の不在ではなく、正義のあるところに宿る。',
    author: 'マルティン・ルター・キング・ジュニア',
    note: '単なる沈黙ではなく、公正さが伴ってこそ平和だ、と講演で繰り返し述べた。',
  ),
  DailyQuote(
    text: '人を動かすには、まず自分が動け。',
    author: '松下幸之助',
    note: '号令より率先垂範。自分の姿勢が周囲を変える、という経営者としての信条。',
  ),
  DailyQuote(
    text: '世界を変えたいなら、まず自分から。',
    author: 'マハトマ・ガンディー',
    note: '「自ら望む変化に、あなた自身がなれ」という趣旨で広く知られる。',
  ),
  DailyQuote(
    text: '記録せよ。記憶は消える。',
    author: 'レオナルド・ダ・ヴィンチ',
    note: '膨大な手稿を残した。見たこと・考えたことは書き残せ、という実践そのもの。',
  ),
  DailyQuote(
    text: '問うことは、恥ではない。知らぬままにするのが恥である。',
    author: '孔子',
    note: '『論語』に通じる学習態度。わからないことを放置せず、尋ねて明らかにせよという意味。',
  ),
  DailyQuote(
    text: '単純さは、究極の洗練である。',
    author: 'レオナルド・ダ・ヴィンチ',
    note: '無駄を削った表現が最も高い、という制作上の信念として引用される。',
  ),
  DailyQuote(
    text: '休息は、怠惰ではなく、次の集中のための準備である。',
    author: 'アリストテレス',
    note: '適度な休みは徳のある生活に必要なだ、という中庸の考えに基づく。',
  ),
  DailyQuote(
    text: '自分の仕事を愛せ。愛する仕事を選べ。',
    author: '孔子',
    note: '『論語』に「これを知る者はこれを好む者に如かず」とあり、好んで学ぶ者が強いと説く。',
  ),
  DailyQuote(
    text: '今日の自分は、昨日の選択の結果である。',
    author: 'ラルフ・ワルド・エマーソン',
    note: '毎日の小さな選択が性格と運命をつくる、という自己信頼の思想。',
  ),
  DailyQuote(
    text: '早く行きたければ一人で行け。遠くまで行きたければみんなで行け。',
    author: 'アフリカの諺（マンデラらも引用）',
    note: 'ネルソン・マンデラなどが協力の大切さを語る際に引いたことわざ。',
  ),
  DailyQuote(
    text: '精神は、使うことで磨かれる。',
    author: 'マルクス・トゥッリウス・キケロ',
    note: 'ローマの哲学者。頭は使わなければ鈍る、学び続けよという主張。',
  ),
  DailyQuote(
    text: 'いちばんの近道は、正しい道を行くことだ。',
    author: 'セネカ',
    note: 'ストア派の哲学者。楽な回り道より、正しい手順のほうが結局早いと述べた。',
  ),
  DailyQuote(
    text: '希望は、眠っている者の夢ではない。起きて働く者の光である。',
    author: 'アリストテレス',
    note: '希望は空想ではなく、善い行為と結びついて意味を持つ、という実践の哲学。',
  ),
  ],
```

ファイル: lib/domain/day_occasions.dart

行数: 1653

```
class DayOccasion {
  const DayOccasion({
    required this.name,
    required this.reason,
    this.kind = '記念日',
    this.alsoKnownAs = const [],
  });

  final String name;
  final String reason;
  final String kind;
  final List<String> alsoKnownAs;
}

DayOccasion occasionFor(DateTime date) {
  final day = DateTime(date.year, date.month, date.day);
  final holiday = japanHoliday(day);
  if (holiday != null) return holiday;

  final fixed = kinenbiByMonthDay['${day.month}-${day.day}'];
  if (fixed != null) return fixed;

  return DayOccasion(
    name: '${day.month}月${day.day}日',
    reason: 'この日に広く定着した記念日は少ないですが、暦のうえでは一年のうち一度きりの日です。',
  );
}

DayOccasion? japanHoliday(DateTime date) {
  final holidays = japanHolidays(date.year);
  return holidays[DateTime(date.year, date.month, date.day)];
}

Map<DateTime, DayOccasion> japanHolidays(int year) {
  final raw = <DateTime, DayOccasion>{
    DateTime(year, 1, 1): const DayOccasion(
      name: '元日',
      kind: '国民の祝日',
      reason: '年の始めを祝う国民の祝日。明治時代に「四方拜」などを経て、1948年の祝日法で「年のはじめを祝う」日と定められた。',
    ),
    _nthWeekday(year, 1, DateTime.monday, 2): const DayOccasion(
      name: '成人の日',
      kind: '国民の祝日',
      reason: 'おとなになったことを自覚し、みずから生き抜こうとする青年を祝い抬げます日。かつては1月15日で、2000年から1月第2月曜日になった。',
    ),
    DateTime(year, 2, 11): const DayOccasion(
      name: '建国記念の日',
      kind: '国民の祝日',
      reason: '建国をしのび、国を愛する心を養う日。神武天皇即位の伝承日（旧暦1月1日＝新暦2月11日）に由来し、1966年に祝日となった。',
    ),
    DateTime(year, 2, 23): const DayOccasion(
      name: '天皇誕生日',
      kind: '国民の祝日',
      reason: '天皇の誕生日を祝う国民の祝日。2019年の即位に伴い、徳仁天皇の誕生日である2月23日に移った。',
    ),
    DateTime(year, 3, _springEquinoxDay(year)): const DayOccasion(
      name: '春分の日',
      kind: '国民の祝日',
      reason: '自然をたたえ、生物をいつくしむ日。天文学上の春分を基準に、国立天文台の計算で毎年の日付が決まる。',
    ),
    DateTime(year, 4, 29): const DayOccasion(
      name: '昭和の日',
      kind: '国民の祝日',
      reason: '激動の日々を経て、復興を遂げた昭和の時代を顧み、国の将来に思いをいたす日。昭和天皇の誕生日だった4月29日を引き継ぐ。',
    ),
    DateTime(year, 5, 3): const DayOccasion(
      name: '憲法記念日',
      kind: '国民の祝日',
```

```

        reason: '日本国憲法の施行を記念し、国の成長を期する日。憲法が施行された1947年5月3日に由来する。',
    ),
    DateTime(year, 5, 4): const DayOccasion(
        name: 'みどりの日',
        kind: '国民の祝日',
        reason: '自然に親しむとともに、その恩恵に感謝し、豊かな心をはぐくむ日。かつての「国民の休日」が2007年から祝日になった。',
    ),
    DateTime(year, 5, 5): const DayOccasion(
        name: 'こどもの日',
        kind: '国民の祝日',
        reason: 'こどもの人格を重んじ、こどもの幸福をはかるとともに、母に感謝する日。端午の節句（5月5日）を引き継いで1948年に祝日となった。',
    ),
    _nthWeekday(year, 7, DateTime.monday, 3): const DayOccasion(
        name: '海の日',
        kind: '国民の祝日',
        reason: '海の恩恵に感謝し、海洋国家日本の繁栄を願う日。明治天皇の巡幸が横浜へ帰着した7月20日が起源で、のちに7月第3月曜日になった。',
    ),
    DateTime(year, 8, 11): const DayOccasion(
        name: '山の日',
        kind: '国民の祝日',
        reason: '山に親しむ機会を得て、山の恩恵に感謝する日。2016年に新設された比較的新しい国民の祝日。',
    ),
    _nthWeekday(year, 9, DateTime.monday, 3): const DayOccasion(
        name: '敬老の日',
        kind: '国民の祝日',
        reason: '多年にわたり社会につくしてきた老人を敬愛し、長寿を祝う日。もとは9月15日で、2003年から9月第3月曜日になった。',
    ),
    DateTime(year, 9, _autumnEquinoxDay(year)): const DayOccasion(
        name: '秋分の日',
        kind: '国民の祝日',
        reason: '祖先をうやまい、なくなった人々をしのぶ日。春分と同じく、天文学上の秋分から毎年の日付が決まる。',
    ),
    _nthWeekday(year, 10, DateTime.monday, 2): const DayOccasion(
        name: 'スポーツの日',
        kind: '国民の祝日',
        reason: 'スポーツに楽しみ、健康な心身を培う日。1964年東京オリンピック開会日の10月10日が起源で、のちに10月第2月曜日になった。',
    ),
    DateTime(year, 11, 3): const DayOccasion(
        name: '文化の日',
        kind: '国民の祝日',
        reason: '自由と平和を愛し、文化をすすめる日。1946年11月3日に日本国憲法が公布されたことを記念する。',
    ),
    DateTime(year, 11, 23): const DayOccasion(
        name: '勤労感謝の日',
        kind: '国民の祝日',
        reason: '勤労をたつとび、生産を祝い、国民がたがいに感謝しあう日。古代の新嘗祭（にいなめさい）を起源とする。',
    ),
};

final mapped = <DateTime, DayOccasion>{};
for (final e in raw.entries) {
    mapped[DateTime(e.key.year, e.key.month, e.key.day)] = e.value;
}

final extra = <DateTime, DayOccasion>{};
for (final day in mapped.keys.toList()..sort()) {
    if (day.weekday == DateTime.sunday) {
        var substitute = day.add(const Duration(days: 1));
        while (mapped.containsKey(substitute) || extra.containsKey(substitute)) {
            substitute = substitute.add(const Duration(days: 1));
        }
        extra[substitute] = const DayOccasion(
            name: '振替休日',
            kind: '国民の祝日',
            reason: '祝日が日曜日と重なったとき、直後の平日を休みにする制度。国民が祝日を祝う機会を失わないように設けられた。',
        );
    }
}
mapped.addAll(extra);

final sandwiched = <DateTime, DayOccasion>{};
for (var month = 1; month <= 12; month++) {
    final days = DateTime(year, month + 1, 0).day;
    for (var d = 1; d <= days; d++) {
        final mid = DateTime(year, month, d);
        final before = mid.subtract(const Duration(days: 1));
        final after = mid.add(const Duration(days: 1));
    }
}

```

```

        if (!mapped.containsKey(mid) &&
            mapped.containsKey(before) &&
            mapped.containsKey(after) &&
            mid.weekday != DateTime.sunday) {
            sandwiched[mid] = const DayOccasion(
                name: '国民の休日',
                kind: '国民の祝日',
                reason: '祝日と祝日に挟まれた日を休日にする制度。連休を生かし、国民が休息をとりやすくするために設けられた。',
            );
        }
    }
}
mapped.addAll(sandwiched);
return mapped;
}

DateTime _nthWeekday(int year, int month, int weekday, int n) {
    final first = DateTime(year, month, 1);
    final offset = (weekday - first.weekday + 7) % 7;
    return DateTime(year, month, 1 + offset + (n - 1) * 7);
}

int _springEquinoxDay(int year) {
    return (20.8431 + 0.242194 * (year - 1980) - ((year - 1980) / 4).floor()).floor();
}

int _autumnEquinoxDay(int year) {
    return (23.2488 + 0.242194 * (year - 1980) - ((year - 1980) / 4).floor()).floor();
}

const kinenbiByMonthDay = <String, DayOccasion>{
    '1-1': DayOccasion(
        name: '元日',
        reason: '年の始めを祝う日。祝日法でも「年のはじめを祝う」と定められている。',
    ),
    '1-2': DayOccasion(
        name: '初夢の日',
        reason: '元日から2日にかけての夢を「初夢」として縁起を見る風習がある。一富士二鷹三なすび、などの語呂もここから広まった。',
    ),
    '1-3': DayOccasion(
        name: 'ひとみの日',
        reason: '1と3で「ひとみ」と読む語呂合わせ。目の健康を考える日として眼科関係の団体が広めた。',
    ),
    '1-4': DayOccasion(
        name: '石の日',
        reason: '1と4で「いし」と読む語呂合わせ。石材や宝石など、石に親しむ日として業界が制定した。',
    ),
    '1-5': DayOccasion(
        name: 'いちごの日',
        reason: '1と5で「いちご」と読む語呂合わせ。全国いちご協議会などが制定し、旬の始まりを知らせる日になっている。',
    ),
    '1-6': DayOccasion(
        name: '色の日',
        reason: '1と6で「いろ」と読む語呂合わせ。色彩の楽しみやデザインを考える日として用いられる。',
    ),
    '1-7': DayOccasion(
        name: '七草の日',
        reason: '人日の節句。春の七草を粥にして食べ、正月の疲れた胃を休める中国由来の風習が日本に残った。',
    ),
    '1-8': DayOccasion(
        name: '外国語の日',
        reason: '「いい (1) は (8)」から「いい話」や語学を連想させる日として、語学学習のきっかけに使われる。',
    ),
    '1-9': DayOccasion(
        name: '風邪の日',
        reason: '寒さが厳しい時期に、かぜの予防と休養を呼びかける日として医療機関や自治体が啓発に使う。手洗い・湿度・睡眠が基本になる。',
    ),
    '1-10': DayOccasion(
        name: '110番の日',
        reason: '警察への緊急通報「110番」が1948年に東京で始まったことにちなみ、正しい通報の仕方を知らせる日。',
    ),
    '1-11': DayOccasion(
        name: '塩の日',
        reason: '1が縦に2本で「11」＝「十一」から「塩」の字を連想する、という業界由来の記念日。塩の役割を見直す日。',
    ),
    '1-12': DayOccasion(
        name: 'スキーの日',

```

```
    reason: '1911年1月12日、レルヒ少佐が新潟・高田で日本にスキー術を伝えた日。日本スキー発祥の記念日。',
  ),
  '1-13': DayOccasion(
    name: '挨拶の日',
    reason: '正月も明け、日常の会話が戻る時期。朝の「おはよう」から人間関係が整う、という趣旨で地域の挨拶運動に使われる。',
  ),
  '1-14': DayOccasion(
    name: 'タロとジロの日',
    reason: '1957年1月14日、南極で置き去りにされたカラフト犬タロとジロの生存が確認された日。絆と生命力の象徴。',
  ),
  '1-15': DayOccasion(
    name: 'いちごの日',
    reason: '1と15で「いちご」と読む語呂合わせ。1月5日とともに、いちごの消費を広げる日として使われる。',
  ),
  '1-16': DayOccasion(
    name: '禁酒の日',
    reason: '1919年1月16日、アメリカで禁酒法が成立した日（発効は翌年）。酒との付き合い方を考えるきっかけの日。',
  ),
  '1-17': DayOccasion(
    name: '防災とボランティアの日',
    reason: '1995年1月17日の阪神・淡路大震災を忘れない日。政府が制定し、備えと助け合いを呼びかける。',
  ),
  '1-18': DayOccasion(
    name: '明暦の大火を覚える日',
    reason: '1657年1月18日から19日にかけて江戸で明暦の大火（振袖火事）が起きた。都市の防火と避難を忘れない日。',
  ),
  '1-19': DayOccasion(
    name: '今昔物語の日',
    reason: '1と19で「いい（1）いく（19）」から物語を読む日、として古典に親しむ動きがある。冬の読書に向く日。',
  ),
  '1-20': DayOccasion(
    name: '玉の輿の日',
    reason: '1683年1月20日、5代将軍徳川綱吉の母・桂昌院が従一位を授けられた日。身分を超えた出世譚の由来日。',
  ),
  '1-21': DayOccasion(
    name: '料理番組の日',
    reason: '1953年1月21日前後からテレビ料理番組が家庭に広まり始めた冬。食を通じた学びの日として、台所の基礎を振り返る。',
  ),
  '1-22': DayOccasion(
    name: 'カレーの日',
    reason: '1982年、全国学校栄養士協議会が1月22日をカレーの日とした。給食で人気のカレーを通じて食育を広める。',
  ),
  '1-23': DayOccasion(
    name: '電子メールの日',
    reason: 'インターネットプロバイダ協会などが制定。1月23日の「123」で、一・二・三と気軽に連絡しようという意味。',
  ),
  '1-24': DayOccasion(
    name: '金の日',
    reason: '1と24で「いいは（いい金）」ではなく、24金から「金の日」。貴金属や価値あるものを大切にする日として用いられる。',
  ),
  '1-25': DayOccasion(
    name: '中華まんの日',
    reason: '「いい（1）ニコニコ（25）」から、冬に中華まんを食べる日として食品メーカーが広めた。',
  ),
  '1-26': DayOccasion(
    name: '文化財防火デー',
    reason: '1949年1月26日、法隆寺金堂が火災で焼損した日。文化財を火から守る全国的な防火運動の日。',
  ),
  '1-27': DayOccasion(
    name: '国際ホロコースト記念日',
    reason: '1945年1月27日、アウシュヴィッツが解放された日。国連が制定し、ジェノサイドの記憶を継ぐ日。',
  ),
  '1-28': DayOccasion(
    name: '衣類の日',
    reason: '1と28で「いいふく」と読む語呂合わせ。衣服の手入れや循環を考える日として繊維業界が用いる。',
  ),
  '1-29': DayOccasion(
    name: '昭和基地の日',
    reason: '1957年1月29日、日本の南極観測隊が昭和基地で観測を開始した。極地科学の出発点を記念する。',
  ),
  '1-30': DayOccasion(
    name: '3分間待ちの日',
    reason: '1月30日は「い（1）さん（3）まる（0）」から、信号や人を待つ3分を大切にする交通安全の啓発に使われる。',
  ),
  '1-31': DayOccasion(
    name: '愛妻の日',
    reason: '末（31）の「妻」にかけて、夫が妻へ感謝する日としてライフスタイル誌や団体が広めた。',
```



```
),
'2-1': DayOccasion(
    name: 'ニッポン放送の日',
    reason: '1954年2月1日、ニッポン放送が開局。ラジオ文化を振り返る日として放送史に残る。',
),
'2-2': DayOccasion(
    name: '頭痛の日',
    reason: '2月2日の「2・2」から、慢性頭痛の正しい知識を広める日として日本頭痛学会などが啓発する。',
),
'2-3': DayOccasion(
    name: '節分',
    reason: '季節の分かれ目。立春の前日に豆をまいて邪気を払う。年によって日付は前後するが、2月3日が多い。',
),
'2-4': DayOccasion(
    name: '立春',
    reason: '二十四節気のひとつで、暦のうえの春の始まり。寒さの底から、少しずつ陽が長くなる目安の日。',
),
'2-5': DayOccasion(
    name: 'プロ野球の日',
    reason: '1936年2月5日、日本職業野球連盟が設立された。日本のプロ野球誕生を記念する日。',
),
'2-6': DayOccasion(
    name: '海苔の日',
    reason: '701年、大宝元年の賦役令で海苔が租税に指定されたとされ、全国海苔貝類漁業協同組合連合会が2月6日を海苔の日とした。',
),
'2-7': DayOccasion(
    name: '北方領土の日',
    reason: '1855年2月7日、日魯通好条約で択捉と得撫の間が国境とされた。政府が北方領土問題への関心を高める日として制定した。',
),
'2-8': DayOccasion(
    name: 'にわとりの日',
    reason: '2と8で「にわ（2）とり（とり＝鳥、8は末広がり）」より、鶏卵業界が卵の栄養を伝える日として使う。',
),
'2-9': DayOccasion(
    name: '服の日',
    reason: '2と9で「ふく」と読む語呂合わせ。衣替え前に衣服を大切にする日として繊維業界が制定した。',
),
'2-10': DayOccasion(
    name: 'ニットの日',
    reason: '2と10で「ニット」と読む語呂合わせ。日本ニット工業組合連合会が制定し、編み物文化を広める。',
),
'2-11': DayOccasion(
    name: '建国記念の日',
    reason: '建国をしのび、国を愛する心を養う国民の祝日。神武天皇即位の伝承日に由来する。',
),
'2-12': DayOccasion(
    name: 'ペンギンの日',
    reason: 'アメリカの研究者らが広めたInternational Penguin Dayとは別で、2月12日はダーウィンの誕生日でもあり、生物多様性を考える日としても使われる。',
    alsoKnownAs: ['ダーウィン誕生日'],
),
'2-13': DayOccasion(
    name: '銀行条例公布の日',
    reason: '1890年2月13日、銀行条例が公布された。日本の近代銀行制度の出発点を覚える日。',
),
'2-14': DayOccasion(
    name: 'バレンタインデー',
    reason: '3世紀ローマの聖職者バレンタインに由来する西欧の祭りが、日本ではチョコレートを贈る日として定着した。',
),
'2-15': DayOccasion(
    name: '春のパン祭り記念',
    reason: '2月はパンの需要期。15日は「いい（1）こ（5）」から食卓を祝う日として製パン業界がキャンペーンに使う。',
),
'2-16': DayOccasion(
    name: '天気図記念日',
    reason: '1883年2月16日、日本で初めて天気図が作成された。気象情報の始まりを記念する日。',
),
'2-17': DayOccasion(
    name: '月齢の日',
    reason: '2月17日ごろは冬の終わりの月が美しく、天体観測の話題が増える。空を見上げる日として親しまれる。',
),
'2-18': DayOccasion(
    name: 'エアメールの日',
    reason: '1911年ごろから航空郵便の実験が世界で始まり、2月18日は国際航空郵便の歴史を語る日として用いられる。',
),
'2-19': DayOccasion(
    name: '散文詩の日',
```

```
      reason: '2と19で「ふみ（文）く」に通じるとして、短い文章を書く日にされることがある。言葉を整える日。',
    ),
    '2-20': DayOccasion(
      name: '歌舞伎の日',
      reason: '1607年2月20日、出雲阿国が江戸城で歌舞伎踊りを演じた記録がある日とされ、歌舞伎の発祥を記念する。',
    ),
    '2-21': DayOccasion(
      name: '言語の国際デー（母語）',
      reason: '国連の国際母語デーは2月21日。1952年、バングラデシュで母語を守る運動が起きた日に由来する。',
    ),
    '2-22': DayOccasion(
      name: '猫の日',
      reason: '「にゃんにゃんにゃん」と2が三つ続く語呂合わせ。1987年に猫の日実行委員会が制定した。',
    ),
    '2-23': DayOccasion(
      name: '富士山の日',
      reason: '2と3で「ふじさん」と読む語呂合わせ。静岡県などが制定し、のちに天皇誕生日とも重なる年がある。',
    ),
    '2-24': DayOccasion(
      name: '月光仮面の日',
      reason: '1958年2月24日、テレビ映画『月光仮面』が放送開始。日本のヒーロー番組史の記念日。',
    ),
    '2-25': DayOccasion(
      name: '夕刊紙の日',
      reason: '2月25日は明治期に夕刊新聞が広まった時期にあたり、活字文化を振り返る日として語られる。',
    ),
    '2-26': DayOccasion(
      name: '二・二六事件の日',
      reason: '1936年2月26日、青年将校らによるクーデターが起きた。昭和史を忘れないための日。',
    ),
    '2-27': DayOccasion(
      name: '新選組の日',
      reason: '1863年2月27日ごろ、壬生浪士組が新選組の名で活動を本格化させたとされる。幕末史を学ぶ日。',
    ),
    '2-28': DayOccasion(
      name: 'ビスケットの日',
      reason: '江戸時代、長崎でパンやビスケットが作られ始めた伝承と、2と8で「にぱん（二番）」「にく（2）や（8）」など諸説あるが、全国ビスケット協会が2月28日を制定。',
    ),
    '2-29': DayOccasion(
      name: '閏日',
      reason: '4年に一度のうるう日。地球の公転と暦のズレを補正するために2月へ1日を足す。この日生まれは4年に一度しか暦の誕生日が来ない。',
    ),
    '3-1': DayOccasion(
      name: 'ビキニデー',
      reason: '1954年3月1日、ビキニ環礁の水爆実験で第五福竜丸が被ばくした。核実験の被害を忘れず、平和を願う日。',
    ),
    '3-2': DayOccasion(
      name: 'ミニチュアの日',
      reason: '3と2で「ミニ」に通じる語呂。小さな模型や鉄道模型を楽しむ日として趣味の世界で使われる。',
    ),
    '3-3': DayOccasion(
      name: 'ひな祭り',
      reason: '桃の節句。女の子の健やかな成長を願い雛人形を飾る。上巳の節句が3月3日に定着した。',
      alsoKnownAs: ['耳の日'],
    ),
    '3-4': DayOccasion(
      name: 'ミニスカートの日',
      reason: '1960年代、マリー・クワントらがミニスカートを流行させた時期に因み、3と4で「みにし」と読む語呂合わせ。',
    ),
    '3-5': DayOccasion(
      name: 'ミスの日',
      reason: '3と5で「みご（35）」ではなく「ミス」に通じる日として、失敗を責めず学び直すキャンペーンに使われる。',
    ),
    '3-6': DayOccasion(
      name: '世界一周記念日',
      reason: '1521年3月6日、マゼラン艦隊が世界一周の途上でグアムに到達した記録がある。冒険と地理を学ぶ日。',
    ),
    '3-7': DayOccasion(
      name: '消防記念日',
      reason: '1948年3月7日、消防組織法が施行された。日本の消防制度の出発点を記念する。',
    ),
    '3-8': DayOccasion(
      name: '国際女性デー',
      reason: '国連が制定。1910年の国際会議で女性の政治的権利を求める日として提案され、世界に広がった。',
    ),
    '3-9': DayOccasion(
```

```
name: '記念切手の日',
reason: '1894年3月9日、日本最初の記念切手（明治天皇銀婚）が発行された。郵便文化の記念日。',
),
'3-10': DayOccasion(
name: '砂糖の日',
reason: '3と10で「さとう」と読む語呂合わせ。精糖工業会が制定し、甘味の歴史と適量を伝える。',
),
'3-11': DayOccasion(
name: '東日本大震災追悼の日',
reason: '2011年3月11日14時46分、東北地方太平洋沖地震が起きた。防災と追悼のための日。',
),
'3-12': DayOccasion(
name: 'スーツの日',
reason: '3と12で「さいふ」ではなく、新生活前にスーツを整える日としてアパレルが用いる。入学・就職の季節。',
),
'3-13': DayOccasion(
name: 'サンドイッチデー',
reason: '3月13日は1年で真ん中に近い日、というより、パンではさむサンドイッチの「3」と具のイメージから食の日とされることがある。春の行楽弁当の日。',
),
'3-14': DayOccasion(
name: 'ホワイトデー',
reason: 'バレンタインのお返しの日として日本の製菓業界が1980年前後に定着させた。また円周率3.14の日でもある。',
alsoKnownAs: ['円周率の日'],
),
'3-15': DayOccasion(
name: '靴の日',
reason: '3と15で「さいふ」ではなく、春の足元を整える日として靴業界が制定。新生活の始まりに合わせた。',
),
'3-16': DayOccasion(
name: '国立公園指定記念日',
reason: '1934年3月16日、瀬戸内海・雲仙・霧島が日本最初の国立公園に指定された。自然保護の出発点。',
),
'3-17': DayOccasion(
name: '漫画週刊誌の日',
reason: '1959年3月17日前後、『週刊少年マガジン』『週刊少年サンデー』が創刊された。少年漫画誌時代の始まり。',
),
'3-18': DayOccasion(
name: '精霊の日',
reason: '春分に近いこの時期、祖先を思う彼岸の入りとなる年が多い。いのちと季節の循環を意識する日。',
),
'3-19': DayOccasion(
name: 'ミュージックの日',
reason: '3と19で「さいく」ではなく、春のコンサートシーズン前に音楽を楽しむ日として使われる。',
),
'3-20': DayOccasion(
name: '上野動物園開園記念日',
reason: '1882年3月20日、上野動物園が開園した。日本最古の動物園の誕生日。春分と重なる年もある。',
),
'3-21': DayOccasion(
name: '世界詩歌記念日',
reason: 'ユネスコが制定したWorld Poetry Day。詩を読み、言葉の力を分かち合う国際的な日。',
),
'3-22': DayOccasion(
name: '世界水の日',
reason: '国連が制定。水資源の大切さを考える日。1992年の環境会議を経て1993年から実施された。',
),
'3-23': DayOccasion(
name: '世界気象デー',
reason: '1950年3月23日、世界気象機関（WMO）が発足した。天気と気候を守る国際的な記念日。',
),
'3-24': DayOccasion(
name: '世界結核デー',
reason: '1882年3月24日、コッホが結核菌発見を発表した。結核の予防と治療を呼びかけるWHOの日。',
),
'3-25': DayOccasion(
name: '電気記念日',
reason: '1878年3月25日、東京・中央電信局の開業祝いで日本初のアーク灯が灯った。電気の普及を記念する。',
),
'3-26': DayOccasion(
name: 'カチューシャの歌の日',
reason: '1914年3月26日、島村抱月・松井須磨子の『カチューシャの唄』が発表された。近代日本の流行歌の原点。',
),
'3-27': DayOccasion(
name: 'さくらの日',
reason: '日本さくらの会が制定。春分のあと、桜が開花する時期に合わせて花を愛でる日。',
),
```

```
'3-28': DayOccasion(  
    name: '三色の日',  
    reason: '3と28で「みつば」や三色旗を連想する日として、色彩や旗の歴史を話すきっかけに使われる。',  
),  
'3-29': DayOccasion(  
    name: 'マリモの日',  
    reason: '1952年3月29日、阿寒湖のマリモが特別天然記念物に指定された。貴重な自然を守る日。',  
),  
'3-30': DayOccasion(  
    name: '国立公園の日',  
    reason: '3月は国立公園指定の季節。自然公園法の趣旨を思い、山と海を大切にする日。',  
),  
'3-31': DayOccasion(  
    name: 'オーケストラの日',  
    reason: '3と31で「み（3）みんな（31）」から、日本オーケストラ連盟が「みんなでオーケストラ」の日とした。',  
),  
'4-1': DayOccasion(  
    name: 'エイプリルフール',  
    reason: '4月1日に嘘をついても許される西欧の風習。起源は諸説あるが、新年度の日本では新しい門出の日でもある。',  
),  
'4-2': DayOccasion(  
    name: '国際子どもの本の日',  
    reason: '童話作家アンデルセンの誕生日。国際児童図書評議会（IBBY）が、子どもの読書を推進する日とした。',  
),  
'4-3': DayOccasion(  
    name: 'シーサーの日',  
    reason: '4と3で「し（4）ーさー」に通じる沖縄の語呂。魔除けの獅子を思い、地域文化を大切にする日。',  
),  
'4-4': DayOccasion(  
    name: 'あんぱんの日',  
    reason: '1875年4月4日、木村屋が桜あんぱんを明治天皇に献上したとされる日。日本のあんぱん誕生譚。',  
),  
'4-5': DayOccasion(  
    name: '囲碁の日',  
    reason: '4と5で「いし」と読む語呂合わせ。日本棋院などが囲碁に親しむ日として制定した。',  
),  
'4-6': DayOccasion(  
    name: '城の日',  
    reason: '4と6で「しろ」と読む語呂合わせ。日本城郭協会が制定し、城郭文化の保存を呼びかける。',  
),  
'4-7': DayOccasion(  
    name: '世界保健デー',  
    reason: '1948年4月7日、世界保健機関（WHO）が発足した。健康を権利として考える国際的な日。',  
),  
'4-8': DayOccasion(  
    name: '花祭り',  
    reason: '釈迦の誕生日（灌仏会）。寺院で甘茶をかけ、いのちの誕生を祝う仏教行事。',  
),  
'4-9': DayOccasion(  
    name: 'フォークソングの日',  
    reason: '4と9で「フォーク」と読む語呂合わせ。日本フォークソング協会などが制定した。',  
),  
'4-10': DayOccasion(  
    name: '駅の日',  
    reason: '1872年（明治5年）9月12日＝新暦10月14日が鉄道の日だが、4月10日は「よ（4）いとう（10）」から良い駅を目指す日としてJRなどが用いるほか、女性の日（1946年の初の女性選挙）でもある。',  
    alsoKnownAs: ['女性の日'],  
),  
'4-11': DayOccasion(  
    name: '世界パーキンソン病デー',  
    reason: 'パーキンソン病を記述したジェームズ・パーキンソンの誕生日。正しい理解と早期受診を呼びかける国際的な日。',  
),  
'4-12': DayOccasion(  
    name: '世界宇宙飛行の日',  
    reason: '1961年4月12日、ガガーリンが人類初の宇宙飛行に成功した。宇宙開発史の記念日。',  
),  
'4-13': DayOccasion(  
    name: '決闘の日',  
    reason: '1584年4月13日、フランスで決闘禁止令が出された、などの西洋史エピソードから、争いを避ける日として語られる。',  
),  
'4-14': DayOccasion(  
    name: 'オレンジデー',  
    reason: 'パレンタインとホワイトデーの中間。愛媛県などが柑橘の消費と交流を広げる日として広めた。',  
),  
'4-15': DayOccasion(  
    name: 'ヘリコプターの日',  
    reason: '1944年4月15日前後、ヘリコプターの実用化が進んだ歴史に因み、航空救難への感謝を表す日として使われる。',
```

```
),
'4-16': DayOccasion(
    name: '女子マラソンの日',
    reason: '1981年4月16日ごろ、日本で女子マラソン公式競技化の動きが進んだ。長距離走の歴史を覚える日。',
),
'4-17': DayOccasion(
    name: '恐竜の日',
    reason: '1994年ごろ、福井県などが恐竜化石の発見を記念して制定。古生物への関心を高める日。',
),
'4-18': DayOccasion(
    name: '発明の日',
    reason: '1885年4月18日、専売特許条例が公布された。日本の特許制度の誕生日で、発明協会が記念日とした。',
),
'4-19': DayOccasion(
    name: '地図の日',
    reason: '1800年4月19日、伊能忠敬が第一次測量の旅に出発した。日本地図づくりの偉業を記念する。',
),
'4-20': DayOccasion(
    name: '郵政記念日',
    reason: '1871年4月20日、日本で郵便制度が始まった（新暦）。手紙と物流の原点を振り返る日。',
),
'4-21': DayOccasion(
    name: '民放の日',
    reason: '1951年4月21日、民間放送が開始された。ラジオ・テレビの民間放送の誕生日。',
),
'4-22': DayOccasion(
    name: 'アースデイ',
    reason: '1970年4月22日、アメリカで始まった地球環境を考える市民運動。世界に広がった環境の日。',
),
'4-23': DayOccasion(
    name: '世界図書・著作権デー',
    reason: 'ユネスコが制定。シェイクスピアとセルバンテスの命日が重なる日で、読書と著作権を守る。',
),
'4-24': DayOccasion(
    name: '植物学の日',
    reason: '植物学者リンネの誕生日に近い春。日本では新緑を観察する日として学校行事にも使われる。',
),
'4-25': DayOccasion(
    name: '世界ペンギンデー',
    reason: '南半球でペンギンの保護を呼びかける日として広まった。海洋環境と生態系を考える春の国際的な啓発日。',
),
'4-26': DayOccasion(
    name: 'チェルノブイリを忘れない日',
    reason: '1986年4月26日、チェルノブイリ原子力発電所で事故が起きた。原子力災害の教訓を継ぐ日。',
),
'4-27': DayOccasion(
    name: '世界グラフィックデザインデー',
    reason: '国際グラフィックデザイン団体協議会が制定。視覚言語の役割と著作権を考える日。',
),
'4-28': DayOccasion(
    name: '象の日',
    reason: '1729年4月28日、ベトナムから象が将軍に献上され江戸で話題になった。日本と象の出会いの日。',
),
'4-29': DayOccasion(
    name: '昭和の日',
    reason: '昭和天皇誕生日を起源とする国民の祝日。昭和の歩みを振り返る日。',
),
'4-30': DayOccasion(
    name: '図書館記念日',
    reason: '1950年4月30日、図書館法が公布された。知る自由と公共図書館を守る日。',
),
'5-1': DayOccasion(
    name: 'メーデー',
    reason: '労働者が権利と連帯を示す国際的な日。1886年のシカゴのストライキに由来する。',
),
'5-2': DayOccasion(
    name: '緑茶の日',
    reason: '日本茶業中央会が、新茶の旬である5月2日を緑茶の日とした。茶の香味と文化を広める。',
),
'5-3': DayOccasion(
    name: '憲法記念日',
    reason: '1947年5月3日、日本国憲法が施行された。立憲主義を考える国民の祝日。',
),
'5-4': DayOccasion(
    name: 'みどりの日',
    reason: '自然に親しみ、その恩恵に感謝する国民の祝日。ゴールデンウィークの中日にあたる。',
),
```



```
name: 'ゴルフ場記念日',
reason: '1903年5月24日、神戸ゴルフ倶楽部が開場した。日本のゴルフ発祥を記念する。',
),
'5-25': DayOccasion(
name: '広辞苑の日',
reason: '1955年5月25日、岩波書店から『広辞苑』初版が刊行された。日本語の辞書文化を記念する。',
),
'5-26': DayOccasion(
name: 'ラッキーゾーンの日',
reason: '1949年5月26日、大阪球場にラッキーゾーンが設けられた野球史のエピソード。記録と記憶の日。',
),
'5-27': DayOccasion(
name: '新潟地震を覚える日',
reason: '1964年5月27日、新潟地震が起きた。液状化など都市災害の教訓を継ぐ日。',
),
'5-28': DayOccasion(
name: 'ゴルフ発祥を継ぐ日',
reason: '1903年5月24日の神戸ゴルフ倶楽部開場に近いこの週、日本にゴルフが根づいたことを記念する。',
),
'5-29': DayOccasion(
name: 'こんにゃくの日',
reason: '5と29で「ゴロ（5）つく」より、胃腸にやさしいこんにゃくを食べる日として産地がPRする。',
),
'5-30': DayOccasion(
name: 'ごみゼロの日',
reason: '5と30で「ごみ（530）ゼロ」と読む語呂合わせ。環境省などが散乱ごみをなくす全国運動の日とした。',
),
'5-31': DayOccasion(
name: '世界禁煙デー',
reason: 'WHOが制定。たばこの健康影響を伝え、禁煙を支援する国際的な日。',
),
'6-1': DayOccasion(
name: '気象記念日',
reason: '1875年6月1日、東京気象台（現・気象庁）が観測を始めた。日本の近代気象業務の誕生日。',
),
'6-2': DayOccasion(
name: '路地の日',
reason: '6と2で「ろ（6）じ」と読む語呂合わせ。狭い路地の文化や防災を考える日。',
),
'6-3': DayOccasion(
name: '測量の日',
reason: '1949年6月3日、測量法が施行された。国土を測る仕事への理解を広める日。',
),
'6-4': DayOccasion(
name: '虫の日',
reason: '6と4で「むし」と読む語呂合わせ。昆虫の観察と自然保護をすすめる日。',
),
'6-5': DayOccasion(
name: '世界環境デー',
reason: '1972年6月5日、ストックホルムで国連人間環境会議が開かれた。日本では「環境の日」でもある。',
),
'6-6': DayOccasion(
name: '楽器の日',
reason: '日本楽器協会が制定。6月は梅雨で室内、楽器に親しむのに良い月として6月6日を選んだ。',
),
'6-7': DayOccasion(
name: '母親大会記念日',
reason: '1955年6月7日、日本母親大会が始まった。平和と暮らしを考える市民運動の日。',
),
'6-8': DayOccasion(
name: '世界海洋デー',
reason: '国連が制定。海の恵みとプラスチックごみなど、海洋環境を考える国際的な日。',
),
'6-9': DayOccasion(
name: 'ロックの日',
reason: '6と9で「ロック」と読む語呂合わせ。ロック音楽を楽しむ日としてファンや業界が用いる。',
),
'6-10': DayOccasion(
name: '時の記念日',
reason: '671年6月10日、天智天皇が漏刻（水時計）を置いて時を知らせたと『日本書紀』にある。日本の「時」の起源。',
),
'6-11': DayOccasion(
name: '傘の日',
reason: '梅雨入りの時期。6月11日は「かさ」に通じる語呂として、傘の安全とマナーを呼びかける。',
),
'6-12': DayOccasion(
name: '恋人の日',
```



```
      reason: '6と12で「ろう（6）ふ（2）」ではなく、日記や手紙を贈る日として雑貨業界がキャンペーンに使う。',
    ),
    '6-13': DayOccasion(
      name: '小さな親切の日',
      reason: '1963年、小さな親切運動が始まった趣旨を継ぎ、身近な善意を重ねる日として啓発される。',
    ),
    '6-14': DayOccasion(
      name: '世界献血者デー',
      reason: '血液型を発見したラントシュタイナーの誕生日。WHOが献血への感謝を表す日とした。',
    ),
    '6-15': DayOccasion(
      name: '信用金庫の日',
      reason: '1951年6月15日、信用金庫法が公布された。地域金融の役割を伝える日。',
    ),
    '6-16': DayOccasion(
      name: '和菓子の日',
      reason: '848年、嘉祥の日に菓子を供えて厄を払った故事に因み、全国和菓子協会が6月16日を和菓子の日とした。',
    ),
    '6-17': DayOccasion(
      name: 'おまわりさんの日',
      reason: '1874年6月17日、東京で巡査が配置された。地域の安全を守る警察の仕事を知る日。',
    ),
    '6-18': DayOccasion(
      name: '海外移住の日',
      reason: '1908年6月18日、第一回移民船笠戸丸がサントスに到着した。日系社会の歴史を覚える日。',
    ),
    '6-19': DayOccasion(
      name: 'ベースボール記念日',
      reason: '1846年6月19日、アメリカで公式ルールに近い試合が行われたとされる。野球の起源を語る日。',
    ),
    '6-20': DayOccasion(
      name: '世界難民の日',
      reason: '国連が制定。難民の権利と支援を考える国際的な日。1951年の難民条約に連なる。',
    ),
    '6-21': DayOccasion(
      name: '夏至',
      reason: '一年で昼が最も長い日。北半球の夏の始まりの天文学的な目安。',
    ),
    '6-22': DayOccasion(
      name: 'ボウリングの日',
      reason: '1967年6月22日、東京で世界選手権が開かれたことなどから、日本ボウリング協会が記念日とした。',
    ),
    '6-23': DayOccasion(
      name: '沖縄慰霊の日',
      reason: '1945年6月23日、沖縄戦の組織的戦闘が終わったとされる日。沖縄県の休日で、平和を祈る。',
    ),
    '6-24': DayOccasion(
      name: 'UFOの日',
      reason: '1947年6月24日、アメリカで「空飛ぶ円盤」目撃報道が出た。未確認飛行物体という言葉が広まった日。',
    ),
    '6-25': DayOccasion(
      name: '住宅デー',
      reason: '6月は住宅月間。25日は「ふく（含む）住まい」を点検する日として建築・防災啓発に使われる。',
    ),
    '6-26': DayOccasion(
      name: '国際麻薬乱用・不正取引反対デー',
      reason: '国連が制定。薬物乱用防止を呼びかける国際的な日。',
    ),
    '6-27': DayOccasion(
      name: '演説の日',
      reason: '1874年6月27日、慶應義塾で日本初の演説会が開かれた。福沢諭吉がspeechを「演説」と訳した。',
    ),
    '6-28': DayOccasion(
      name: '貿易記念日',
      reason: '1859年6月28日（新暦）、横浜など五港が開港した。日本の近代貿易の始まりを記念する。',
    ),
    '6-29': DayOccasion(
      name: '佃煮の日',
      reason: '6と29で「ろ（6）ふく（29）」より、保存食の佃煮を見直す日として東京の業者がPRする。',
    ),
    '6-30': DayOccasion(
      name: 'ハーフタイムの日',
      reason: '1年の半分が終わる日。大祓（なごしのはらえ）で半年の穢れを払う神事もある。',
      alsoKnownAs: ['夏越しの祓'],
    ),
    '7-1': DayOccasion(
      name: '山開き',
```



```
    reason: '多くの山岳地で夏山シーズンが始まる日。安全登山を呼びかける区切り。',
  ),
  '7-2': DayOccasion(
    name: 'うどんの日',
    reason: '7月2日は「な（7）る（る）」より、讃岐などで麺を味わうキャンペーン日として使われる。夏の麺の日。',
  ),
  '7-3': DayOccasion(
    name: 'ソフトクリームの日',
    reason: '1951年7月3日、日本でソフトクリームが本格販売されたとされる。夏の味覚の記念日。',
  ),
  '7-4': DayOccasion(
    name: '梨の日',
    reason: '7と4で「なし」と読む語呂合わせ。ただし梨の旬は秋で、産地が先行してPRする日。',
  ),
  '7-5': DayOccasion(
    name: '穴子の日',
    reason: '7と5で「あなご」と読む語呂合わせ。土用の丑に近い夏、うなぎの仲間を知る日。',
  ),
  '7-6': DayOccasion(
    name: 'サラダ記念日',
    reason: '歌人・俵万智の歌集『サラダ記念日』から。日常の小さな記念を大切にする日。',
  ),
  '7-7': DayOccasion(
    name: '七夕',
    reason: '織姫と彦星が年に一度会うという中国の伝説が日本の星祭りになった。笹に短冊を飾る。',
  ),
  '7-8': DayOccasion(
    name: '質屋の日',
    reason: '7と8で「しち」と読む語呂合わせ。全国質屋組合連合会が制定した。',
  ),
  '7-9': DayOccasion(
    name: 'ジェットコースターの日',
    reason: '1955年7月9日、豊島園に日本初の本格的ジェットコースターが登場したとされる。',
  ),
  '7-10': DayOccasion(
    name: '納豆の日',
    reason: '7月10日を「なっとう」の語呂に近づけ、納豆協同組合連合会が制定。発酵食品の日。',
  ),
  '7-11': DayOccasion(
    name: '世界人口デー',
    reason: '1987年7月11日、世界人口が50億人に達したと国連が発表した日。人口と資源を考える。',
  ),
  '7-12': DayOccasion(
    name: '人間ドックの日',
    reason: '1954年7月12日、日本で人間ドックが開始された。予防医学を広める日。',
  ),
  '7-13': DayOccasion(
    name: '盆の入り',
    reason: '地域によって盆の入りとなる日。祖先を迎え、家族で過ごす夏の区切り。精霊棚を飾る家もある。',
  ),
  '7-14': DayOccasion(
    name: '検疫記念日',
    reason: '1879年7月14日、日本で検疫が始まった。感染症から暮らしを守る仕事を知る日。',
  ),
  '7-15': DayOccasion(
    name: 'お中元の日',
    reason: '中元は中国の三元に由来し、日本では夏の贈り物として定着。日ごろの感謝を届ける日。',
  ),
  '7-16': DayOccasion(
    name: '駅弁の日',
    reason: '1885年7月16日、宇都宮で日本初の駅弁が売られたという説がある。旅と食の記念日。',
  ),
  '7-17': DayOccasion(
    name: '東京の日',
    reason: '1868年7月17日（慶応4年7月17日）、江戸が東京と改称された。首都の名前が決まった日。',
  ),
  '7-18': DayOccasion(
    name: '光化学スモッグの日',
    reason: '1970年7月18日、東京で光化学スモッグが大きな被害を出した。大気汚染を忘れない日。',
  ),
  '7-19': DayOccasion(
    name: 'バリ・メトロの日',
    reason: '1900年7月19日、バリ万博に合わせてメトロが開業した。都市交通の歴史を学ぶ日。',
  ),
  '7-20': DayOccasion(
    name: 'ハンバーガーの日',
    reason: 'モスバーガーなどが7月20日を記念日とした例があり、夏の軽食を楽しむ日。海の日と重なる年もある。',
```

```
),
'7-21': DayOccasion(
    name: '自然公園の日',
    reason: '自然公園法の趣旨を夏山シーズンに合わせ、国立・国定公園を大切にする日。',
),
'7-22': DayOccasion(
    name: 'ナッツの日',
    reason: '7月22日は「ナッツ」の語感と夏の栄養補給から、ナッツ業界がPRする日。',
),
'7-23': DayOccasion(
    name: 'ふみの日',
    reason: '日本郵便が7月23日を「ふみの日」とした。手紙を書く文化を守るキャンペーン。',
),
'7-24': DayOccasion(
    name: '劇画の日',
    reason: '1959年ごろ劇画という語が広まり、7月は貸本漫画の季節。物語表現の多様性を覚える日。',
),
'7-25': DayOccasion(
    name: 'かき氷の日',
    reason: '7と25で「なつ（7）にこごり」より、夏の風物詩かき氷を味わう日として製氷業が用いる。',
),
'7-26': DayOccasion(
    name: '番茶の日',
    reason: '夏に冷やして飲む番茶・麦茶の習慣から、日本茶を気軽に飲む日として産地がPRする。',
),
'7-27': DayOccasion(
    name: 'スイカの日',
    reason: '7月の土用にスイカを食べる風習。27日は「つな（土用）西瓜」キャンペーンに使われる。',
),
'7-28': DayOccasion(
    name: 'なにわの日',
    reason: '7と28で「なにわ」と読む語呂合わせ。大阪の歴史と食文化を発信する日。',
),
'7-29': DayOccasion(
    name: 'アマチュア無線の日',
    reason: '1952年7月29日、日本でアマチュア無線が再開された。通信趣味の記念日。',
),
'7-30': DayOccasion(
    name: 'プロレス記念日',
    reason: '1953年7月30日、日本力道山のプロレスがテレビ放映されブームになった。大衆娯楽の日。',
),
'7-31': DayOccasion(
    name: '蓄音機の日',
    reason: '1877年7月ごろエジソンが蓄音機を発明した。音を記録する技術の起源を記念する。',
),
'8-1': DayOccasion(
    name: '水の日',
    reason: '国土交通省などが制定。水資源の大切さを考え、節水と水源保全を呼びかける。',
),
'8-2': DayOccasion(
    name: 'パンツの日',
    reason: '8と2で「パンツ」と読む語呂合わせ。下着メーカーが快適さと洗濯を啓発する日。',
),
'8-3': DayOccasion(
    name: 'はちみつの日',
    reason: '8月3日の「はち（8）」から、日本養蜂協会がはちみつの日とした。',
),
'8-4': DayOccasion(
    name: '箸の日',
    reason: '8と4で「はし」と読む語呂合わせ。正しい箸使いと食文化を伝える日。',
),
'8-5': DayOccasion(
    name: 'タクシーの日',
    reason: '1912年8月5日、東京でタクシー営業が始まった。公共交通としてのタクシーを知る日。',
),
'8-6': DayOccasion(
    name: '広島原爆の日',
    reason: '1945年8月6日、広島に原子爆弾が投下された。平和を祈り、核兵器廃絶を願う日。',
),
'8-7': DayOccasion(
    name: '鼻の日',
    reason: '8と7で「はな」と読む語呂合わせ。耳鼻咽喉科領域の健康を考える日。',
),
'8-8': DayOccasion(
    name: '世界猫の日',
    reason: 'International Cat Day。猫の福祉を考える国際的な日。日本では「八八（パパ）」の語呂で父の日とは別。',
    alsoKnownAs: ['笑いの日'],
),
```

```
),
'8-9': DayOccasion(
    name: '長崎原爆の日',
    reason: '1945年8月9日、長崎に原子爆弾が投下された。二度と繰り返さないための追悼の日。',
),
'8-10': DayOccasion(
    name: '帽子の日',
    reason: '8と10で「ハット」や「ハチ（8）トー（10）」から、暑さ対策として帽子をかぶる日。',
),
'8-11': DayOccasion(
    name: '山の日',
    reason: '山に親しむ機会を得て、山の恩恵に感謝する国民の祝日。',
),
'8-12': DayOccasion(
    name: '国際青少年デー',
    reason: '国連が制定。若者の社会参加と権利を考える国際的な日。',
),
'8-13': DayOccasion(
    name: '左利きの日',
    reason: 'International Lefthanders Day。左利きの不便さを知り、道具や環境を見直す日。',
),
'8-14': DayOccasion(
    name: '専業主婦の日',
    reason: '家庭の仕事の価値を考える日として市民団体が提唱。家事労働の見えにくさを問い直す。',
),
'8-15': DayOccasion(
    name: '終戦記念日',
    reason: '1945年8月15日、日本がポツダム宣言を受諾し、昭和天皇が終戦を放送した。平和を誓う日。',
),
'8-16': DayOccasion(
    name: '送り火の日',
    reason: '京都の五山送り火など、お盆の締めくくりに祖先を送る日。夏の精霊を送り、日常へ戻る区切り。',
),
'8-17': DayOccasion(
    name: 'パイナップルの日',
    reason: '8と17で「パイ（8）ナッ（な）プル」に通じる語呂。熱帯果実に親しむ日。',
),
'8-18': DayOccasion(
    name: '米の日',
    reason: '漢字の「米」を分解すると「八・十・八」になることから、8月18日が米の日とされた。米作りには八十八の手間がかかると伝えられ、農家への感謝を込めた日でもある。1915年のこの日には第1回全国中等学校優勝野球大会（高校野球の前身）も開かれている。',
    alsoKnownAs: ['高校野球記念日'],
),
'8-19': DayOccasion(
    name: '俳句の日',
    reason: '8と19で「はい（8）く（19）」と読む語呂合わせ。日本文芸家協会俳句部門などが親しむ日とした。',
),
'8-20': DayOccasion(
    name: '蚊の日',
    reason: '1897年8月20日、ロナルド・ロスがマラリアを媒介する蚊を確認した。感染症と衛生を考える日。',
),
'8-21': DayOccasion(
    name: '噴水の日',
    reason: '1957年8月21日、日比谷公園の大噴水が完成した。都市の水辺を楽しむ日。',
),
'8-22': DayOccasion(
    name: '餃子の日',
    reason: '宇都宮など餃子の街が、8月22日を「パーフェクト（8が末広がり）に似合う日」としてPRするほか、語呂で「ば（8）ふ（2）に（2）」と読む動きもある。',
),
'8-23': DayOccasion(
    name: '処暑',
    reason: '二十四節気のひとつ。暑さが峠を越え、朝夕に秋の気配が混じる目安の日。',
),
'8-24': DayOccasion(
    name: 'ボンベイの日',
    reason: '西暦79年8月24日ごろ、ヴェスヴィオ火山の噴火でボンベイが埋まった。災害の記録が都市の記憶になることを示す日。',
),
'8-25': DayOccasion(
    name: '即席ラーメン記念日',
    reason: '1958年8月25日、日清食品がチキンラーメンを発売した。世界に広がった即席麺の誕生日。',
),
'8-26': DayOccasion(
    name: '人権宣言の日',
    reason: '1789年8月26日、フランスで人権宣言が採択された。自由と平等の理念が文書になった日。',
),
'8-27': DayOccasion(
```

```
name: '男女平等の日',
reason: '1985年8月27日、日本が女子差別撤廃条約を批准した。平等な社会を考える日。',
),
'8-28': DayOccasion(
name: 'バイオリンの日',
reason: '8と28で「ばい（8）おりん」に通じる語呂。弦楽器に親しむ日。',
),
'8-29': DayOccasion(
name: '焼き肉の日',
reason: '8と29で「や（8）きにく（29）」と読む語呂合わせ。全国焼肉協会が制定した。',
),
'8-30': DayOccasion(
name: '夏休み明け準備の日',
reason: '多くの学校で二学期に近い時期。生活リズムを整え、学びの習慣を戻すための区切りの日。',
),
'8-31': DayOccasion(
name: '野菜の日',
reason: '8と31で「やさい」と読む語呂合わせ。農林水産省なども野菜摂取を呼びかける。',
),
'9-1': DayOccasion(
name: '防災の日',
reason: '1923年9月1日の関東大震災に由来。政府が制定し、台風シーズン前の備えを促す。',
),
'9-2': DayOccasion(
name: '宝くじの日',
reason: '1954年9月2日、宝くじが地方財政法で位置づけられた。夢と税収の仕組みを知る日。',
),
'9-3': DayOccasion(
name: 'ホームラン記念日',
reason: '1949年9月3日、藤村富美男が当時の日本記録となるホームランを打った野球史の一日。',
),
'9-4': DayOccasion(
name: '櫛の日',
reason: '9と4で「くし」と読む語呂合わせ。髪と頭皮の手入れ、日本の櫛職人文化を伝える日。',
),
'9-5': DayOccasion(
name: '国際慈善デー',
reason: '国連が制定。マザー・テレサの命日でもあり、寄付やボランティアなど分かち合いを考える日。',
),
'9-6': DayOccasion(
name: '黒の日',
reason: '9と6で「くろ」と読む語呂合わせ。ファッションやカメラ（黒機材）の日として使われる。',
),
'9-7': DayOccasion(
name: 'クリーナーの日',
reason: '9と7で「く（9）リーナー」に通じる。秋の衣替えと掃除を始める日。',
),
'9-8': DayOccasion(
name: '国際識字デー',
reason: 'ユネスコが制定。読み書きの権利を守る国際的な日。',
),
'9-9': DayOccasion(
name: '重陽の節句',
reason: '陽数の9が重なる日。菊を飾り、長寿を願う中国由来の五節句のひとつ。',
alsoKnownAs: ['救急の日'],
),
'9-10': DayOccasion(
name: '世界自殺予防デー',
reason: 'WHOが制定。いのちの相談と予防の大切さを伝える国際的な日。',
),
'9-11': DayOccasion(
name: '公衆衛生の日',
reason: '衛生環境を見直す秋。防災の日から続く備えの週間に、暮らしの清潔を考える。',
),
'9-12': DayOccasion(
name: '宇宙の日',
reason: '1992年9月12日、毛利衛さんが日本人として初めてスペースシャトルで宇宙へ行った。',
),
'9-13': DayOccasion(
name: '世界法の日',
reason: '国際法曹協会などが、法の支配を考える日として啓発する。',
),
'9-14': DayOccasion(
name: 'コスモスの日',
reason: '秋の花コスモスが見ごろになる時期。花の名はギリシア語で「調和」を意味する。',
),
'9-15': DayOccasion(
```

```
name: '老人の日',
reason: 'かつての敬老の日（9月15日）。今も老人福祉法上の「老人の日」として残る。',
),
'9-16': DayOccasion(
name: '国際オゾン層保護デー',
reason: '1987年9月16日、モントリオール議定書が採択された。空の守り方を国際協調で決めた日。',
),
'9-17': DayOccasion(
name: 'モノレール開業記念',
reason: '1964年9月17日、東京モノレールが開業した。都市交通の革新を記念する。',
),
'9-18': DayOccasion(
name: 'かいわれ大根の日',
reason: '9と18で「く（9）さわ（18）」より、芽物野菜の栄養を伝える日として生産者がPRする。',
),
'9-19': DayOccasion(
name: '苗字の日',
reason: '1870年9月19日、平民苗字許可令が出された。誰もが姓を名乗れるようになった日。',
),
'9-20': DayOccasion(
name: 'バスの日',
reason: '1903年9月20日、京都で日本初の乗合バスが走った。公共交通の誕生日。',
),
'9-21': DayOccasion(
name: '国際平和デー',
reason: '国連が制定。停戦と非暴力を呼びかける世界的な日。敬老の日と重なる年もある。',
),
'9-22': DayOccasion(
name: 'カーフリーデー',
reason: '欧州発の「車に頼らない日」。公共交通と自転車を見直す国際的な取り組み。',
),
'9-23': DayOccasion(
name: '海王星の日',
reason: '1846年9月23日、海王星が発見された。秋分と重なる年が多く、天文と季節の日。',
),
'9-24': DayOccasion(
name: '清掃の日',
reason: '秋の全国清掃運動。地域の美化とリサイクルを進める日。',
),
'9-25': DayOccasion(
name: '主婦休みの日',
reason: '主婦連合会が3月29日・6月28日・9月25日を制定。家事を担う人の休養を社会で認める日。',
),
'9-26': DayOccasion(
name: 'ワープロの日',
reason: '1985年9月26日前後、日本語ワープロが急速に普及した。文書作成のデジタル化を記念。',
),
'9-27': DayOccasion(
name: '世界観光の日',
reason: '国連世界観光機関が制定。旅行が相互理解を深めることを伝える国際的な日。',
),
'9-28': DayOccasion(
name: 'パソコン記念日',
reason: '1982年9月28日、NECがPC-9801を発売した。日本のパーソナルコンピュータ史の画期。',
),
'9-29': DayOccasion(
name: 'クリーニングの日',
reason: '9と29で「クツ（9）rel」より、衣替えの仕上げに衣類を清潔にする日。全国クリーニング生活衛生同業組合連合会が制定。',
),
'9-30': DayOccasion(
name: 'クレーンの日',
reason: '労働災害防止のため、クレーンなどの安全を呼びかける日として業界が制定した。',
),
'10-1': DayOccasion(
name: '法の日',
reason: '1960年、政府が10月1日を法の日とした。司法の役割と権利を国民が考える日。',
alsoKnownAs: ['コーヒーの日'],
),
'10-2': DayOccasion(
name: '豆腐の日',
reason: '10月の「十」と「豆」を組み合わせで豆腐に見立て、日本豆腐協会が制定した。',
),
'10-3': DayOccasion(
name: '登山の日',
reason: '10と3で「とざん」と読む語呂合わせ。秋の高山を安全に楽しむ日。',
),
'10-4': DayOccasion(
```

```
name: '天使の日',
reason: '10月4日はアッシジの聖フランチェスコの記念日で、動物愛護の日としても世界で知られる。',
),
'10-5': DayOccasion(
name: '時計の日',
reason: '660年ごろ天智天皇が漏刻を置いた故事と、10月の「時の記念日」の秋版として時計業界が用いる。',
),
'10-6': DayOccasion(
name: '国際協力の日',
reason: '1954年10月6日、日本がコロンボ・プランに加盟し、政府開発援助を始めた。',
),
'10-7': DayOccasion(
name: 'ミステリー記念日',
reason: '1849年10月7日、エドガー・アラン・ポーが亡くなった。推理小説の父を偲ぶ日。',
),
'10-8': DayOccasion(
name: '入れ歯の日',
reason: '10と8で「いれば」と読む語呂合わせ。口腔ケアを呼びかける歯科関連の記念日。',
),
'10-9': DayOccasion(
name: '世界郵便デー',
reason: '1874年10月9日、万国郵便連合が創設された。手紙と国際郵便の協力を記念する。',
),
'10-10': DayOccasion(
name: '目の愛護デー',
reason: '10を横に倒すと眉と目に見えることから、視力を大切にすることから、厚生労働省などが啓発。',
),
'10-11': DayOccasion(
name: 'ウィンナーソーセージの日',
reason: 'ソーセージの原材料や食文化を秋にPRする日として食肉加工業が制定した。',
),
'10-12': DayOccasion(
name: '豆乳の日',
reason: '10と12で「とうにゅう」に通じる語呂。大豆の栄養を伝える日。',
),
'10-13': DayOccasion(
name: 'さつまいもの日',
reason: '川越いもなど秋の味覚。13日は「いも」の語呂に近い日として産地がPRする。',
),
'10-14': DayOccasion(
name: '鉄道の日',
reason: '1872年10月14日、新橋～横浜間で日本初の鉄道が開業した（新暦）。鉄道の誕生日。',
),
'10-15': DayOccasion(
name: 'きのこの日',
reason: '日本特用林産振興会が制定。秋の味覚きのこを食べる日。',
),
'10-16': DayOccasion(
name: '世界食糧デー',
reason: '1945年10月16日、国連食糧農業機関（FAO）が創設された。飢えと食料を考える国際的な日。',
),
'10-17': DayOccasion(
name: '貯蓄の日',
reason: '1983年、10月17日を貯蓄の日とした。将来に備える家計の習慣を呼びかける。',
),
'10-18': DayOccasion(
name: '統計の日',
reason: '1973年、総理府が10月18日を統計の日とした。数字で社会を見る大切さを伝える。',
),
'10-19': DayOccasion(
name: '海外旅行の日',
reason: '1964年4月の海外渡航自由化の流れを秋に記念し、旅の安全と相互理解を考える日。',
),
'10-20': DayOccasion(
name: 'リサイクルの日',
reason: '環境庁（当時）が、ごみの再資源化を呼びかける日として啓発した。',
),
'10-21': DayOccasion(
name: 'あかりの日',
reason: '1879年10月21日、エジソンが白熱電球の実験に成功した。照明の歴史を記念する。',
),
'10-22': DayOccasion(
name: '平安遷都の日',
reason: '794年10月22日、桓武天皇が平安京へ遷都した。京都の誕生日。',
),
'10-23': DayOccasion(
name: '津軽弁の日',
```

```
      reason: '10と23で「つ（10）がる（23）」と読む語呂合わせ。青森の方言文化を大切にする日。',
    ),
    '10-24': DayOccasion(
      name: '国連の日',
      reason: '1945年10月24日、国連憲章が発効した。国際平和と協力を考える日。',
    ),
    '10-25': DayOccasion(
      name: '漫画の日',
      reason: '1952年10月25日、『鉄腕アトム』の連載が始まったとされる日として、漫画文化を祝う動きがある。',
    ),
    '10-26': DayOccasion(
      name: '原子力の日',
      reason: '1963年10月26日、日本で初めて原子力発電に成功した。エネルギー政策を考える日。',
    ),
    '10-27': DayOccasion(
      name: '世界新記録の日',
      reason: 'スポーツの秋。記録更新の偉業を振り返り、挑戦する心を養う日として使われる。',
    ),
    '10-28': DayOccasion(
      name: '速記記念日',
      reason: '1882年10月28日、日本で速記法が公開された。言葉を正確に残す技術の日。',
    ),
    '10-29': DayOccasion(
      name: 'ホームビデオ記念日',
      reason: '家庭用ビデオカメラが普及し始めた秋。記録と記憶のメディアを考える日。',
    ),
    '10-30': DayOccasion(
      name: '初恋の日',
      reason: '詩人・島崎藤村の『初恋』が秋のイメージで読まれることから、若い感情を大切にする日として語られる。',
    ),
    '10-31': DayOccasion(
      name: 'ハロウィン',
      reason: '古代ケルトの収穫祭が起源。日本では仮装と菓子を楽しむ秋の行事として定着した。',
    ),
    '11-1': DayOccasion(
      name: '古典の日',
      reason: '1008年11月1日ごろ、『源氏物語』が文献に現れたとされる。紫式部と古典に親しむ日。',
    ),
    '11-2': DayOccasion(
      name: '阪神タイガース記念日',
      reason: '1936年、大阪タイガース（のち阪神）が公式戦を始めた歴史を秋に振り返るファンの日。',
    ),
    '11-3': DayOccasion(
      name: '文化の日',
      reason: '憲法公布の日を起源とする国民の祝日。自由・平和・文化をすすめる。',
    ),
    '11-4': DayOccasion(
      name: 'ユネスコ憲章記念日',
      reason: '1946年11月4日、ユネスコ憲章が発効した。教育・科学・文化の国際協力の日。',
    ),
    '11-5': DayOccasion(
      name: '津波防災の日',
      reason: '1854年11月5日、安政南海地震で浜口梧陵が稲むらに火をつけ住民を救った故事。世界津波の日でもある。',
    ),
    '11-6': DayOccasion(
      name: 'アパート記念日',
      reason: '日本に近代アパートが広まった明治・大正の住宅史を振り返り、住まいを考える日。',
    ),
    '11-7': DayOccasion(
      name: '鍋の日',
      reason: '11月7日は立冬のころ。冬の始めに鍋料理で体を温める日として食卓に定着した。',
    ),
    '11-8': DayOccasion(
      name: 'いい歯の日',
      reason: '11と8で「いい歯」と読む語呂合わせ。日本歯科医師会が歯の健康を啓発する日。',
    ),
    '11-9': DayOccasion(
      name: '換気の日',
      reason: '11と9で「いい（11）くうき（9＝く）」に通じ、空気の入替えを呼びかける日。',
    ),
    '11-10': DayOccasion(
      name: '技能の日',
      reason: '1963年、技能尊重の機運を高める日として制定。ものづくりの技術をたたえる。',
    ),
    '11-11': DayOccasion(
      name: 'ポッキーの日',
      reason: '11が細い棒のように並ぶ見た目から、江崎グリコがポッキー&ブリッツの日とした。第一次世界大戦休戦記念日でもある。',
```

```
      alsoKnownAs: ['介護の日'],
    ),
    '11-12': DayOccasion({
      name: '皮膚の日',
      reason: '11と12で「いい（11）ひふ（12）」と読む語呂合わせ。日本皮膚科学会が啓発する。',
    }),
    '11-13': DayOccasion({
      name: 'うるしの日',
      reason: '11と13で「いい（11）うるし」に通じるとして、漆器文化を守る日。',
    }),
    '11-14': DayOccasion({
      name: '世界糖尿病デー',
      reason: 'インスリンを発見したバンティングの誕生日。青いサークルで予防と治療を呼びかける。',
    }),
    '11-15': DayOccasion({
      name: '七五三',
      reason: '3歳・5歳・7歳の成長を祝う日本の行事。11月15日は鬼が居ぬ日とされる風習から選ばれた。',
    }),
    '11-16': DayOccasion({
      name: '幼稚園記念日',
      reason: '1876年11月16日、東京女子師範学校に附属幼稚園（日本初の幼稚園）が開園した。',
    }),
    '11-17': DayOccasion({
      name: '将棋の日',
      reason: '日本将棋連盟が制定。11月17日は「いい（11）いな（17＝いな＝盤）」より、将棋に親しむ日。',
    }),
    '11-18': DayOccasion({
      name: '音楽の日（録音）',
      reason: '1877年11月ごろエジソンが蓄音機で録音に成功した歴史に因み、音を残す文化を考える日。',
    }),
    '11-19': DayOccasion({
      name: '農協記念日',
      reason: '1948年11月19日、農業協同組合法が施行された。地域農業の協同を考える日。',
    }),
    '11-20': DayOccasion({
      name: '世界子どもの日',
      reason: '国連が制定。1959年の児童権利宣言に由来し、子どもの権利を守る国際的な日。',
    }),
    '11-21': DayOccasion({
      name: 'インターネット記念日',
      reason: '1969年11月21日、ARPANETの最初の常設リンクがつながったとされる。ネット社会の起源。',
    }),
    '11-22': DayOccasion({
      name: 'いい夫婦の日',
      reason: '11と22で「いいふうふ」と読む語呂合わせ。夫婦の絆を見直す日として広まった。',
    }),
    '11-23': DayOccasion({
      name: '勤労感謝の日',
      reason: '新嘗祭を起源とする国民の祝日。働くことと収穫に感謝する。',
      alsoKnownAs: ['ゲームの日'],
    }),
    '11-24': DayOccasion({
      name: '演芸の日',
      reason: '落語や講談など、冬の寄席文化に親しむ日。言葉の芸を聞く季節。',
    }),
    '11-25': DayOccasion({
      name: '女子大の日',
      reason: '日本女子大学創立（1901年）などに因み、女性の高等教育を考える日として語られる。',
    }),
    '11-26': DayOccasion({
      name: 'いい風呂の日',
      reason: '11と26で「いいふろ」と読む語呂合わせ。温泉・入浴業界が制定し、入浴の効能を伝える。',
    }),
    '11-27': DayOccasion({
      name: 'ノーベル賞制定記念',
      reason: '1895年11月27日、アルフレッド・ノーベルが遺言に賞の創設を記した。科学と平和を考える日。',
    }),
    '11-28': DayOccasion({
      name: '税関記念日',
      reason: '1952年11月28日、関税法が施行された。貿易の公正を守る仕事を知る日。',
    }),
    '11-29': DayOccasion({
      name: 'いい肉の日',
      reason: '11と29で「いいにく」と読む語呂合わせ。食肉業界が安全な肉食をPRする日。',
    }),
    '11-30': DayOccasion({
      name: '本を読む日',
```



```
    reason: '11月は読書月間。30日はその締めくくりとして、一年の学びを本で振り返る日。',
  ),
  '12-1': DayOccasion(
    name: '世界エイズデー',
    reason: 'WHOが制定。HIV/AIDSへの正しい知識と偏見のない支援を呼びかける国際的な日。',
  ),
  '12-2': DayOccasion(
    name: '原子炉の日',
    reason: '1942年12月2日、フェルミらがシカゴで最初の人工原子炉を臨界させた。原子力の始まり。',
  ),
  '12-3': DayOccasion(
    name: '国際障害者デー',
    reason: '国連が制定。障害のある人の権利と社会参加をすすめる国際的な日。',
  ),
  '12-4': DayOccasion(
    name: '世界チーターの日',
    reason: '絶滅が危惧されるチーターの保護を呼びかける国際的な啓発日。野生動物と生息地を考える。',
  ),
  '12-5': DayOccasion(
    name: '国際ボランティアデー',
    reason: '国連が制定。無償の貢献が社会を支えることを伝える日。',
  ),
  '12-6': DayOccasion(
    name: '音の日',
    reason: '1877年12月6日、エジソンが蓄音機の特許を出願した。日本音響学会などが制定。',
  ),
  '12-7': DayOccasion(
    name: 'クリスマスツリーの日',
    reason: '1926年12月7日、銀座に日本初の公開クリスマスツリーが飾られたとされる。',
  ),
  '12-8': DayOccasion(
    name: '事始め',
    reason: 'この日から正月の準備を始める習わし。仏滅ではなく、煤払いの季節の入口。',
  ),
  '12-9': DayOccasion(
    name: '国際腐敗防止デー',
    reason: '国連が制定。公正な社会と透明性を考える国際的な日。',
  ),
  '12-10': DayOccasion(
    name: '世界人権デー',
    reason: '1948年12月10日、世界人権宣言が採択された。すべての人の尊厳を守る日。',
  ),
  '12-11': DayOccasion(
    name: '胃腸の日',
    reason: '12と11で「いい（11）胃（1・2）」に通じ、年末の暴飲暴食を戒める啓発に使われる。',
  ),
  '12-12': DayOccasion(
    name: '漢字の日',
    reason: '12と12で「いい（1）じ（2）いいじ」＝「良い字」と読む語呂合わせ。漢字能力検定協会が制定。',
  ),
  '12-13': DayOccasion(
    name: 'ビタミンの日',
    reason: '1910年、鈴木梅太郎がオリザニン（ビタミンB1）を発表した業績を冬に記念し、栄養を考える日。',
  ),
  '12-14': DayOccasion(
    name: '南極の日',
    reason: '1911年12月14日、アムンセン隊が南極点に到達した。探検史の記念日。',
  ),
  '12-15': DayOccasion(
    name: '観光バス記念日',
    reason: '1925年12月15日、東京で遊覧バスが運行を始めた。観光交通の始まり。',
  ),
  '12-16': DayOccasion(
    name: '電話の日',
    reason: '1890年12月16日、東京～横浜間で電話交換業務が始まった。日本の電話開通記念日。',
  ),
  '12-17': DayOccasion(
    name: '飛行機の日',
    reason: '1903年12月17日、ライト兄弟が初の動力飛行に成功した。航空の誕生日。',
  ),
  '12-18': DayOccasion(
    name: '東京駅の日',
    reason: '1914年12月18日、東京駅が開業した。日本の玄関駅の誕生日。',
  ),
  '12-19': DayOccasion(
    name: '日本初飛行の日',
    reason: '1910年12月19日、徳川好敏大尉が日本で初めて公式の動力飛行に成功した。',
```

```
),
'12-20': DayOccasion(
    name: '霧の日',
    reason: '冬の放射霧が出やすい時期。交通安全と気象を意識する日。',
),
'12-21': DayOccasion(
    name: 'クロスワードの日',
    reason: '1913年12月21日、世界初のクロスワード・パズルが新聞に載った。言葉遊びの誕生日。',
),
'12-22': DayOccasion(
    name: '冬至',
    reason: '一年で昼が最も短い日。ゆず湯と南瓜で厄を払い、これから日が長くなることを祝う。',
),
'12-23': DayOccasion(
    name: '東京タワー開業記念日',
    reason: '1958年12月23日、東京タワーが開業した。戦後復興の象徴を記念する。',
),
'12-24': DayOccasion(
    name: 'クリスマス・イブ',
    reason: 'キリストの降誕を前夜から祝う西欧の習慣が、日本ではイルミネーションと贈り物の夜として定着した。',
),
'12-25': DayOccasion(
    name: 'クリスマス',
    reason: 'イエス・キリストの降誕を祝う日。日本では宗教を超えて冬の祝祭として広まった。',
),
'12-26': DayOccasion(
    name: '年末挨拶の日',
    reason: '英連邦の Boxing Day にあたる日でもあり、日本では歳暮や挨拶回りが本格化する年末の区切り。',
),
'12-27': DayOccasion(
    name: 'ピーター・パンの日',
    reason: '1904年12月27日、戯曲『ピーター・パン』が初演された。永遠の少年の物語の誕生日。',
),
'12-28': DayOccasion(
    name: '御用納め',
    reason: '官庁の仕事納めが12月28日とされる。一年の公務を締め、正月を迎える日本の行政の区切り。',
),
'12-29': DayOccasion(
    name: '年の瀬',
    reason: '大晦日を前に、一年の片づけと来年の準備を急ぐ日。商家では勘定を合わせる「締め」の時期でもある。',
),
'12-30': DayOccasion(
    name: '地下鉄記念日',
    reason: '1927年12月30日、東京地下鉄道（銀座線）が開業した。日本初の地下鉄の誕生日。',
),
'12-31': DayOccasion(
    name: '大晦日',
    reason: '一年の最後の日。年越しそばや除夜の鐘で、一年の穢れを払い新年を迎える日本の習わし。',
),
};
```

ファイル: lib/screens/app_shell.dart

行数: 51

```
import 'package:flutter/material.dart';

import '../data/app_store.dart';
import '../widgets/nexus_nav_bar.dart';
import 'home/home_screen.dart';
import 'life/life_screen.dart';
import 'money/money_screen.dart';
import 'settings/settings_screen.dart';
import 'study/study_screen.dart';

class AppShell extends StatelessWidget {
  const AppShell({super.key});

  @override
  Widget build(BuildContext context) {
    final store = AppScope.of(context);
    final bottom = MediaQuery.paddingOf(context).bottom;

    return Scaffold(
      backgroundColor: Colors.transparent,
      extendBody: true,
      body: Stack(
        children: [
          Padding(
            padding: EdgeInsets.only(bottom: 70 + (bottom > 0 ? bottom : 10)),
            child: IndexedStack(
              index: store.tabIndex,
              children: const [
                HomeScreen(),
                StudyScreen(),
                LifeScreen(),
                MoneyScreen(),
                SettingsScreen(),
              ],
            ),
          ),
          Positioned(
            left: 0,
            right: 0,
            bottom: 0,
            child: NexusNavBar(
              currentIndex: store.tabIndex,
              onTap: store.goTo,
            ),
          ),
        ],
      ),
    );
  }
}
```

ファイル: lib/screens/home/home_screen.dart

行数: 357

```
import 'dart:async';

import 'package:flutter/material.dart';

import '../..../app/theme.dart';
import '../..../core/format.dart';
import '../..../data/app_store.dart';
import '../..../domain/daily_quotes.dart';
import '../..../domain/day_occasions.dart';
import '../..../widgets/glass_card.dart';
import '../..../widgets/schedule_sheet.dart';
import '../..../widgets/ui_bits.dart';
import 'home_header.dart';
import 'home_widgets.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  Timer? _tick;

  @override
  void initState() {
    super.initState();
    _tick = Timer.periodic(const Duration(seconds: 1), (_) {
      if (!mounted) return;
      final store = AppScope.of(context);
      if (store.timerRunning) setState(() {});
    });
  }

  @override
  void dispose() {
    _tick?.cancel();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    final store = AppScope.of(context);

    return PageScaffold(
      child: ListView(
        padding: const EdgeInsets.fromLTRB(16, 8, 16, 20),
        children: [
          const HomeHeader(),
          const SizedBox(height: 14),
          const _HeroRow(),
          const SizedBox(height: 12),
          _ScheduleCard(store: store),
          const SizedBox(height: 12),
          const HomeWidgetCarousel(),
        ],
      ),
    );
  }
}

class _HeroRow extends StatelessWidget {
  const _HeroRow();

  @override
  Widget build(BuildContext context) {
    final store = AppScope.of(context);
    final occasion = store.todayOccasion;
```

```

final quote = store.todayQuote;

return Row(
  children: [
    Expanded(
      child: InkWell(
        onTap: () => _openOccasion(context, store.focusedDate, occasion),
        borderRadius: BorderRadius.circular(NexusColors.cardRadius),
        child: GlassCard(
          fill: NexusColors.sky,
          padding: const EdgeInsets.fromLTRB(14, 12, 14, 14),
          child: SizedBox(
            height: 92,
            child: Stack(
              children: [
                Positioned(
                  right: -18,
                  top: -10,
                  child: Container(
                    width: 90,
                    height: 90,
                    decoration: BoxDecoration(
                      shape: BoxShape.circle,
                      gradient: RadialGradient(
                        colors: [
                          NexusColors.cyan.withValues(alpha: 0.35),
                          Colors.transparent,
                        ],
                      ),
                    ),
                  ),
                ),
              ],
            ),
          ),
          Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              const Text(
                '今日は',
                style: TextStyle(
                  color: NexusColors.textSecondary,
                  fontSize: 12,
                ),
              ),
              const Spacer(),
              Text(
                occasion.name,
                maxLines: 2,
                overflow: TextOverflow.ellipsis,
                style: TextStyle(
                  color: NexusColors.text,
                  fontSize: occasion.name.length >= 8 ? 16 : 22,
                  fontWeight: FontWeight.w600,
                  letterSpacing: -0.4,
                  height: 1.15,
                ),
              ),
            ],
          ),
        ),
      ),
    ),
    const SizedBox(width: 10),
    Expanded(
      child: InkWell(
        onTap: () => _openQuote(context, quote),
        borderRadius: BorderRadius.circular(NexusColors.cardRadius),
        child: GlassCard(
          fill: NexusColors.peach,
          padding: const EdgeInsets.fromLTRB(14, 12, 14, 14),
          child: SizedBox(
            height: 92,
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                const Text(

```

```

        '今日の名言',
        style: TextStyle(
          color: NexusColors.textSecondary,
          fontSize: 12,
        ),
      ),
      const SizedBox(height: 6),
      Expanded(
        child: Text(
          quote.text,
          maxLines: 2,
          overflow: TextOverflow.ellipsis,
          style: const TextStyle(
            color: NexusColors.text,
            fontSize: 13,
            height: 1.35,
            fontWeight: FontWeight.w700,
          ),
        ),
      ),
      Text(
        '– ${quote.author}',
        maxLines: 1,
        overflow: TextOverflow.ellipsis,
        style: const TextStyle(
          color: NexusColors.purple,
          fontSize: 10,
          fontWeight: FontWeight.w700,
        ),
      ),
    ],
  ),
),
),
),
),
),
],
);
}
}

```

```

Future<void> _openOccasion(
  BuildContext context,
  DateTime date,
  DayOccasion occasion,
) {
  return showNexusSheet<void>(
    context: context,
    builder: (_) => Column(
      mainAxisAlignment: MainAxisAlignment.min,
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(
          occasion.name,
          style: const TextStyle(
            color: NexusColors.text,
            fontSize: 20,
            fontWeight: FontWeight.w600,
          ),
        ),
        const SizedBox(height: 6),
        Text(
          '${jpDate(date)} · ${occasion.kind}',
          style: const TextStyle(color: NexusColors.cyan, fontSize: 12),
        ),
        if (occasion.alsoKnownAs.isNotEmpty) ...[
          const SizedBox(height: 6),
          Text(
            'あわせて ${occasion.alsoKnownAs.join('、')}',
            style: const TextStyle(color: NexusColors.textMuted, fontSize: 12),
          ),
        ],
        const SizedBox(height: 14),
        const Text(
          '由来',
          style: TextStyle(
            color: NexusColors.textSecondary,

```

```

        fontSize: 12,
        fontWeight: FontWeight.w700,
      ),
    ),
    const SizedBox(height: 6),
    Text(
      occasion.reason,
      style: const TextStyle(
        color: NexusColors.text,
        height: 1.55,
        fontSize: 14,
      ),
    ),
  ),
  const SizedBox(height: 8),
],
),
);
}

```

```

Future<void> _openQuote(BuildContext context, DailyQuote quote) {
  return showNexusSheet<void>(
    context: context,
    builder: (_) => Column(
      mainAxisAlignment: MainAxisAlignment.start,
      children: [
        const Text(
          '今日の名言',
          style: TextStyle(color: NexusColors.textSecondary, fontSize: 12),
        ),
        const SizedBox(height: 10),
        Text(
          quote.text,
          style: const TextStyle(
            color: NexusColors.text,
            fontSize: 18,
            fontWeight: FontWeight.w700,
            height: 1.45,
          ),
        ),
        const SizedBox(height: 10),
        Text(
          '- ${quote.author}',
          style: const TextStyle(
            color: NexusColors.purple,
            fontWeight: FontWeight.w700,
          ),
        ),
        const SizedBox(height: 14),
        Text(
          quote.note,
          style: const TextStyle(
            color: NexusColors.textSecondary,
            height: 1.5,
            fontSize: 13,
          ),
        ),
        const SizedBox(height: 8),
      ],
    ),
  );
}

```

```

class _ScheduleCard extends StatelessWidget {
  const _ScheduleCard({required this.store});

  final AppStore store;

  @override
  Widget build(BuildContext context) {
    final items = store.schedulesOn(store.focusedDate);

    return GlassCard(
      padding: const EdgeInsets.fromLTRB(14, 12, 10, 12),
      child: Column(
        children: [
          SectionRow(

```

```

        title: '今日の予定',
        trailing: AddChip(
          label: '予定を追加',
          onTap: () => openScheduleEditor(context),
        ),
      ),
    ),
    const SizedBox(height: 10),
    SizedBox(
      height: 148,
      child: items.isEmpty
        ? const Center(
            child: EmptyHint(
              text: '予定はまだありません',
              icon: Icons.event_available_outlined,
            ),
          )
        : ListView.separated(
            itemCount: items.length,
            separatorBuilder: (_, _) => const SizedBox(height: 8),
            itemBuilder: (context, index) {
              final item = items[index];
              return InkWell(
                onTap: () => openScheduleEditor(context, item: item),
                borderRadius: BorderRadius.circular(14),
                child: Container(
                  padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 10),
                  decoration: BoxDecoration(
                    color: NexusColors.sky.withValues(alpha: 0.55),
                    borderRadius: BorderRadius.circular(14),
                  ),
                  child: Row(
                    children: [
                      TimePill(hm(item.startAt)),
                      const SizedBox(width: 10),
                      Expanded(
                        child: Text(
                          item.title,
                          style: const TextStyle(
                            color: NexusColors.text,
                            fontSize: 14,
                            fontWeight: FontWeight.w700,
                          ),
                        ),
                      ),
                    ],
                  ),
                ),
              ),
            ],
          ),
        );
  },
),
],
),
);
}
}

```


ファイル: lib/screens/home/home_header.dart

行数: 145

```
import 'package:flutter/material.dart';

import '../..../app/theme.dart';
import '../..../core/format.dart';
import '../..../data/app_store.dart';
import '../..../widgets/nexus_logo.dart';
import '../..../widgets/nexus_nav_bar.dart';

class HomeHeader extends StatelessWidget {
  const HomeHeader({super.key});

  @override
  Widget build(BuildContext context) {
    final store = AppScope.of(context);

    return Row(
      children: [
        const NexusLogo(size: 46),
        const SizedBox(width: 10),
        Expanded(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(
                store.userName,
                style: const TextStyle(
                  color: NexusColors.text,
                  fontSize: 15,
                  fontWeight: FontWeight.w600,
                  letterSpacing: 0.2,
                ),
              ),
              const SizedBox(height: 2),
              Row(
                children: [
                  Text(
                    'Lv.${store.level}',
                    style: const TextStyle(
                      color: NexusColors.gold,
                      fontSize: 11,
                      fontWeight: FontWeight.w600,
                      letterSpacing: 0.4,
                    ),
                  ),
                  const SizedBox(width: 6),
                  Expanded(
                    child: ClipRRect(
                      borderRadius: BorderRadius.circular(4),
                      child: LinearProgressIndicator(
                        value: store.levelProgress,
                        minHeight: 4,
                        backgroundColor: NexusColors.border,
                        color: NexusColors.gold,
                      ),
                    ),
                  ),
                ],
              ),
              const SizedBox(width: 6),
              Expanded(
                child: ClipRRect(
                  borderRadius: BorderRadius.circular(4),
                  child: LinearProgressIndicator(
                    value: store.levelProgress,
                    minHeight: 4,
                    backgroundColor: NexusColors.border,
                    color: NexusColors.gold,
                  ),
                ),
              ),
            ],
          ),
        ),
        const SizedBox(width: 8),
        Flexible(
          child: FittedBox(
            fit: BoxFit.scaleDown,
            alignment: Alignment.centerRight,
            child: _DateChip(
              label: jpDate(store.focusedDate),
              onTap: () async {
```

```

        final picked = await showDatePicker(
          context: context,
          initialDate: store.focusedDate,
          firstDate: DateTime(store.focusedDate.year - 1, 1, 1),
          lastDate: DateTime(store.focusedDate.year + 1, 12, 31),
        );
        if (picked != null) store.setFocusedDate(picked);
      },
    ),
  ),
),
const SizedBox(width: 8),
InkWell(
  onTap: () => store.goTo(NexusTab.settings),
  borderRadius: BorderRadius.circular(10),
  child: Container(
    width: 36,
    height: 36,
    decoration: BoxDecoration(
      color: NexusColors.sky,
      borderRadius: BorderRadius.circular(12),
    ),
    child: const Icon(
      Icons.push_pin_rounded,
      size: 16,
      color: NexusColors.cyan,
    ),
  ),
),
],
);
}
}

class _DateChip extends StatelessWidget {
  const _DateChip({required this.label, required this.onTap});

  final String label;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    return Material(
      color: Colors.transparent,
      child: InkWell(
        onTap: onTap,
        borderRadius: BorderRadius.circular(20),
        child: Container(
          padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 8),
          decoration: BoxDecoration(
            color: NexusColors.cyan.withValues(alpha: 0.08),
            borderRadius: BorderRadius.circular(20),
            border: Border.all(color: NexusColors.cyan.withValues(alpha: 0.28)),
          ),
          child: Row(
            children: [
              const Icon(Icons.calendar_month_rounded, size: 14, color: NexusColors.cyan),
              const SizedBox(width: 6),
              Text(
                label,
                style: const TextStyle(
                  color: NexusColors.text,
                  fontSize: 11,
                  fontWeight: FontWeight.w600,
                ),
              ),
              const SizedBox(width: 2),
              const Icon(Icons.expand_more_rounded, size: 16, color: NexusColors.textMuted),
            ],
          ),
        ),
      ),
    );
  }
}

```

ファイル: lib/screens/home/home_widgets.dart

行数: 331

```
import 'package:flutter/material.dart';

import '../..../app/theme.dart';
import '../..../core/format.dart';
import '../..../data/app_store.dart';
import '../..../widgets/count_up_yen.dart';
import '../..../widgets/glass_card.dart';
import '../..../widgets/nexus_nav_bar.dart';
import '../..../widgets/progress_ring.dart';
import '../..../widgets/ui_bits.dart';

class HomeWidgetCarousel extends StatelessWidget {
  const HomeWidgetCarousel({super.key});

  @override
  Widget build(BuildContext context) {
    final store = AppScope.of(context);
    final money = store.money;

    return Column(
      children: [
        Row(
          children: [
            Expanded(
              child: _GoalCard(
                progress: store.goalProgress,
                remainingLabel: remainingStudyLabel(store.remainingStudyMinutes()),
                goalLabel: studyGoalLabel(store.dailyStudyGoalMinutes),
                onTap: () => _openStudyGoalEditor(context, store),
              ),
            ),
            const SizedBox(width: 10),
            Expanded(
              child: _BalanceCard(
                balance: money.balance,
                income: money.income,
                expense: money.expense,
                onTap: () => store.goTo(NexusTab.money),
              ),
            ),
          ],
        ),
        const SizedBox(height: 10),
        _StudyTimeCard(
          hours: store.weekStudyHours,
          bars: store.weekBars,
          onTap: () => store.goTo(NexusTab.study),
        ),
      ],
    );
  }
}

class WidgetLabel extends StatelessWidget {
  const WidgetLabel({super.key, required this.code, required this.title});

  final String code;
  final String title;

  @override
  Widget build(BuildContext context) {
    return Row(
      children: [
        Container(
          padding: const EdgeInsets.symmetric(horizontal: 6, vertical: 2),
          decoration: BoxDecoration(
            color: NexusColors.cyan.withValues(alpha: 0.12),
            borderRadius: BorderRadius.circular(6),
          ),
        ),
      ],
    );
  }
}
```

```

        child: Text(
          code,
          style: const TextStyle(
            color: NexusColors.cyan,
            fontSize: 10,
            fontWeight: FontWeight.w800,
          ),
        ),
      ),
    ),
    const SizedBox(width: 6),
    Expanded(
      child: Text(
        title,
        overflow: TextOverflow.ellipsis,
        style: const TextStyle(
          color: NexusColors.textSecondary,
          fontSize: 12,
          fontWeight: FontWeight.w600,
        ),
      ),
    ),
  ],
);
}
}

class _GoalCard extends StatelessWidget {
  const _GoalCard({
    required this.progress,
    required this.remainingLabel,
    required this.goallabel,
    required this.onTap,
  });

  final double progress;
  final String remainingLabel;
  final String goallabel;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    final done = remainingLabel == '達成';
    return InkWell(
      onTap: onTap,
      borderRadius: BorderRadius.circular(NexusColors.cardRadius),
      child: GlassCard(
        height: 168,
        fill: NexusColors.sky,
        child: Column(
          children: [
            const WidgetLabel(code: 'W01', title: '今日の目標'),
            Expanded(
              child: Center(
                child: ProgressRing(
                  progress: progress,
                  size: 88,
                  stroke: 9,
                  child: Column(
                    mainAxisAlignment: MainAxisAlignment.min,
                    children: [
                      Text(
                        done ? '達成' : '残り',
                        style: const TextStyle(color: NexusColors.textMuted, fontSize: 10),
                      ),
                      Text(
                        done ? '' : remainingLabel,
                        textAlign: TextAlign.center,
                        style: TextStyle(
                          color: NexusColors.text,
                          fontSize: remainingLabel.length >= 6 ? 13 : 16,
                          fontWeight: FontWeight.w800,
                          height: 1.05,
                        ),
                      ),
                    ],
                  ),
                ),
              ),
            ],
          ),
        ),
      ),
    ),
  ),
);

```

```

    ),
  ),
  Text('目標 $goalLabel', style: const TextStyle(color: NexusColors.textMuted, fontSize: 11)),
],
),
),
);
}
}

Future<void> _openStudyGoalEditor(BuildContext context, AppStore store) async {
  const presets = [30, 60, 90, 120, 180, 240];
  await showNexusSheet<void>(
    context: context,
    builder: (context) {
      return StatefulBuilder(
        builder: (context, setSheet) {
          final remaining = remainingStudyLabel(store.remainingStudyMinutes());
          return Column(
            mainAxisAlignment: MainAxisAlignment.min,
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              const Text('今日の勉強時間', style: TextStyle(fontSize: 18, fontWeight: FontWeight.w600)),
              const SizedBox(height: 8),
              Text(
                remaining == '達成' ? '今日の目標は達成しています' : '残り $remaining',
                style: const TextStyle(color: NexusColors.cyan, fontWeight: FontWeight.w700),
              ),
              const SizedBox(height: 14),
              Wrap(
                spacing: 8,
                runSpacing: 8,
                children: [
                  for (final minutes in presets)
                    ChoiceChip(
                      label: Text(studyGoalLabel(minutes)),
                      selected: store.dailyStudyGoalMinutes == minutes,
                      selectedColor: NexusColors.cyan.withValues(alpha: 0.25),
                      labelStyle: TextStyle(
                        color: store.dailyStudyGoalMinutes == minutes ? NexusColors.cyan : NexusColors.text,
                        fontWeight: FontWeight.w700,
                      ),
                      onPressed: () {
                        store.setDailyStudyGoalMinutes(minutes);
                        setSheet(() {});
                      },
                    ),
                ],
              ),
              Slider(
                value: store.dailyStudyGoalMinutes.toDouble(),
                min: 10,
                max: 360,
                divisions: 35,
                label: studyGoalLabel(store.dailyStudyGoalMinutes),
                onChanged: (value) {
                  store.setDailyStudyGoalMinutes(value.round());
                  setSheet(() {});
                },
              ),
              FilledButton(onPressed: () => Navigator.pop(context), child: const Text('閉じる')),
            ],
          );
        },
      );
    },
  );
}

class _StudyTimeCard extends StatelessWidget {
  const _StudyTimeCard({
    required this.hours,
    required this.bars,
    required this.onTap,
  });

  final double hours;

```

```

final List<double> bars;
final VoidCallback onTap;

@override
Widget build(BuildContext context) {
  return InkWell(
    onTap: onTap,
    borderRadius: BorderRadius.circular(NexusColors.cardRadius),
    child: GlassCard(
      height: 256,
      fill: NexusColors.sage,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          const WidgetLabel(code: 'W04', title: '今週の学習時間'),
          Text(
            '${hours}h',
            style: const TextStyle(
              color: NexusColors.text,
              fontSize: 32,
              fontWeight: FontWeight.w800,
              height: 1.05,
              letterSpacing: -0.8,
            ),
          ),
          const SizedBox(height: 8),
          Expanded(
            child: Row(
              crossAxisAlignment: CrossAxisAlignment.end,
              children: [
                for (var i = 0; i < weekLabels.length; i++)
                  Expanded(
                    child: Padding(
                      padding: const EdgeInsets.symmetric(horizontal: 4),
                      child: Column(
                        children: [
                          Expanded(
                            child: TrackBar(value: i < bars.length ? bars[i] : 0),
                          ),
                          const SizedBox(height: 6),
                          Text(
                            weekLabels[i],
                            style: const TextStyle(
                              color: NexusColors.textMuted,
                              fontSize: 11,
                              fontWeight: FontWeight.w600,
                            ),
                          ),
                        ],
                      ),
                    ),
                ],
              ),
            ),
          ),
        ],
      ),
    ),
  );
}

class _BalanceCard extends StatelessWidget {
  const _BalanceCard({
    required this.balance,
    required this.income,
    required this.expense,
    required this.onTap,
  });

  final int balance;
  final int income;
  final int expense;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {

```

```
return InkWell(
  onTap: onTap,
  borderRadius: BorderRadius.circular(NexusColors.cardRadius),
  child: GlassCard(
    height: 168,
    fill: NexusColors.peach,
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        const WidgetLabel(code: 'W13', title: '今月の残高'),
        const Spacer(),
        CountUpYen(
          value: balance,
          style: const TextStyle(
            color: NexusColors.text,
            fontSize: 26,
            fontWeight: FontWeight.w800,
            letterSpacing: -0.6,
          ),
        ),
        const Spacer(),
        StatChip(label: '収入', value: yen(income), color: NexusColors.income),
        const SizedBox(height: 6),
        StatChip(label: '支出', value: yen(-expense), color: NexusColors.expense),
      ],
    ),
  ),
);
```

ファイル: lib/screens/study/study_screen.dart

行数: 1121

```
import 'dart:async';
import 'dart:typed_data';

import 'package:flutter/material.dart';
import 'package:image_picker/image_picker.dart';

import '../..../app/theme.dart';
import '../..../core/format.dart';
import '../..../data/app_store.dart';
import '../..../data/models.dart';
import '../..../domain/money_calc.dart';
import '../..../widgets/duration_picker.dart';
import '../..../widgets/glass_card.dart';
import '../..../widgets/progress_ring.dart';
import '../..../widgets/ui_bits.dart';
import 'focus_timer_page.dart';

class StudyScreen extends StatefulWidget {
  const StudyScreen({super.key});

  @override
  State<StudyScreen> createState() => _StudyScreenState();
}

class _StudyScreenState extends State<StudyScreen> {
  Timer? _tick;

  @override
  void initState() {
    super.initState();
    _tick = Timer.periodic(const Duration(seconds: 1), (_) {
      if (!mounted) return;
      final store = AppScope.of(context);
      if (store.timerRunning) setState(() {});
    });
  }

  @override
  void dispose() {
    _tick?.cancel();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    final store = AppScope.of(context);

    return PageScaffold(
      child: ListView(
        padding: const EdgeInsets.fromLTRB(16, 8, 16, 24),
        children: [
          Row(
            children: [
              const Expanded(child: GradientTitle('Study')),
              AddChip(
                label: '学習を追加',
                onTap: () => _addStudySession(context, store),
              ),
            ],
          ),
          const SizedBox(height: 4),
          Text(
            jpDate(store.focusedDate),
            style: const TextStyle(color: NexusColors.textSecondary, fontSize: 13),
          ),
          const SizedBox(height: 14),
          _TotalStudyCard(store: store),
          const SizedBox(height: 12),
          const SectionRow(title: '科目別・今週の勉強時間'),
        ],
      ),
    );
  }
}
```



```

const SizedBox(height: 8),
if (store.subjects.isEmpty)
  const SizedBox(
    height: 108,
    child: Center(
      child: EmptyHint(text: '教科はまだありません', icon: Icons.menu_book_outlined),
    ),
  )
else
  SizedBox(
    height: 108,
    child: ListView.separated(
      scrollDirection: Axis.horizontal,
      itemCount: store.subjects.length,
      separatorBuilder: (_, _) => const SizedBox(width: 10),
      itemBuilder: (context, i) {
        final s = store.subjects[i];
        return GlassCard(
          fill: s.color.withValues(alpha: 0.12),
          borderColor: s.color.withValues(alpha: 0.2),
          child: SizedBox(
            width: 132,
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Icon(s.icon, color: s.color, size: 18),
                const Spacer(),
                Text(s.name, style: const TextStyle(fontWeight: FontWeight.w700)),
                Text(
                  '${s.weekHours}h',
                  style: TextStyle(color: s.color, fontWeight: FontWeight.w600),
                ),
              ],
            ),
          ),
        );
      },
    ),
  ),
const SizedBox(height: 12),
GlassCard(
  child: Column(
    children: [
      SectionRow(
        title: '提出物',
        trailing: AddChip(
          label: '提出物を追加',
          onTap: () => _addAssignment(context, store),
        ),
      ),
      const SizedBox(height: 8),
      if (store.assignments.isEmpty)
        const Padding(
          padding: EdgeInsets.symmetric(vertical: 12),
          child: EmptyHint(text: '提出物はまだありません', icon: Icons.assignment_outlined),
        ),
      for (final a in store.assignments)
        _AssignmentRow(assignment: a, today: store.focusedDate, store: store),
    ],
  ),
),
const SizedBox(height: 12),
_TimerCard(store: store),
const SizedBox(height: 12),
GlassCard(
  fill: NexusColors.peach,
  child: Row(
    children: [
      Expanded(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const Text('復習カード', style: TextStyle(fontWeight: FontWeight.w700)),
            const SizedBox(height: 4),
            Text(
              '今日のキュー ${store.reviewDueCount()}枚',
              style: const TextStyle(color: NexusColors.textSecondary),
            ),
          ],
        ),
      ),
    ],
  ),
),

```

```

    ),
  ],
),
),
FilledButton(
  onPressed: () => _openReview(context, store),
  child: const Text('復習する'),
),
],
),
),
const SizedBox(height: 12),
GlassCard(
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      SectionRow(
        title: '目標ロードマップ',
        trailing: AddChip(
          label: '目標を追加',
          onTap: () => _addGoal(context, store),
        ),
      ),
      const SizedBox(height: 8),
      if (store.goals.isEmpty)
        const EmptyHint(text: '目標はまだありません', icon: Icons.flag_outlined),
      for (final goal in store.goals) ...[
        _GoalBlock(goal: goal, store: store),
        const SizedBox(height: 12),
      ],
    ],
  ),
),
const SizedBox(height: 12),
GlassCard(
  child: Column(
    children: [
      SectionRow(
        title: '試験カウントダウン',
        trailing: AddChip(
          label: '試験日を追加',
          onTap: () => _addExam(context, store),
        ),
      ),
      const SizedBox(height: 12),
      if (store.exams.isEmpty)
        const EmptyHint(text: '試験日はまだありません', icon: Icons.timer_outlined)
      else
        SizedBox(
          height: 128,
          child: ListView.separated(
            scrollDirection: Axis.horizontal,
            itemCount: store.exams.length,
            separatorBuilder: (_, _) => const SizedBox(width: 12),
            itemBuilder: (context, i) {
              return SizedBox(
                width: 96,
                child: _ExamRing(exam: store.exams[i], today: store.focusedDate),
              );
            },
          ),
        ),
    ],
  ),
),
const SizedBox(height: 12),
GlassCard(
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      const SectionRow(title: '今日の問題'),
      const SizedBox(height: 4),
      const Text(
        '写真で記録すると、1日後と5日後の復習カードが作られます。',
        style: TextStyle(color: NexusColors.textMuted, fontSize: 12),
      ),
      const SizedBox(height: 10),
    ],
  ),
),

```

```

        for (final p in store.problems) _ProblemRow(problem: p, store: store),
        const SizedBox(height: 8),
        AddChip(label: '問題を記録', onTap: () => _recordProblem(context, store)),
    ],
),
),
],
),
);
}
}

class _TotalStudyCard extends StatelessWidget {
  const _TotalStudyCard({required this.store});

  final AppStore store;

  @override
  Widget build(BuildContext context) {
    return GlassCard(
      fill: NexusColors.sage,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          const SectionRow(title: '総勉強時間'),
          const SizedBox(height: 8),
          Row(
            children: [
              _Hours(label: '今週', value: '${store.weekStudyHours}h', color: NexusColors.purple),
              const SizedBox(width: 24),
              _Hours(label: '累計', value: '${store.totalStudyHours}h', color: NexusColors.cyan),
            ],
          ),
          const SizedBox(height: 10),
          Wrap(
            spacing: 10,
            runSpacing: 6,
            children: [
              for (final s in store.subjects)
                Row(
                  mainAxisSize: MainAxisSize.min,
                  children: [
                    Container(
                      width: 8,
                      height: 8,
                      decoration: BoxDecoration(color: s.color, shape: BoxShape.circle),
                    ),
                    const SizedBox(width: 4),
                    Text(s.name, style: const TextStyle(color: NexusColors.textMuted, fontSize: 11)),
                  ],
                ),
            ],
          ),
          const SizedBox(height: 12),
          SizedBox(
            height: 126,
            child: CustomPaint(
              painter: _StackedBarPainter(
                stacks: store.weekStackedHours(),
                colors: [for (final s in store.subjects) s.color],
              ),
            child: const SizedBox.expand(),
          ),
          const SizedBox(height: 6),
          Row(
            children: [
              for (final label in weekLabels)
                Expanded(
                  child: Text(
                    label,
                    textAlign: TextAlign.center,
                    style: const TextStyle(color: NexusColors.textMuted, fontSize: 11),
                  ),
                ),
            ],
          ),
        ],
      ),
    ),
  ),
}

```

```

    ],
  ),
);
}
}

class _StackedBarPainter extends CustomPainter {
  _StackedBarPainter({required this.stacks, required this.colors});

  final List<List<double>> stacks;
  final List<Color> colors;

  @override
  void paint(Canvas canvas, Size size) {
    if (stacks.isEmpty) return;
    var maxTotal = 0.0;
    for (final day in stacks) {
      final total = day.fold<double>(0, (a, b) => a + b);
      if (total > maxTotal) maxTotal = total;
    }
    if (maxTotal <= 0) maxTotal = 1;

    final slot = size.width / stacks.length;
    final barWidth = slot * 0.48;
    for (var i = 0; i < stacks.length; i++) {
      final x = slot * i + (slot - barWidth) / 2;
      var y = size.height;
      for (var s = 0; s < stacks[i].length; s++) {
        final hours = stacks[i][s];
        if (hours <= 0) continue;
        final h = size.height * (hours / maxTotal);
        y -= h;
        final rect = RRect.fromRectAndRadius(
          Rect.fromLTWH(x, y, barWidth, h),
          const Radius.circular(3),
        );
        canvas.drawRRect(
          rect,
          Paint()..color = colors[s % colors.length],
        );
      }
    }
  }

  @override
  bool shouldRepaint(covariant _StackedBarPainter oldDelegate) {
    return oldDelegate.stacks != stacks || oldDelegate.colors != colors;
  }
}

class _TimerCard extends StatelessWidget {
  const _TimerCard({required this.store});

  final AppStore store;

  @override
  Widget build(BuildContext context) {
    return InkWell(
      onTap: () => openFocusTimer(context),
      borderRadius: BorderRadius.circular(16),
      child: GlassCard(
        fill: NexusColors.sky,
        child: Column(
          children: [
            SectionRow(
              title: '集中タイマー',
              trailing: Container(
                width: 36,
                height: 36,
                decoration: const BoxDecoration(color: Colors.white, shape: BoxShape.circle),
                child: const Icon(Icons.play_arrow_rounded, color: NexusColors.cyan),
              ),
            ),
            const SizedBox(height: 8),
            Text(
              mmss(store.timerRemainingSeconds()),
              style: const TextStyle(

```

```

        fontSize: 40,
        fontWeight: FontWeight.w600,
        letterSpacing: 1,
      ),
    ),
    const SizedBox(height: 4),
    const Text(
      'タップして全画面で集中',
      style: TextStyle(color: NexusColors.textMuted, fontSize: 12),
    ),
  ],
),
),
);
}
}

class _AssignmentRow extends StatelessWidget {
  const _AssignmentRow({
    required this.assignment,
    required this.today,
    required this.store,
  });

  final Assignment assignment;
  final DateTime today;
  final AppStore store;

  @override
  Widget build(BuildContext context) {
    final risk = assignmentRisk(dueAt: assignment.dueAt, today: today, done: assignment.done);
    final color = switch (risk) {
      '要注意' => NexusColors.expense,
      '注意' => const Color(0xFFFFC857),
      _ => NexusColors.cyan,
    };
    final subject = store.subjectById(assignment.subjectId)?.name ?? '';
    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 4),
      child: Container(
        padding: const EdgeInsets.fromLTRB(10, 8, 10, 8),
        decoration: BoxDecoration(
          color: color.withValues(alpha: 0.08),
          borderRadius: BorderRadius.circular(12),
        ),
        child: Row(
          children: [
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(assignment.title, style: const TextStyle(fontWeight: FontWeight.w700)),
                  Text(
                    subject,
                    style: const TextStyle(color: NexusColors.textMuted, fontSize: 11),
                  ),
                ],
              ),
            ),
            Column(
              crossAxisAlignment: CrossAxisAlignment.end,
              children: [
                Text(
                  daysLeftLabel(assignment.dueAt, today),
                  style: TextStyle(color: color, fontSize: 12, fontWeight: FontWeight.w800),
                ),
                Text(
                  '${assignment.dueAt.month}/${assignment.dueAt.day}まで',
                  style: const TextStyle(color: NexusColors.textMuted, fontSize: 10),
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}

```

```

}

class _GoalBlock extends StatelessWidget {
  const _GoalBlock({required this.goal, required this.store});

  final StudyGoal goal;
  final AppStore store;

  @override
  Widget build(BuildContext context) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(goal.title, style: const TextStyle(fontWeight: FontWeight.w600, fontSize: 16)),
        const SizedBox(height: 4),
        Text(
          '現在 ${goal.current} / 目標 ${goal.target} / ${jpDate(goal.dueAt)}まで',
          style: const TextStyle(color: NexusColors.textSecondary, fontSize: 12),
        ),
        const SizedBox(height: 8),
        ClipRRect(
          borderRadius: BorderRadius.circular(6),
          child: LinearProgressIndicator(
            value: goal.progress,
            minHeight: 8,
            color: NexusColors.cyan,
            backgroundColor: NexusColors.border,
          ),
        ),
        const SizedBox(height: 8),
        for (var i = 0; i < goal.subGoals.length; i++)
          if (goal.subGoals[i].title.trim().isNotEmpty)
            InkWell(
              onTap: () => store.toggleSubGoal(goal.id, i),
              child: Padding(
                padding: const EdgeInsets.symmetric(vertical: 4),
                child: Row(
                  children: [
                    ThingsCheck(
                      checked: goal.subGoals[i].done,
                      color: NexusColors.cyan,
                      size: 20,
                    ),
                    const SizedBox(width: 8),
                    Text(
                      'サブゴール ${i + 1}  ${goal.subGoals[i].title}',
                      style: TextStyle(
                        color: goal.subGoals[i].done ? NexusColors.textSecondary : NexusColors.text,
                        decoration: goal.subGoals[i].done ? TextDecoration.lineThrough : null,
                      ),
                    ),
                  ],
                ),
              ),
            ),
      ],
    );
  }
}

```

```

class _ExamRing extends StatelessWidget {
  const _ExamRing({required this.exam, required this.today});

  final Exam exam;
  final DateTime today;

  @override
  Widget build(BuildContext context) {
    final days = dateOnly(exam.examAt).difference(dateOnly(today)).inDays;
    final label = days < 0 ? '終了' : days == 0 ? '今日' : '$days日';
    return Container(
      padding: const EdgeInsets.fromLTRB(8, 8, 8, 6),
      decoration: BoxDecoration(
        color: NexusColors.sky.withValues(alpha: 0.75),
        borderRadius: BorderRadius.circular(18),
      ),
      child: Column(

```

```

        children: [
          ProgressRing(
            progress: days <= 0 ? 1 : (1 - days / 40).clamp(0.1, 1),
            size: 78,
            stroke: 6,
            child: Text(
              label,
              style: const TextStyle(fontWeight: FontWeight.w600, fontSize: 13),
            ),
          ),
          const SizedBox(height: 6),
          Text(
            exam.title,
            maxLines: 1,
            overflow: TextOverflow.ellipsis,
            style: const TextStyle(fontSize: 11, fontWeight: FontWeight.w700),
          ),
          Text(
            '${exam.examAt.month}/${exam.examAt.day} (${exam.weekdayLabel}}',
            style: const TextStyle(color: NexusColors.textMuted, fontSize: 10),
          ),
        ],
      ),
    );
  }
}

class _Hours extends StatelessWidget {
  const _Hours({required this.label, required this.value, required this.color});

  final String label;
  final String value;
  final Color color;

  @override
  Widget build(BuildContext context) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(label, style: const TextStyle(color: NexusColors.textMuted, fontSize: 12)),
        Text(value, style: TextStyle(color: color, fontSize: 24, fontWeight: FontWeight.w600)),
      ],
    );
  }
}

class _ProblemRow extends StatelessWidget {
  const _ProblemRow({required this.problem, required this.store});

  final ProblemRecord problem;
  final AppStore store;

  @override
  Widget build(BuildContext context) {
    final subject = store.subjectById(problem.subjectId)?.name ?? '';
    final cards = store.reviewCards.where((c) => c.problemId == problem.id).toList();
    return Padding(
      padding: const EdgeInsets.only(bottom: 10),
      child: SoftTile(
        color: NexusColors.sage.withValues(alpha: 0.7),
        child: Row(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            if (problem.photoBytes != null) ...[
              ClipRRect(
                borderRadius: BorderRadius.circular(8),
                child: Image.memory(
                  problem.photoBytes!,
                  width: 52,
                  height: 52,
                  fit: BoxFit.cover,
                ),
              ),
              const SizedBox(width: 10),
            ],
            Expanded(
              child: Column(

```

```

        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            children: [
              const Icon(Icons.check_circle, color: NexusColors.cyan, size: 16),
              const SizedBox(width: 6),
              Expanded(
                child: Text(
                  '$subject ${problem.title}',
                  style: const TextStyle(fontWeight: FontWeight.w700),
                ),
              ),
            ],
          ),
          const SizedBox(height: 6),
          Text(
            cards.map((c) => '${c.intervalStep}日後').join(' · '),
            style: const TextStyle(color: NexusColors.textMuted, fontSize: 11),
          ),
        ],
      ),
    ],
  ),
);
}
}

```

```

Future<StudySubject?> promptNewSubject(BuildContext context, AppStore store) async {
  final name = TextEditingController();
  final saved = await showNexusSheet<bool>(<
    context: context,
    useRootNavigator: true,
    builder: (context) {
      return Column(
        mainAxisAlignment: MainAxisAlignment.min,
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          const Text('教科を追加', style: TextStyle(fontSize: 18, fontWeight: FontWeight.w600)),
          const SizedBox(height: 12),
          TextField(
            controller: name,
            autofocus: true,
            style: const TextStyle(color: NexusColors.text),
            decoration: const InputDecoration(labelText: '教科名'),
          ),
          const SizedBox(height: 12),
          FilledButton(
            onPressed: () => Navigator.pop(context, true),
            child: const Text('追加'),
          ),
        ],
      );
    },
  );
  final text = name.text.trim();
  name.dispose();
  if (saved == true && text.isNotEmpty) {
    final subject = store.addSubject(name: text);
    if (context.mounted) showNexusToast(context, store.lastToast);
    return subject;
  }
  return null;
}

```

```

Future<void> _addStudySession(BuildContext context, AppStore store) async {
  String? subjectId = store.subjects.isEmpty ? null : store.subjects.first.id;
  var minutes = 30;
  var focus = StudyFocus.high;
  final saved = await showNexusSheet<bool>(<
    context: context,
    builder: (context) {
      return StatefulBuilder(
        builder: (context, setSheet) {
          return SingleChildScrollView(
            child: Column(

```



```

mainAxisSize: MainAxisSize.min,
crossAxisAlignment: CrossAxisAlignment.stretch,
children: [
  const Text('学習を追加', style: TextStyle(fontSize: 18, fontWeight: FontWeight.w600)),
  const SizedBox(height: 12),
  SizedBox(
    height: 40,
    child: ListView(
      scrollDirection: Axis.horizontal,
      children: [
        for (final s in store.subjects) ...[
          ChoiceChip(
            label: Text(s.name),
            selected: subjectId == s.id,
            selectedColor: s.color.withValues(alpha: 0.28),
            labelStyle: TextStyle(
              color: subjectId == s.id ? s.color : NexusColors.text,
              fontWeight: FontWeight.w700,
            ),
            onPressed: (_) => setSheet(() => subjectId = s.id),
          ),
          const SizedBox(width: 8),
        ],
        ActionChip(
          avatar: const Icon(Icons.add, size: 16, color: NexusColors.cyan),
          label: const Text('+ 教科を追加'),
          labelStyle: const TextStyle(
            color: NexusColors.cyan,
            fontWeight: FontWeight.w700,
          ),
          onPressed: () async {
            final created = await promptNewSubject(context, store);
            if (created != null) setSheet(() => subjectId = created.id);
          },
        ),
      ],
    ),
  ),
  const SizedBox(height: 8),
  DurationMinutesPicker(
    minutes: minutes,
    onChanged: (value) => setSheet(() => minutes = value),
  ),
  const SizedBox(height: 8),
  const Text('集中度', style: TextStyle(color: NexusColors.textSecondary, fontSize: 12)),
  const SizedBox(height: 8),
  Wrap(
    spacing: 8,
    children: [
      for (final value in StudyFocus.values)
        ChoiceChip(
          label: Text(value.label),
          selected: focus == value,
          selectedColor: NexusColors.cyan.withValues(alpha: 0.25),
          labelStyle: TextStyle(
            color: focus == value ? NexusColors.cyan : NexusColors.text,
            fontWeight: FontWeight.w700,
          ),
          onPressed: (_) => setSheet(() => focus = value),
        ),
    ],
  ),
  const SizedBox(height: 14),
  FilledButton(
    onPressed: subjectId == null ? null : () => Navigator.pop(context, true),
    child: const Text('記録する'),
  ),
],
),
);
},
);
};
if (saved == true && subjectId != null) {
  store.addStudySession(subjectId: subjectId!, minutes: minutes, focus: focus);
  if (context.mounted) {

```

```

        nexusHaptic();
        showNexusToast(context, store.lastToast);
    }
}
}

Future<void> _addAssignment(BuildContext context, AppStore store) async {
  if (store.subjects.isEmpty) {
    final created = await promptNewSubject(context, store);
    if (created == null || !context.mounted) return;
  }
  final title = TextEditingController();
  var subjectId = store.subjects.first.id;
  var due = store.focusedDate.add(const Duration(days: 3));
  final saved = await showNexusSheet<bool>(<
    context: context,
    builder: (context) {
      return StatefulBuilder(
        builder: (context, setSheet) {
          return Column(
            mainAxisAlignment: MainAxisAlignment.min,
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              const Text('提出物を追加', style: TextStyle(fontSize: 18, fontWeight: FontWeight.w600)),
              const SizedBox(height: 12),
              DropdownButton<String>(
                value: subjectId,
                dropdownColor: NexusColors.card,
                isExpanded: true,
                items: [
                  for (final s in store.subjects)
                    DropdownMenuItem(value: s.id, child: Text(s.name)),
                ],
                onChanged: (v) => setSheet(() => subjectId = v ?? subjectId),
              ),
              TextField(
                controller: title,
                style: const TextStyle(color: NexusColors.text),
                decoration: const InputDecoration(labelText: '提出物名'),
              ),
              const SizedBox(height: 8),
              OutlinedButton(
                onPressed: () async {
                  final picked = await showDatePicker(
                    context: context,
                    initialDate: due,
                    firstDate: store.focusedDate.subtract(const Duration(days: 1)),
                    lastDate: DateTime(store.focusedDate.year + 2),
                  );
                  if (picked != null) setSheet(() => due = picked);
                },
                child: Text('期限  ${jpDate(due)} (${daysLeftLabel(due, store.focusedDate)}) '),
              ),
              const SizedBox(height: 12),
              FilledButton(
                onPressed: () => Navigator.pop(context, true),
                child: const Text('追加'),
              ),
            ],
          );
        },
      );
    ),
  );
  if (saved == true && title.text.trim().isNotEmpty) {
    store.addAssignment(subjectId: subjectId, title: title.text.trim(), dueAt: due);
    if (context.mounted) showNexusToast(context, store.lastToast);
  }
  title.dispose();
}

Future<void> _addExam(BuildContext context, AppStore store) async {
  final title = TextEditingController();
  var examAt = store.focusedDate.add(const Duration(days: 14));
  final saved = await showNexusSheet<bool>(<
    context: context,
    builder: (context) {

```

```

return StatefulBuilder(
  builder: (context, setSheet) {
    return Column(
      mainAxisAlignment: MainAxisAlignment.min,
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        const Text('試験日を追加', style: TextStyle(fontSize: 18, fontWeight: FontWeight.w600)),
        const SizedBox(height: 12),
        TextField(
          controller: title,
          style: const TextStyle(color: NexusColors.text),
          decoration: const InputDecoration(labelText: '科目・試験名'),
        ),
        const SizedBox(height: 8),
        OutlinedButton(
          onPressed: () async {
            final picked = await showDatePicker(
              context: context,
              initialDate: examAt,
              firstDate: store.focusedDate,
              lastDate: DateTime(store.focusedDate.year + 2),
            );
            if (picked != null) setSheet(() => examAt = picked);
          },
          child: Text('試験日 ${jpDate(examAt)} (${daysLeftLabel(examAt, store.focusedDate)}) '),
        ),
        const SizedBox(height: 12),
        FilledButton(
          onPressed: () => Navigator.pop(context, true),
          child: const Text('追加'),
        ),
      ],
    );
  },
);

if (saved == true && title.text.trim().isNotEmpty) {
  store.addExam(title: title.text.trim(), examAt: examAt);
  if (context.mounted) showNexusToast(context, store.lastToast);
}
title.dispose();
}

Future<void> _addGoal(BuildContext context, AppStore store) async {
  final title = TextEditingController();
  final current = TextEditingController(text: '0');
  final target = TextEditingController(text: '100');
  final subs = List.generate(4, (_) => TextEditingController());
  var due = DateTime(store.focusedDate.year, store.focusedDate.month + 3, store.focusedDate.day);
  final saved = await showNexusSheet<bool>(
    context: context,
    builder: (context) {
      return StatefulBuilder(
        builder: (context, setSheet) {
          return SingleChildScrollView(
            child: Column(
              mainAxisAlignment: MainAxisAlignment.min,
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                const Text('目標を追加', style: TextStyle(fontSize: 18, fontWeight: FontWeight.w600)),
                const SizedBox(height: 12),
                TextField(
                  controller: title,
                  style: const TextStyle(color: NexusColors.text),
                  decoration: const InputDecoration(labelText: '目標名 (例: TOEIC 800) '),
                ),
                Row(
                  children: [
                    Expanded(
                      child: TextField(
                        controller: current,
                        keyboardType: TextInputType.number,
                        style: const TextStyle(color: NexusColors.text),
                        decoration: const InputDecoration(labelText: '現在'),
                      ),
                    ),
                  ],
                ),
              ],
            ),
          );
        },
      );
    },
  );
}

```

```

        const SizedBox(width: 10),
        Expanded(
          child: TextField(
            controller: target,
            keyboardType: TextInputType.number,
            style: const TextStyle(color: NexusColors.text),
            decoration: const InputDecoration(labelText: '目標値'),
          ),
        ),
      ],
    ),
    const SizedBox(height: 8),
    OutlinedButton(
      onPressed: () async {
        final picked = await showDatePicker(
          context: context,
          initialDate: due,
          firstDate: store.focusedDate,
          lastDate: DateTime(store.focusedDate.year + 3),
        );
        if (picked != null) setSheet(() => due = picked);
      },
      child: Text('期限 ${jpDate(due)}'),
    ),
    const SizedBox(height: 10),
    const Text('サブゴール (4つ)', style: TextStyle(color: NexusColors.textSecondary, fontSize: 12)),
    for (var i = 0; i < 4; i++)
      TextField(
        controller: subs[i],
        style: const TextStyle(color: NexusColors.text),
        decoration: InputDecoration(labelText: 'サブゴール ${i + 1}'),
      ),
    const SizedBox(height: 12),
    FilledButton(
      onPressed: () => Navigator.pop(context, true),
      child: const Text('追加'),
    ),
  ],
),
);
},
);
},
);
if (saved == true && title.text.trim().isNotEmpty) {
  store.addGoal(
    title: title.text.trim(),
    current: int.tryParse(current.text) ?? 0,
    target: int.tryParse(target.text) ?? 100,
    dueAt: due,
    subGoalTitles: [for (final c in subs) c.text],
  );
  if (context.mounted) showNexusToast(context, store.lastToast);
}
title.dispose();
current.dispose();
target.dispose();
for (final c in subs) {
  c.dispose();
}
}

Future<Uint8List?> _takeProblemPhoto(BuildContext context) async {
  final picker = ImagePicker();
  Future<Uint8List?> from(ImageSource source) async {
    final file = await picker.pickImage(source: source, imageQuality: 70, maxWidth: 1600);
    if (file == null) return null;
    return file.readAsBytes();
  }
}

try {
  final bytes = await from(ImageSource.camera);
  if (bytes != null) return bytes;
} catch (_) {
  if (context.mounted) {
    showNexusToast(context, 'カメラを起動できないので、ギャラリーから選びます');
  }
}

```

```

    }
    try {
        return await from(ImageSource.gallery);
    } catch (_) {
        if (context.mounted) {
            showNexusToast(context, '写真を取得できませんでした');
        }
        return null;
    }
}

Future<void> _recordProblem(BuildContext context, AppStore store) async {
    final photo = await _takeProblemPhoto(context);
    if (photo == null || !context.mounted) return;

    final title = TextEditingController();
    if (store.subjects.isEmpty) {
        final created = await promptNewSubject(context, store);
        if (created == null || !context.mounted) {
            title.dispose();
            return;
        }
    }
    var subjectId = store.subjects.first.id;
    final saved = await showNexusSheet<bool>(
        context: context,
        builder: (context) {
            return StatefulBuilder(
                builder: (context, setSheet) {
                    return Column(
                        mainAxisAlignment: MainAxisAlignment.min,
                        crossAxisAlignment: CrossAxisAlignment.stretch,
                        children: [
                            const Text('問題を記録', style: TextStyle(fontSize: 18, fontWeight: FontWeight.w600)),
                            const SizedBox(height: 12),
                            ClipRRect(
                                borderRadius: BorderRadius.circular(12),
                                child: Image.memory(photo, height: 140, fit: BoxFit.cover),
                            ),
                            const SizedBox(height: 12),
                            DropdownButton<String>(
                                value: subjectId,
                                dropdownColor: NexusColors.card,
                                isExpanded: true,
                                items: [
                                    for (final s in store.subjects)
                                        DropdownMenuItem(value: s.id, child: Text(s.name)),
                                ],
                                onChanged: (v) => setSheet(() => subjectId = v ?? subjectId),
                            ),
                            TextField(
                                controller: title,
                                style: const TextStyle(color: NexusColors.text),
                                decoration: const InputDecoration(labelText: '問題名 (任意)'),
                            ),
                            const SizedBox(height: 12),
                            FilledButton(
                                onPressed: () => Navigator.pop(context, true),
                                child: const Text('保存'),
                            ),
                        ],
                    );
                },
            );
        },
    );
    if (saved == true) {
        final name = title.text.trim().isEmpty ? '問題 ${store.problems.length + 1}' : title.text.trim();
        store.addProblem(subjectId: subjectId, title: name, photoBytes: photo);
        if (context.mounted) showNexusToast(context, store.lastToast);
    }
    title.dispose();
}

Future<void> _openReview(BuildContext context, AppStore store) async {
    final due = store.reviewCards
        .where((c) => c.status == 'pending' && !c.dueAt.isAfter(store.focusedDate))

```

```

        .toList();
    await showNexusSheet<void>(
      context: context,
      builder: (context) {
        if (due.isEmpty) {
          return const Padding(
            padding: EdgeInsets.all(12),
            child: Text('今日の復習はありません。', style: TextStyle(color: NexusColors.textSecondary)),
          );
        }
        final card = due.first;
        final problem = store.problems.firstWhere((p) => p.id == card.problemId);
        return Column(
          mainAxisAlignment: MainAxisAlignment.min,
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            if (problem.photoBytes != null) ...[
              ClipRect(
                borderRadius: BorderRadius.circular(12),
                child: Image.memory(problem.photoBytes!, height: 160, fit: BoxFit.cover),
              ),
              const SizedBox(height: 12),
            ],
            Text(problem.title, style: const TextStyle(fontSize: 18, fontWeight: FontWeight.w700)),
            const SizedBox(height: 6),
            Text('${card.intervalStep}日後カード', style: const TextStyle(color: NexusColors.textMuted)),
            const SizedBox(height: 12),
            Wrap(
              spacing: 8,
              children: [
                for (final rating in ReviewRating.values)
                  OutlinedButton(
                    onPressed: () {
                      store.rateReview(card.id, rating);
                      Navigator.pop(context);
                    },
                    child: Text(switch (rating) {
                      ReviewRating.again => 'もう一度',
                      ReviewRating.hard => '難しい',
                      ReviewRating.normal => '普通',
                      ReviewRating.easy => '簡単',
                    })),
              ],
            ),
          ],
        );
      },
    );
  }
}

```

ファイル: lib/screens/study/focus_timer_page.dart

行数: 205

```
import 'dart:async';

import 'package:flutter/material.dart';

import '../..../app/theme.dart';
import '../..../core/format.dart';
import '../..../data/app_store.dart';
import '../..../data/models.dart';
import '../..../widgets/duration_picker.dart';
import '../..../widgets/progress_ring.dart';

Future<void> openFocusTimer(BuildContext context) {
  return Navigator.of(context, rootNavigator: true).push(
    PageRouteBuilder<void>(
      opaque: true,
      transitionDuration: const Duration(milliseconds: 280),
      pageBuilder: (context, animation, _) {
        return FadeTransition(
          opacity: animation,
          child: const FocusTimerPage(),
        );
      },
    ),
  );
}

class FocusTimerPage extends StatefulWidget {
  const FocusTimerPage({super.key});

  @override
  State<FocusTimerPage> createState() => _FocusTimerPageState();
}

class _FocusTimerPageState extends State<FocusTimerPage> {
  Timer? _tick;

  @override
  void initState() {
    super.initState();
    _tick = Timer.periodic(const Duration(seconds: 1), (_) {
      if (mounted) setState(() {});
    });
  }

  @override
  void dispose() {
    _tick?.cancel();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    final store = AppScope.of(context);
    final remaining = store.timerRemainingSeconds();
    final total = store.timerTotalSeconds == 0 ? 1 : store.timerTotalSeconds;
    final progress = 1 - remaining / total;
    final subject = store.subjectById(store.timerSubjectId ?? store.nextStudySubjectId);

    return PopScope(
      canPop: !store.timerRunning,
      child: Scaffold(
        backgroundColor: NexusColors.background,
        body: SafeArea(
          child: Padding(
            padding: const EdgeInsets.fromLTRB(20, 8, 20, 20),
            child: Column(
              children: [
                Row(
                  children: [
```

```

IconButton(
  onPressed: store.timerRunning ? null : () => Navigator.pop(context),
  icon: Icon(
    Icons.close_rounded,
    color: store.timerRunning ? NexusColors.textMuted : NexusColors.text,
  ),
),
const Spacer(),
Text(
  subject?.name ?? '集中タイマー',
  style: const TextStyle(
    color: NexusColors.textSecondary,
    fontWeight: FontWeight.w700,
  ),
),
const Spacer(),
const SizedBox(width: 48),
],
),
Expanded(
  child: SingleChildScrollView(
    child: Column(
      children: [
        const SizedBox(height: 12),
        ProgressRing(
          progress: progress.clamp(0, 1),
          size: 220,
          stroke: 14,
          animate: false,
          child: Column(
            mainAxisAlignment: MainAxisAlignment.min,
            children: [
              Text(
                mmss(remaining),
                style: TextStyle(
                  color: NexusColors.text,
                  fontSize: remaining >= 3600 ? 36 : 48,
                  fontWeight: FontWeight.w600,
                  letterSpacing: 1.4,
                ),
              ),
              Text(
                store.timerRunning ? '集中中' : '停止中',
                style: const TextStyle(color: NexusColors.textMuted),
              ),
            ],
          ),
        ),
        const SizedBox(height: 20),
        DurationMinutesPicker(
          minutes: (store.timerTotalSeconds / 60).round().clamp(
            kMinStudyDurationMinutes,
            kMaxStudyDurationMinutes,
          ),
          onChanged: store.setTimerMinutes,
          enabled: !store.timerRunning,
        ),
      ],
    ),
  ),
),
Row(
  children: [
    Expanded(
      child: FilledButton(
        onPressed: store.timerRunning ? store.pauseTimer : store.startTimer,
        child: Padding(
          padding: const EdgeInsets.symmetric(vertical: 12),
          child: Text(store.timerRunning ? '一時停止' : '開始'),
        ),
      ),
    ),
    const SizedBox(width: 10),
    Expanded(
      child: OutlinedButton(
        onPressed: () => _finish(context, store),
        child: const Padding(

```



```

        padding: EdgeInsets.symmetric(vertical: 12),
        child: Text('終了'),
      ),
    ),
  ],
),
],
),
),
),
),
);
}

Future<void> _finish(BuildContext context, AppStore store) async {
  if (store.timerElapsedSeconds() <= 0) {
    store.finishTimer();
    if (context.mounted) Navigator.pop(context);
    return;
  }

  final focus = await showModalBottomSheet<StudyFocus>(
    context: context,
    backgroundColor: NexusColors.card,
    shape: const RoundedRectangleBorder(
      borderRadius: BorderRadius.vertical(top: Radius.circular(20)),
    ),
    builder: (context) {
      return Padding(
        padding: const EdgeInsets.fromLTRB(16, 16, 16, 24),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            const Text(
              '集中度は?',
              style: TextStyle(fontSize: 18, fontWeight: FontWeight.w600),
            ),
            const SizedBox(height: 12),
            for (final value in StudyFocus.values)
              Padding(
                padding: const EdgeInsets.only(bottom: 8),
                child: OutlinedButton(
                  onPressed: () => Navigator.pop(context, value),
                  child: Text(value.label),
                ),
              ),
          ],
        ),
      );
    },
  );

  store.finishTimer(focus: focus ?? StudyFocus.high);
  if (context.mounted) Navigator.pop(context);
}
}

```



```

        onPressed: () => store.setLifeDate(DateTime(day.year, day.month + 1, 1)),
        icon: const Icon(Icons.chevron_right, color: NexusColors.cyan),
      ),
    ],
  ),
),
const SizedBox(height: 8),
_MonthGrid(store: store),
],
),
),
const SizedBox(height: 12),
GlassCard(
  child: Column(
    children: [
      SectionRow(
        title: '${day.month}月${day.day}日の予定',
        trailing: AddChip(label: '予定を追加', onTap: () => openScheduleEditor(context, day: day)),
      ),
      const SizedBox(height: 8),
      SizedBox(
        height: 132,
        child: items.isEmpty
          ? const Center(
              child: EmptyHint(text: '予定はありません', icon: Icons.event_outlined),
            )
          : ListView.separated(
              itemCount: items.length,
              separatorBuilder: (_, _) => const SizedBox(height: 8),
              itemBuilder: (context, i) {
                final item = items[i];
                return InkWell(
                  onTap: () => openScheduleEditor(context, item: item),
                  child: Container(
                    padding: const EdgeInsets.fromLTRB(10, 9, 10, 9),
                    decoration: BoxDecoration(
                      color: NexusColors.sky,
                      borderRadius: BorderRadius.circular(14),
                    ),
                    child: Row(
                      children: [
                        TimePill(hm(item.startAt)),
                        const SizedBox(width: 10),
                        Expanded(
                          child: Text(item.title, style: const TextStyle(fontWeight: FontWeight.w700)),
                        ),
                      ],
                    ),
                  ),
                ),
              ),
            );
      ],
    ),
  ),
),
const SizedBox(height: 12),
Row(
  children: [
    Expanded(child: SizedBox(height: _lifeCardHeight, child: _HabitCard(store: store))),
    const SizedBox(width: 10),
    Expanded(child: SizedBox(height: _lifeCardHeight, child: _SleepCard(store: store))),
  ],
),
const SizedBox(height: 10),
Row(
  children: [
    Expanded(child: SizedBox(height: _lifeCardHeight, child: _MoodCard(store: store))),
    const SizedBox(width: 10),
    Expanded(child: SizedBox(height: _lifeCardHeight, child: _StepsCard(store: store))),
  ],
),
const SizedBox(height: 12),
GlassCard(
  fill: NexusColors.peach,
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,

```

```

        children: [
          const SectionRow(title: '日記'),
          const SizedBox(height: 4),
          const Text('今日を1分で振り返る', style: TextStyle(color: NexusColors.textMuted, fontSize: 12)),
          const SizedBox(height: 8),
          Text(store.diary, style: const TextStyle(height: 1.4)),
          const SizedBox(height: 10),
          OutlinedButton(
            onPressed: () => _editDiary(context, store),
            child: const Text('続きを書く'),
          ),
        ],
      ),
    ),
  ],
),
);
}

```

```

Future<void> _editDiary(BuildContext context, AppStore store) async {
  final controller = TextEditingController(text: store.diary);
  final saved = await showNexusSheet<bool>(
    context: context,
    builder: (context) {
      return Column(
        mainAxisAlignment: MainAxisAlignment.min,
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          const Text('日記', style: TextStyle(fontSize: 18, fontWeight: FontWeight.w700)),
          const SizedBox(height: 8),
          TextField(
            controller: controller,
            maxLines: 5,
            style: const TextStyle(color: NexusColors.text),
          ),
          const SizedBox(height: 12),
          FilledButton(
            onPressed: () => Navigator.pop(context, true),
            child: const Text('保存'),
          ),
        ],
      );
    },
  );
  if (saved == true) store.setDiary(controller.text.trim());
  controller.dispose();
}
}

```

```

Future<void> _openAddHabit(BuildContext context, AppStore store) async {
  final name = TextEditingController();
  var icon = Icons.wb_sunny_rounded;
  var color = const Color(0xFFFFC857);
  const colors = [
    Color(0xFFFFC857),
    NexusColors.cyan,
    NexusColors.green,
    NexusColors.purple,
    Color(0xFFFF8AD2),
    NexusColors.gold,
  ];
  const icons = [
    Icons.wb_sunny_rounded,
    Icons.menu_book_rounded,
    Icons.directions_run_rounded,
    Icons.bedtime_rounded,
    Icons.spa_rounded,
    Icons.nightlight_round,
    Icons.fitness_center_rounded,
    Icons.self_improvement_rounded,
  ];
  final saved = await showNexusSheet<bool>(
    context: context,
    builder: (context) {
      return StatefulBuilder(
        builder: (context, setSheet) {
          return Column(

```

```

mainAxisSize: MainAxisSize.min,
crossAxisAlignment: CrossAxisAlignment.stretch,
children: [
  const Text('習慣を追加', style: TextStyle(fontSize: 18, fontWeight: FontWeight.w600)),
  const SizedBox(height: 12),
  TextField(
    controller: name,
    autofocus: true,
    style: const TextStyle(color: NexusColors.text),
    decoration: const InputDecoration(labelText: '習慣名'),
  ),
  const SizedBox(height: 12),
  const Text('アイコン', style: TextStyle(color: NexusColors.textSecondary, fontSize: 12)),
  Wrap(
    spacing: 4,
    children: [
      for (final i in icons)
        IconButton(
          onPressed: () => setSheet(() => icon = i),
          icon: Icon(i, color: i == icon ? color : NexusColors.textMuted),
        ),
    ],
  ),
  const Text('色', style: TextStyle(color: NexusColors.textSecondary, fontSize: 12)),
  Row(
    children: [
      for (final c in colors)
        GestureDetector(
          onTap: () => setSheet(() => color = c),
          child: Container(
            width: 28,
            height: 28,
            margin: const EdgeInsets.only(right: 8, top: 6, bottom: 6),
            decoration: BoxDecoration(
              color: c,
              shape: BoxShape.circle,
              border: Border.all(
                color: c == color ? NexusColors.text : Colors.transparent,
                width: 2,
              ),
            ),
          ),
        ),
    ],
  ),
  const SizedBox(height: 12),
  FilledButton(onPressed: () => Navigator.pop(context, true), child: const Text('追加')),
],
);
},
);
};
);
if (saved == true && name.text.trim().isNotEmpty) {
  store.addHabit(name: name.text.trim(), icon: icon, color: color);
  if (context.mounted) showNexusToast(context, store.lastToast);
}
name.dispose();
}

class _DateButton extends StatelessWidget {
  const _DateButton({required this.label, required this.onTap});

  final String label;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    return InkWell(
      onTap: onTap,
      borderRadius: BorderRadius.circular(20),
      child: Container(
        padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 8),
        decoration: BoxDecoration(
          borderRadius: BorderRadius.circular(20),
          color: NexusColors.sky,
        ),
      ),
    );
  }
}

```

```

        child: Row(
          children: [
            const Icon(Icons.calendar_month, size: 14, color: NexusColors.cyan),
            const SizedBox(width: 6),
            Text(label, style: const TextStyle(fontSize: 11, fontWeight: FontWeight.w600)),
          ],
        ),
      ),
    );
  }
}

class _MonthGrid extends StatelessWidget {
  const _MonthGrid({required this.store});

  final AppStore store;

  @override
  Widget build(BuildContext context) {
    final focused = store.lifeDate;
    final first = DateTime(focused.year, focused.month, 1);
    final daysInMonth = DateTime(focused.year, focused.month + 1, 0).day;
    final leading = mondayIndex(first);
    final marked = {
      for (final s in store.schedules)
        if (s.startAt.year == focused.year && s.startAt.month == focused.month) s.startAt.day,
    };

    return Column(
      children: [
        Row(
          children: [
            for (final label in weekLabels)
              Expanded(
                child: Center(
                  child: Text(label, style: const TextStyle(color: NexusColors.textMuted, fontSize: 11)),
                ),
              ),
          ],
        ),
        const SizedBox(height: 6),
        for (var row = 0; row < 6; row++)
          Padding(
            padding: const EdgeInsets.only(bottom: 4),
            child: Row(
              children: [
                for (var col = 0; col < 7; col++)
                  Expanded(
                    child: _DayCell(
                      day: row * 7 + col - leading + 1,
                      daysInMonth: daysInMonth,
                      selected: focused.day,
                      marked: marked,
                      onTap: (d) => store.setLifeDate(DateTime(focused.year, focused.month, d)),
                    ),
                  ),
              ],
            ),
          ),
      ],
    );
  }
}

class _DayCell extends StatelessWidget {
  const _DayCell({
    required this.day,
    required this.daysInMonth,
    required this.selected,
    required this.marked,
    required this.onTap,
  });

  final int day;
  final int daysInMonth;
  final int selected;
  final Set<int> marked;

```

```

final ValueChanged<int> onTap;

@override
Widget build(BuildContext context) {
  if (day < 1 || day > daysInMonth) return const SizedBox(height: 32);
  final isSelected = day == selected;
  return InkWell(
    onTap: () => onTap(day),
    borderRadius: BorderRadius.circular(8),
    child: SizedBox(
      height: 32,
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Container(
            width: 26,
            height: 26,
            alignment: Alignment.center,
            decoration: isSelected
              ? const BoxDecoration(
                  shape: BoxShape.circle,
                  color: NexusColors.cyan,
                )
              : null,
            child: Text(
              '$day',
              style: TextStyle(
                fontSize: 12,
                color: isSelected ? Colors.white : NexusColors.text,
                fontWeight: isSelected ? FontWeight.w700 : FontWeight.w500,
              ),
            ),
          ),
          if (marked.contains(day) && !isSelected)
            Container(
              width: 4,
              height: 4,
              decoration: const BoxDecoration(color: NexusColors.cyan, shape: BoxShape.circle),
            ),
        ],
      ),
    ),
  );
}

class _HabitCard extends StatelessWidget {
  const _HabitCard({required this.store});

  final AppStore store;

  @override
  Widget build(BuildContext context) {
    return GlassCard(
      height: _lifeCardHeight,
      fill: NexusColors.sage,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            children: [
              const Expanded(
                child: Text('習慣チェーン', style: TextStyle(fontWeight: FontWeight.w700)),
              ),
              IconButton(
                onPressed: () => _openAddHabit(context, store),
                visualDensity: VisualDensity.compact,
                padding: EdgeInsets.zero,
                constraints: const BoxConstraints(minWidth: 28, minHeight: 28),
                icon: const Icon(Icons.add_rounded, size: 20, color: NexusColors.cyan),
              ),
            ],
          ),
          const SizedBox(height: 4),
          Expanded(
            child: store.habits.isEmpty
              ? const Align(

```

```

        alignment: Alignment.topLeft,
        child: EmptyHint(text: '習慣はまだありません', icon: Icons.spa_outlined),
      ),
    : ListView(
      padding: EdgeInsets.zero,
      children: [
        for (final habit in store.habits)
          _HabitRow(key: ValueKey(habit.id), store: store, habit: habit),
      ],
    ),
  ),
],
),
);
}
}

class _HabitRow extends StatefulWidget {
  const _HabitRow({super.key, required this.store, required this.habit});

  final AppStore store;
  final Habit habit;

  @override
  State<_HabitRow> createState() => _HabitRowState();
}

class _HabitRowState extends State<_HabitRow> {
  var _lit = false;

  Future<void> _flash() async {
    setState(() => _lit = true);
    await Future<void>.delayed(const Duration(milliseconds: 420));
    if (mounted) setState(() => _lit = false);
  }

  @override
  Widget build(BuildContext context) {
    final habit = widget.habit;
    final day = widget.store.lifeDate;
    return InkWell(
      onTap: () {
        final turningOn = !habit.doneOn(day);
        widget.store.toggleHabit(habit.id, day);
        if (turningOn) {
          nexusHaptic();
          _flash();
        }
      },
      borderRadius: BorderRadius.circular(10),
      child: AnimatedContainer(
        duration: const Duration(milliseconds: 280),
        curve: Curves.easeOutCubic,
        padding: const EdgeInsets.symmetric(vertical: 6, horizontal: 4),
        decoration: BoxDecoration(
          borderRadius: BorderRadius.circular(10),
          color: _lit ? NexusColors.gold.withValues(alpha: 0.12) : Colors.transparent,
        ),
        child: Row(
          children: [
            ThingsCheck(checked: habit.doneOn(day), color: habit.color, size: 20),
            const SizedBox(width: 8),
            Expanded(
              child: Text(habit.name, style: const TextStyle(fontSize: 13, fontWeight: FontWeight.w600)),
            ),
            AnimatedDefaultTextStyle(
              duration: const Duration(milliseconds: 280),
              style: TextStyle(
                color: _lit ? NexusColors.gold : habit.color,
                fontSize: 16,
                fontWeight: FontWeight.w600,
              ),
              child: Text('${habit.currentStreak(day)}日連続'),
            ),
          ],
        ),
      ),
    ),
  ),
}

```



```

    );
  }
}

class _SleepCard extends StatelessWidget {
  const _SleepCard({required this.store});

  final AppStore store;

  @override
  Widget build(BuildContext context) {
    final week = store.sleepWeekHours(store.lifeDate);
    return InkWell(
      onTap: () => openSleepLogger(context, store),
      borderRadius: BorderRadius.circular(16),
      child: GlassCard(
        height: _lifeCardHeight,
        fill: NexusColors.sky,
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const Text('睡眠の記録', style: TextStyle(fontWeight: FontWeight.w700)),
            const SizedBox(height: 8),
            Text(
              store.isSleeping ? '就寝中' : '${store.sleepHours.toStringAsFixed(1)}h',
              style: const TextStyle(fontSize: 24, fontWeight: FontWeight.w600),
            ),
            Text(
              store.isSleeping ? 'タップして起床を記録' : '就寝・起床で記録 目標 7-8時間',
              style: const TextStyle(color: NexusColors.textMuted, fontSize: 11),
            ),
            const SizedBox(height: 10),
            SizedBox(
              height: 48,
              child: Row(
                crossAxisAlignment: CrossAxisAlignment.end,
                children: [
                  for (var i = 0; i < week.length; i++)
                    Expanded(
                      child: Padding(
                        padding: const EdgeInsets.symmetric(horizontal: 2),
                        child: TrackBar(
                          value: (week[i] / 9).clamp(0.0, 1.0),
                          color: i == mondayIndex(store.lifeDate)
                            ? NexusColors.cyan
                            : NexusColors.cyan.withValues(alpha: 0.45),
                        ),
                    ),
                ],
              ),
            ),
          ],
        ),
      ),
    );
  }
}

class _MoodCard extends StatelessWidget {
  const _MoodCard({required this.store});

  final AppStore store;

  @override
  Widget build(BuildContext context) {
    return GlassCard(
      height: _lifeCardHeight,
      fill: NexusColors.peach,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          const Text('気分とエネルギー', style: TextStyle(fontWeight: FontWeight.w700, fontSize: 13)),
          const SizedBox(height: 8),
          const Text('いまの気分は?', style: TextStyle(color: NexusColors.textMuted, fontSize: 11)),
          Row(
            children: [

```

```

        for (var i = 1; i <= 5; i++)
            Expanded(
                child: IconButton(
                    onPressed: () => store.setMood(i),
                    padding: EdgeInsets.zero,
                    visualDensity: VisualDensity.compact,
                    constraints: const BoxConstraints(minWidth: 28, minHeight: 28),
                    icon: Icon(
                        i <= store.mood ? Icons.sentiment_satisfied_alt : Icons.sentiment_neutral,
                        size: 22,
                        color: i == store.mood ? NexusColors.green : NexusColors.textMuted,
                    ),
                ),
            ),
        ],
    ),
    const Text('エネルギーは?', style: TextStyle(color: NexusColors.textMuted, fontSize: 11)),
    Row(
        children: [
            for (var i = 1; i <= 5; i++)
                Expanded(
                    child: IconButton(
                        onPressed: () => store.setEnergy(i),
                        padding: EdgeInsets.zero,
                        visualDensity: VisualDensity.compact,
                        constraints: const BoxConstraints(minWidth: 28, minHeight: 28),
                        icon: Icon(
                            Icons.bolt_rounded,
                            size: 22,
                            color: i <= store.energy ? NexusColors.green : NexusColors.textMuted,
                        ),
                    ),
                ),
        ],
    ),
],
),
],
),
);
}
}

class _StepsCard extends StatelessWidget {
  const _StepsCard({required this.store});

  final AppStore store;

  @override
  Widget build(BuildContext context) {
    final left = (store.stepGoal - store.steps).clamp(0, store.stepGoal);
    return GlassCard(
      height: _lifeCardHeight,
      fill: NexusColors.lilac,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          const Text('今日の歩数', style: TextStyle(fontWeight: FontWeight.w700)),
          const SizedBox(height: 10),
          Text(
            '${store.steps} 歩',
            style: const TextStyle(fontSize: 24, fontWeight: FontWeight.w600),
          ),
          const SizedBox(height: 10),
          SoftProgress(
            value: store.steps / store.stepGoal,
            color: NexusColors.green,
            height: 8,
          ),
          const SizedBox(height: 8),
          Text('目標まであと $left 歩', style: const TextStyle(color: NexusColors.textMuted, fontSize: 11)),
          const Spacer(),
        ],
      ),
    );
  }
}

```

ファイル: lib/screens/life/sleep_sheet.dart

行数: 171

```
import 'package:flutter/material.dart';

import '../..../app/theme.dart';
import '../..../core/format.dart';
import '../..../data/app_store.dart';
import '../..../widgets/ui_bits.dart';

Future<void> openSleepLogger(BuildContext context, AppStore store) async {
  var bed = store.sleepStartedAt ??
    DateTime(
      store.focusedDate.year,
      store.focusedDate.month,
      store.focusedDate.day,
    ).subtract(const Duration(hours: 7, minutes: 30));
  if (bed.hour < 12) {
    bed = DateTime(bed.year, bed.month, bed.day - 1, 23, 30);
  }
  var wake = DateTime(
    store.focusedDate.year,
    store.focusedDate.month,
    store.focusedDate.day,
    7,
    0,
  );
  final latest = store.latestSleepLog;
  if (latest != null && store.sleepStartedAt == null) {
    bed = latest.bedAt;
    wake = latest.wakeAt;
  }
  var quality = latest?.quality ?? 4;

  await showNexusSheet<void>(
    context: context,
    builder: (context) {
      return StatefulBuilder(
        builder: (context, setSheet) {
          final hours = wake.difference(bed).inMinutes / 60.0;
          return SingleChildScrollView(
            child: Column(
              mainAxisAlignment: MainAxisAlignment.min,
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                const Text('睡眠を記録', style: TextStyle(fontSize: 18, fontWeight: FontWeight.w600)),
                const SizedBox(height: 6),
                Text(
                  store.isSleeping
                    ? '就寝中です。起きたらボタンを押してください。'
                    : '就寝と起床を分けて記録します。あとから直すこともできます。',
                  style: const TextStyle(color: NexusColors.textMuted, fontSize: 12),
                ),
                const SizedBox(height: 14),
                if (store.isSleeping) ...[
                  Text(
                    '就寝 ${hm(store.sleepStartedAt!)}',
                    style: const TextStyle(color: NexusColors.purple, fontWeight: FontWeight.w700),
                  ),
                  const SizedBox(height: 10),
                  FilledButton(
                    onPressed: () {
                      store.wakeUp(quality);
                      Navigator.pop(context);
                    },
                    child: const Text('起きた'),
                  ),
                ],
                TextButton(
                  onPressed: () {
                    store.cancelSleep();
                    setSheet({});
                  },
                ),
              ],
            ),
          );
        },
      );
    },
  );
}
```

```

        child: const Text('就寝を取り消す'),
      ),
    ] else ...[
      FilledButton.tonal(
        onPressed: () {
          store.startSleep();
          Navigator.pop(context);
        },
        child: const Text('今から寝る'),
      ),
    ],
    const SizedBox(height: 12),
    const Text('あとから記録', style: TextStyle(color: NexusColors.textSecondary, fontSize: 12)),
    const SizedBox(height: 8),
    OutlinedButton(
      onPressed: () async {
        final picked = await showTimePicker(
          context: context,
          initialTime: TimeOfDay.fromDateTime(bed),
        );
        if (picked == null) return;
        setSheet(() {
          bed = DateTime(bed.year, bed.month, bed.day, picked.hour, picked.minute);
          if (!wake.isAfter(bed)) {
            wake = bed.add(const Duration(hours: 8));
          }
        });
      },
      child: Text('就寝  ${hm(bed)}'),
    ),
    const SizedBox(height: 8),
    OutlinedButton(
      onPressed: () async {
        final picked = await showTimePicker(
          context: context,
          initialTime: TimeOfDay.fromDateTime(wake),
        );
        if (picked == null) return;
        setSheet(() {
          wake = DateTime(wake.year, wake.month, wake.day, picked.hour, picked.minute);
          if (!wake.isAfter(bed)) {
            wake = wake.add(const Duration(days: 1));
          }
        });
      },
      child: Text('起床  ${hm(wake)}'),
    ),
    const SizedBox(height: 10),
    Text(
      hours <= 0 ? '時刻を確認してください' : '${hours.toStringAsFixed(1)} 時間',
      textAlign: TextAlign.center,
      style: const TextStyle(fontSize: 22, fontWeight: FontWeight.w600),
    ),
    const SizedBox(height: 8),
    Wrap(
      spacing: 8,
      children: [
        for (final h in const [6.0, 7.0, 7.5, 8.0])
          ActionChip(
            label: Text('${h}h'),
            onPressed: () => setSheet(() {
              wake = DateTime(store.focusedDate.year, store.focusedDate.month, store.focusedDate.day, 7);
              bed = wake.subtract(Duration(minutes: (h * 60).round()));
            }),
          ),
      ],
    ),
    const SizedBox(height: 10),
    const Text('眠りの質', style: TextStyle(color: NexusColors.textSecondary, fontSize: 12)),
    Slider(
      value: quality.toDouble(),
      min: 1,
      max: 5,
      divisions: 4,
      label: switch (quality) {
        1 => '浅い',
        2 => 'いまいち',
        3 => '普通',

```

```
      4 => 'よい',
      _ => 'とてもよい',
    },
    onChanged: (v) => setSheet(() => quality = v.round()),
  ),
  const SizedBox(height: 8),
  FilledButton(
    onPressed: hours <= 0
      ? null
      : () {
          store.logSleep(bedAt: bed, wakeAt: wake, quality: quality);
          Navigator.pop(context);
        },
    child: const Text('保存'),
  ),
],
],
),
);
},
);
},
);
}
```

ファイル: lib/screens/money/money_screen.dart

行数: 398

```
import 'package:flutter/material.dart';

import '../..../app/theme.dart';
import '../..../core/format.dart';
import '../..../data/app_store.dart';
import '../..../data/models.dart';
import '../..../widgets/count_up_yen.dart';
import '../..../widgets/glass_card.dart';
import '../..../widgets/ui_bits.dart';
import 'box_detail_page.dart';
import 'money_forms.dart';

class MoneyScreen extends StatefulWidget {
  const MoneyScreen({super.key});

  @override
  State<MoneyScreen> createState() => _MoneyScreenState();
}

class _MoneyScreenState extends State<MoneyScreen> {
  bool todayTab = true;

  @override
  Widget build(BuildContext context) {
    final store = AppScope.of(context);
    final money = store.money;
    final remainRatio = money.todayBudget == 0 ? 0.0 : money.spendableToday / money.todayBudget;

    return PageScaffold(
      child: ListView(
        padding: const EdgeInsets.fromLTRB(16, 8, 16, 24),
        children: [
          Row(
            children: [
              const Expanded(child: GradientTitle('Money')),
              AddChip(
                label: '収入を追加',
                onTap: () => openAddIncome(context, store),
              ),
            ],
          ),
          const SizedBox(height: 4),
          const Text('未来のために、今日を知る。', style: TextStyle(color: NexusColors.textSecondary)),
          const SizedBox(height: 14),
          GlassCard(
            fill: NexusColors.peach,
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  '${focusedMonthLabel(store.focusedDate)}の残高',
                  style: const TextStyle(color: NexusColors.textSecondary),
                ),
                const SizedBox(height: 6),
                CountUpYen(
                  value: money.balance,
                  style: const TextStyle(
                    fontSize: 36,
                    fontWeight: FontWeight.w800,
                    color: NexusColors.text,
                    letterSpacing: -0.8,
                  ),
                ),
                const SizedBox(height: 12),
                Row(
                  children: [
                    StatChip(label: '収入', value: yen(money.income), color: NexusColors.income),
                    const SizedBox(width: 8),
                    StatChip(label: '支出', value: yen(-money.expense), color: NexusColors.expense),
                  ],
                ),
              ],
            ),
          ),
        ],
      ),
    );
  }
}
```

```

    ],
  ),
],
),
),
const SizedBox(height: 14),
const SectionRow(title: 'ボックス'),
const SizedBox(height: 8),
Wrap(
  spacing: 8,
  runSpacing: 8,
  children: [
    AddChip(label: 'ボックスを追加', onTap: () => openAddBox(context, store)),
    AddChip(label: 'カードを追加', onTap: () => openAddCard(context, store)),
  ],
),
const SizedBox(height: 8),
SizedBox(
  height: 168,
  child: ListView.separated(
    scrollDirection: Axis.horizontal,
    itemCount: store.bboxes.length,
    separatorBuilder: (_, _) => const SizedBox(width: 10),
    itemBuilder: (context, i) {
      final box = store.bboxes[i];
      return SizedBox(
        width: 148,
        child: Stack(
          children: [
            Positioned.fill(
              child: InkWell(
                onTap: () => _openBox(context, box.id),
                borderRadius: BorderRadius.circular(16),
                child: box.isSavings
                  ? _SavingsTile(store: store, box: box)
                  : _BudgetTile(store: store, box: box),
              ),
            ),
            Positioned(
              top: 6,
              right: 6,
              child: _DeleteMark(
                onTap: () => _confirmDeleteBox(context, store, box),
              ),
            ),
          ],
        ),
      ),
    ],
  ),
);
},
),
const SizedBox(height: 12),
GlassCard(
  fill: NexusColors.sky,
  child: Column(
    children: [
      Row(
        children: [
          _Tab(
            label: '今日使える額',
            selected: todayTab,
            onTap: () => setState(() => todayTab = true),
          ),
          _Tab(
            label: '今週使える額',
            selected: !todayTab,
            onTap: () => setState(() => todayTab = false),
          ),
        ],
      ),
      const SizedBox(height: 16),
      Text(
        yen(todayTab ? money.spendableToday : money.spendableWeek),
        style: const TextStyle(fontSize: 34, fontWeight: FontWeight.w600),
      ),
      const SizedBox(height: 8),
      Row(

```

```

        children: [
          Text('今日の予算 ${yen(money.todayBudget)}', style: const TextStyle(color: NexusColors.textMuted, fontSize:
12)),
          const Spacer(),
          Text('残り ${((remainRatio * 100).round())}%', style: const TextStyle(color: NexusColors.textMuted, fontSize:
12)),
        ],
      ),
      const SizedBox(height: 10),
      SoftProgress(value: remainRatio, height: 8),
    ],
  ),
),
const SizedBox(height: 12),
GlassCard(
  child: Column(
    children: [
      SectionRow(
        title: '今後の支払い予定',
        trailing: AddChip(
          label: '支払予定を追加',
          onTap: () => openAddPayment(context, store),
        ),
      ),
      const SizedBox(height: 8),
      for (final p in store.payments)
        Padding(
          padding: const EdgeInsets.only(bottom: 8),
          child: SoftTile(
            child: Row(
              children: [
                const AccentIcon(Icons.event, color: NexusColors.cyan),
                const SizedBox(width: 8),
                Expanded(
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Text(p.title, style: const TextStyle(fontWeight: FontWeight.w700)),
                      Text(
                        [
                          if (p.boxId != null) store.boxById(p.boxId)?.name,
                          switch (p.repeat) {
                            PaymentRepeat.none => null,
                            PaymentRepeat.monthly => '毎月',
                            PaymentRepeat.yearly => '毎年',
                          },
                        ].whereType<String>().join(' · '),
                        style: const TextStyle(color: NexusColors.textMuted, fontSize: 11),
                      ),
                    ],
                  ),
                ),
              ),
            Text(
              '${store.nextPaymentDue(p, store.focusedDate).month}/${store.nextPaymentDue(p,
store.focusedDate).day}',
              style: const TextStyle(color: NexusColors.textMuted),
            ),
            const SizedBox(width: 10),
            Text(yen(p.amount), style: const TextStyle(fontWeight: FontWeight.w800)),
            _DeleteMark(
              onTap: () => _confirmDeletePayment(context, store, p),
            ),
          ],
        ),
      ),
    ],
  ),
),
],
),
),
);
}

Future<void> _openBox(BuildContext context, String boxId) {
  return Navigator.of(context, rootNavigator: true).push(
    MaterialPageRoute<void>(

```



```

        builder: (_) => BoxDetailPage(boxId: boxId),
      ),
    );
  }

Future<void> _confirmDeleteBox(BuildContext context, AppStore store, BudgetBox box) async {
  final ok = await _confirm(
    context,
    title: 'ボックスを削除',
    body: '「${box.name}」を削除します。中のカードは未振り分けへ移します。',
  );
  if (!ok || !context.mounted) return;
  store.deleteBox(box.id);
  showNexusToast(context, store.lastToast);
}

Future<void> _confirmDeletePayment(BuildContext context, AppStore store, PaymentPlan plan) async {
  final ok = await _confirm(
    context,
    title: '支払予定を削除',
    body: '「${plan.title}」を削除します。',
  );
  if (!ok || !context.mounted) return;
  store.deletePayment(plan.id);
  showNexusToast(context, store.lastToast);
}

Future<bool> _confirm(BuildContext context, {required String title, required String body}) async {
  final result = await showDialog<bool>(
    context: context,
    builder: (context) {
      return AlertDialog(
        backgroundColor: NexusColors.card,
        title: Text(title),
        content: Text(body),
        actions: [
          TextButton(onPressed: () => Navigator.pop(context, false), child: const Text('やめる')),
          FilledButton(onPressed: () => Navigator.pop(context, true), child: const Text('削除')),
        ],
      );
    },
  );
  return result == true;
}

String focusedMonthLabel(DateTime d) => '${d.month}月';

class _BudgetTile extends StatelessWidget {
  const _BudgetTile({required this.store, required this.box});

  final AppStore store;
  final BudgetBox box;

  @override
  Widget build(BuildContext context) {
    final spent = store.spentOfBox(box.id, periodOf: box, day: store.focusedDate);
    final remaining = box.monthlyBudget - spent;
    final ratio = box.monthlyBudget == 0 ? 0.0 : spent / box.monthlyBudget;
    return GlassCard(
      fill: box.color.withValues(alpha: 0.14),
      borderColor: box.color.withValues(alpha: 0.28),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Icon(box.icon, color: box.color),
          const SizedBox(height: 8),
          Text(box.name, style: const TextStyle(fontWeight: FontWeight.w700)),
          const Spacer(),
          Text('残り ${yen(remaining)}', style: const TextStyle(fontSize: 12, fontWeight: FontWeight.w700)),
          Text('予算 ${yen(box.monthlyBudget)}', style: const TextStyle(color: NexusColors.textMuted, fontSize: 11)),
          const SizedBox(height: 6),
          SoftProgress(value: ratio, color: box.color, height: 6),
        ],
      ),
    );
  }
}

```

```

}

class _SavingsTile extends StatelessWidget {
  const _SavingsTile({required this.store, required this.box});

  final AppStore store;
  final BudgetBox box;

  @override
  Widget build(BuildContext context) {
    final current = store.savingsBalance(box);
    final ratio = box.targetAmount == 0 ? 0.0 : current / box.targetAmount;
    return GlassCard(
      fill: NexusColors.cream,
      borderColor: box.color.withValues(alpha: 0.28),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Icon(box.icon, color: box.color),
          const SizedBox(height: 8),
          Text(box.name, style: const TextStyle(fontWeight: FontWeight.w700)),
          const Spacer(),
          Text(yen(current), style: const TextStyle(fontSize: 16, fontWeight: FontWeight.w800, color: NexusColors.text)),
          Text(
            '${yen(current)} / ${yen(box.targetAmount)}',
            style: const TextStyle(color: NexusColors.textMuted, fontSize: 11),
          ),
          const SizedBox(height: 6),
          SoftProgress(value: ratio, color: box.color, height: 6),
        ],
      ),
    );
  }
}

```

```

class _DeleteMark extends StatelessWidget {
  const _DeleteMark({required this.onTap});

  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    return Material(
      color: Colors.transparent,
      child: InkWell(
        onTap: onTap,
        customBorder: const CircleBorder(),
        child: Container(
          width: 22,
          height: 22,
          alignment: Alignment.center,
          decoration: BoxDecoration(
            color: NexusColors.surface,
            shape: BoxShape.circle,
            border: Border.all(color: NexusColors.border),
          ),
          child: const Icon(Icons.close, size: 13, color: NexusColors.textMuted),
        ),
      ),
    );
  }
}

```

```

class _Tab extends StatelessWidget {
  const _Tab({required this.label, required this.selected, required this.onTap});

  final String label;
  final bool selected;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    return Expanded(
      child: InkWell(
        onTap: onTap,
        borderRadius: BorderRadius.circular(12),
        child: AnimatedContainer(

```

```
duration: const Duration(milliseconds: 180),
padding: const EdgeInsets.symmetric(vertical: 8),
decoration: BoxDecoration(
  color: selected ? NexusColors.cyan.withValues(alpha: 0.14) : Colors.transparent,
  borderRadius: BorderRadius.circular(12),
  border: Border.all(
    color: selected ? NexusColors.cyan.withValues(alpha: 0.28) : Colors.transparent,
  ),
),
child: Text(
  label,
  textAlign: TextAlign.center,
  style: TextStyle(
    color: selected ? NexusColors.cyan : NexusColors.textMuted,
    fontWeight: selected ? FontWeight.w700 : FontWeight.w500,
    fontSize: 13,
  ),
),
),
),
);
}
```

```

        isExpanded: true,
        items: [
          for (var y = deposited.year - 1; y <= deposited.year + 1; y++)
            DropdownMenuItem(value: y, child: Text('$y年')),
        ],
        onChanged: (v) => setSheet(() => useYear = v ?? useYear),
      ),
    ),
    const SizedBox(width: 8),
    Expanded(
      child: DropdownButton<int>(
        value: useMonth,
        dropdownColor: NexusColors.card,
        isExpanded: true,
        items: [
          for (var m = 1; m <= 12; m++)
            DropdownMenuItem(value: m, child: Text('$m月')),
        ],
        onChanged: (v) => setSheet(() => useMonth = v ?? useMonth),
      ),
    ),
  ],
),
),
TextField(
  controller: memo,
  style: const TextStyle(color: NexusColors.text),
  decoration: const InputDecoration(labelText: 'メモ (任意)'),
),
const SizedBox(height: 12),
FilledButton(onPressed: () => Navigator.pop(context, true), child: const Text('保存')),
],
),
);
},
);
};
final value = int.tryParse(amount.text.replaceAll(',', ''));
if (saved == true && name.text.trim().isNotEmpty && value != null && value > 0) {
  store.addIncome(
    name: name.text.trim(),
    amount: value,
    depositedAt: deposited,
    useYear: useYear,
    useMonth: useMonth,
    memo: memo.text.trim(),
  );
  if (context.mounted) {
    nexusHaptic();
    showNexusToast(context, store.lastToast);
  }
}
name.dispose();
amount.dispose();
memo.dispose();
}

Future<void> openAddBox(BuildContext context, AppStore store) async {
  final kind = await showNexusSheet<BoxKind>(
    context: context,
    builder: (context) {
      return Column(
        mainAxisAlignment: MainAxisAlignment.min,
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          const Text('ボックスの種類', style: TextStyle(fontSize: 18, fontWeight: FontWeight.w600)),
          const SizedBox(height: 12),
          FilledButton(
            onPressed: () => Navigator.pop(context, BoxKind.budget),
            child: const Text('予算ボックス'),
          ),
          const SizedBox(height: 8),
          OutlinedButton(
            onPressed: () => Navigator.pop(context, BoxKind.savings),
            child: const Text('貯蓄ボックス'),
          ),
          const SizedBox(height: 8),

```

```

        const Text(
          '予算は毎月リセット、貯蓄は残高を持ち越します。',
          style: TextStyle(color: NexusColors.textMuted, fontSize: 12),
        ),
      ],
    );
  },
);
if (kind == null || !context.mounted) return;
if (kind == BoxKind.budget) {
  await _openBudgetBoxForm(context, store);
} else {
  await _openSavingsBoxForm(context, store);
}
}
}

Future<void> _openBudgetBoxForm(BuildContext context, AppStore store) async {
  final name = TextEditingController();
  final budget = TextEditingController(text: '30000');
  final memo = TextEditingController();
  final tag = TextEditingController();
  var icon = Icons.restaurant_rounded;
  var color = const Color(0xFF3DA9FC);
  var renewalDay = 1;
  var tags = <String>['その他'];
  final saved = await showNexusSheet<bool>(
    context: context,
    builder: (context) {
      return StatefulBuilder(
        builder: (context, setSheet) {
          return Column(
            mainAxisAlignment: MainAxisAlignment.min,
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              const Text('予算ボックス', style: TextStyle(fontSize: 18, fontWeight: FontWeight.w600)),
              const SizedBox(height: 8),
              TextField(
                controller: name,
                style: const TextStyle(color: NexusColors.text),
                decoration: const InputDecoration(labelText: 'ボックス名'),
              ),
              TextField(
                controller: budget,
                keyboardType: TextInputType.number,
                style: const TextStyle(color: NexusColors.text),
                decoration: const InputDecoration(labelText: '月間予算'),
              ),
              const SizedBox(height: 8),
              const Text('更新日', style: TextStyle(color: NexusColors.textSecondary, fontSize: 12)),
              DropdownButton<int>(
                value: renewalDay,
                dropdownColor: NexusColors.card,
                isExpanded: true,
                items: [
                  for (var d = 1; d <= 31; d++)
                    DropdownMenuItem(value: d, child: Text('毎月$d日')),
                ],
                onChanged: (v) => setSheet(() => renewalDay = v ?? 1),
              ),
              const Text('アイコン', style: TextStyle(color: NexusColors.textSecondary, fontSize: 12)),
              SizedBox(
                height: 96,
                child: SingleChildScrollView(
                  child: Wrap(
                    spacing: 8,
                    children: [
                      for (final i in boxIconChoices)
                        IconButton(
                          onPressed: () => setSheet(() => icon = i),
                          icon: Icon(i, color: i == icon ? color : NexusColors.textMuted),
                        ),
                    ],
                  ),
                ),
              ),
              const Text('色', style: TextStyle(color: NexusColors.textSecondary, fontSize: 12)),
              Padding(

```

```

padding: const EdgeInsets.only(top: 6, bottom: 8),
child: Row(
  children: [
    for (final c in _boxColors)
      GestureDetector(
        onTap: () => setSheet(() => color = c),
        child: Container(
          width: 24,
          height: 24,
          margin: const EdgeInsets.only(right: 8),
          decoration: BoxDecoration(
            color: c,
            shape: BoxShape.circle,
            border: Border.all(
              color: c == color ? NexusColors.text : Colors.transparent,
              width: 2,
            ),
          ),
        ),
      ),
    ],
  ),
),
const Text('タグ', style: TextStyle(color: NexusColors.textSecondary, fontSize: 12)),
Wrap(
  spacing: 6,
  children: [
    for (final t in tags)
      Chip(
        label: Text(t),
        onDelete: t == 'その他'
          ? null
          : () => setSheet(() => tags = [...tags]..remove(t)),
      ),
    ActionChip(
      label: const Text('+ タグを追加'),
      onPressed: () {
        final text = tag.text.trim();
        if (text.isEmpty || tags.contains(text)) return;
        setSheet(() {
          tags = [...tags, text];
          tag.clear();
        });
      },
    ),
  ],
),
TextField(
  controller: tag,
  style: const TextStyle(color: NexusColors.text),
  decoration: const InputDecoration(labelText: '新しいタグ名'),
),
TextField(
  controller: memo,
  style: const TextStyle(color: NexusColors.text),
  decoration: const InputDecoration(labelText: 'メモ'),
),
const SizedBox(height: 12),
FilledButton(onPressed: () => Navigator.pop(context, true), child: const Text('作成')),
],
);
},
);
},
);
final value = int.tryParse(budget.text.replaceAll(',', ''));
if (saved == true && name.text.trim().isEmpty && value != null && value >= 0) {
  store.addBudgetBox(
    name: name.text.trim(),
    icon: icon,
    color: color,
    monthlyBudget: value,
    renewalDay: renewalDay,
    tags: tags,
    memo: memo.text.trim(),
  );
}
if (context.mounted) showNexusToast(context, store.lastToast);

```

```

}
name.dispose();
budget.dispose();
memo.dispose();
tag.dispose();
}

Future<void> _openSavingsBoxForm(BuildContext context, AppStore store) async {
  final name = TextEditingController();
  final target = TextEditingController(text: '150000');
  final current = TextEditingController(text: '0');
  final memo = TextEditingController();
  final tag = TextEditingController();
  var icon = Icons.savings_rounded;
  var color = const Color(0xFFFC857);
  DateTime? targetDate;
  var tags = <String>['積立', 'その他'];
  final saved = await showNexusSheet<bool>(
    context: context,
    builder: (context) {
      return StatefulBuilder(
        builder: (context, setSheet) {
          return Column(
            mainAxisAlignment: MainAxisAlignment.min,
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              const Text('貯蓄ボックス', style: TextStyle(fontSize: 18, fontWeight: FontWeight.w600)),
              const SizedBox(height: 8),
              TextField(
                controller: name,
                style: const TextStyle(color: NexusColors.text),
                decoration: const InputDecoration(labelText: 'ボックス名'),
              ),
              TextField(
                controller: target,
                keyboardType: TextInputType.number,
                style: const TextStyle(color: NexusColors.text),
                decoration: const InputDecoration(labelText: '目標金額'),
              ),
              TextField(
                controller: current,
                keyboardType: TextInputType.number,
                style: const TextStyle(color: NexusColors.text),
                decoration: const InputDecoration(labelText: '現在金額'),
              ),
              OutlinedButton(
                onPressed: () async {
                  final picked = await showDatePicker(
                    context: context,
                    initialDate: store.focusedDate.add(const Duration(days: 90)),
                    firstDate: store.focusedDate,
                    lastDate: DateTime(store.focusedDate.year + 5),
                  );
                  if (picked != null) setSheet(() => targetDate = picked);
                },
                child: Text(targetDate == null ? '目標日 (任意)' : '目標日 ${jpDate(targetDate!)}'),
              ),
              const Text('アイコン', style: TextStyle(color: NexusColors.textSecondary, fontSize: 12)),
              SizedBox(
                height: 96,
                child: SingleChildScrollView(
                  child: Wrap(
                    spacing: 8,
                    children: [
                      for (final i in boxIconChoices)
                        IconButton(
                          onPressed: () => setSheet(() => icon = i),
                          icon: Icon(i, color: i == icon ? color : NexusColors.textMuted),
                        ),
                    ],
                  ),
                ),
              const Text('色', style: TextStyle(color: NexusColors.textSecondary, fontSize: 12)),
              Padding(
                padding: const EdgeInsets.only(top: 6, bottom: 8),
                child: Row(

```



```

        children: [
          for (final c in _boxColors)
            GestureDetector(
              onTap: () => setSheet(() => color = c),
              child: Container(
                width: 24,
                height: 24,
                margin: const EdgeInsets.only(right: 8),
                decoration: BoxDecoration(
                  color: c,
                  shape: BoxShape.circle,
                  border: Border.all(
                    color: c == color ? NexusColors.text : Colors.transparent,
                    width: 2,
                  ),
                ),
              ),
            ),
        ],
      ),
    ),
    const Text('タグ', style: TextStyle(color: NexusColors.textSecondary, fontSize: 12)),
    Wrap(
      spacing: 6,
      children: [
        for (final t in tags)
          Chip(
            label: Text(t),
            onDelete: t == 'その他'
              ? null
              : () => setSheet(() => tags = [...tags]..remove(t)),
          ),
        ActionChip(
          label: const Text('+ タグを追加'),
          onPressed: () {
            final text = tag.text.trim();
            if (text.isEmpty || tags.contains(text)) return;
            setSheet(() {
              tags = [...tags, text];
              tag.clear();
            });
          },
        ),
      ],
    ),
    TextField(
      controller: tag,
      style: const TextStyle(color: NexusColors.text),
      decoration: const InputDecoration(labelText: '新しいタグ名'),
    ),
    TextField(
      controller: memo,
      style: const TextStyle(color: NexusColors.text),
      decoration: const InputDecoration(labelText: 'メモ'),
    ),
    const SizedBox(height: 12),
    FilledButton(onPressed: () => Navigator.pop(context, true), child: const Text('作成')),
  ],
);
};
},
);
},
);
final goal = int.tryParse(target.text.replaceAll(',', ''));
final now = int.tryParse(current.text.replaceAll(',', '')) ?? 0;
if (saved == true && name.text.trim().isNotEmpty && goal != null && goal > 0) {
  store.addSavingsBox(
    name: name.text.trim(),
    icon: icon,
    color: color,
    targetAmount: goal,
    openingAmount: now,
    targetDate: targetDate,
    tags: tags,
    memo: memo.text.trim(),
  );
}
if (context.mounted) showNexusToast(context, store.lastToast);

```



```

    );
    if (picked != null) setSheet(() => at = picked);
  },
  child: Text('日付  ${jpDate(at)}'),
),
const Text('タグ', style: TextStyle(color: NexusColors.textSecondary, fontSize: 12)),
Wrap(
  spacing: 6,
  children: [
    for (final t in tags)
      ChoiceChip(
        label: Text(t),
        selected: tag == t,
        onSelected: (_) => setSheet(() => tag = t),
      ),
    ActionChip(
      label: const Text('+ タグを追加'),
      onPressed: () async {
        await openAddTag(context, store, selected);
        setSheet(() {});
      },
    ),
  ],
),
TextField(
  controller: memo,
  style: const TextStyle(color: NexusColors.text),
  decoration: const InputDecoration(labelText: 'メモ (任意)'),
),
const SizedBox(height: 12),
FilledButton(onPressed: () => Navigator.pop(context, true), child: const Text('保存')),
],
),
);
},
);
},
);
};
final value = int.tryParse(amount.text.replaceAll(',', ''));
final box = store.boxById(selected);
if (saved == true && title.text.trim().isNotEmpty && value != null && value > 0 && box != null) {
  store.addMoneyCard(
    boxId: selected,
    title: title.text.trim(),
    amount: value,
    at: at,
    tag: tag,
    memo: memo.text.trim(),
    kind: box.isSavings
      ? (saveIn ? MoneyCardKind.saveIn : MoneyCardKind.saveOut)
      : MoneyCardKind.spend,
  );
  if (context.mounted) showNexusToast(context, store.lastToast);
}
title.dispose();
amount.dispose();
memo.dispose();
}

```

```

Future<void> openEditCard(BuildContext context, AppStore store, MoneyCard card) async {
  var boxId = card.boxId;
  final title = TextEditingController(text: card.title);
  final amount = TextEditingController(text: '${card.amount}');
  final memo = TextEditingController(text: card.memo);
  var at = card.at;
  var tag = card.tag;
  var kind = card.kind;
  final saved = await showNexusSheet<bool>(
    context: context,
    builder: (context) {
      return StatefulBuilder(
        builder: (context, setSheet) {
          final box = store.boxById(boxId);
          return SingleChildScrollView(
            child: Column(
              mainAxisAlignment: MainAxisAlignment.min,
              crossAxisAlignment: CrossAxisAlignment.stretch,

```

```

children: [
  const Text('カードを編集', style: TextStyle(fontSize: 18, fontWeight: FontWeight.w600)),
  DropdownButton<String>(
    value: boxId,
    dropdownColor: NexusColors.card,
    isExpanded: true,
    items: [
      for (final b in store_boxes) DropdownMenuItem(value: b.id, child: Text(b.name)),
    ],
    onChanged: (v) => setSheet(() {
      boxId = v ?? boxId;
      final next = store.boxById(boxId);
      kind = next?.isSavings == true
        ? (kind == MoneyCardKind.spend ? MoneyCardKind.saveOut : kind)
        : MoneyCardKind.spend;
      tag = '';
    })),
),
TextField(
  controller: title,
  style: const TextStyle(color: NexusColors.text),
  decoration: const InputDecoration(labelText: '内容'),
),
TextField(
  controller: amount,
  keyboardType: TextInputType.number,
  style: const TextStyle(color: NexusColors.text),
  decoration: const InputDecoration(labelText: '金額'),
),
OutlinedButton(
  onPressed: () async {
    final picked = await showDatePicker(
      context: context,
      initialDate: at,
      firstDate: DateTime(at.year - 1),
      lastDate: DateTime(at.year + 1),
    );
    if (picked != null) setSheet(() => at = picked);
  },
  child: Text('日付 ${jpDate(at)}'),
),
Wrap(
  spacing: 6,
  children: [
    for (final t in box?.tags ?? const <String>[])
      ChoiceChip(
        label: Text(t),
        selected: tag == t,
        onSelected: (_) => setSheet(() => tag = t),
      ),
  ],
),
TextField(
  controller: memo,
  style: const TextStyle(color: NexusColors.text),
  decoration: const InputDecoration(labelText: 'メモ'),
),
const SizedBox(height: 12),
FilledButton(onPressed: () => Navigator.pop(context, true), child: const Text('保存')),
],
),
);
},
);
);
);
final value = int.tryParse(amount.text.replaceAll(',', ''));
if (saved == true && title.text.trim().isNotEmpty && value != null && value > 0) {
  store.updateMoneyCard(
    card.copyWith(
      boxId: boxId,
      title: title.text.trim(),
      amount: value,
      at: at,
      tag: tag,
      memo: memo.text.trim(),
      kind: kind,
    ),
  );
}
}
);

```

```

    ),
  );
  if (context.mounted) showNexusToast(context, store.lastToast);
}
title.dispose();
amount.dispose();
memo.dispose();
}

Future<void> openAddTag(BuildContext context, AppStore store, String boxId) async {
  final name = TextEditingController();
  final saved = await showNexusSheet<bool>(
    context: context,
    useRootNavigator: true,
    builder: (context) {
      return Column(
        mainAxisAlignment: MainAxisAlignment.min,
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          const Text('タグを追加', style: TextStyle(fontSize: 18, fontWeight: FontWeight.w600)),
          TextField(
            controller: name,
            autofocus: true,
            style: const TextStyle(color: NexusColors.text),
            decoration: const InputDecoration(labelText: 'タグ名'),
          ),
          const SizedBox(height: 12),
          FilledButton(onPressed: () => Navigator.pop(context, true), child: const Text('追加')),
        ],
      );
    },
  );
  if (saved == true && name.text.trim().isNotEmpty) {
    store.addBoxTag(boxId, name.text.trim());
  }
  name.dispose();
}

Future<void> openAddPayment(BuildContext context, AppStore store) async {
  final title = TextEditingController();
  final amount = TextEditingController();
  final memo = TextEditingController();
  var due = store.focusedDate.add(const Duration(days: 10));
  String? boxId = store.bboxes.any((b) => b.id == 'box-unassigned')
    ? 'box-unassigned'
    : (store.bboxes.isEmpty ? null : store.bboxes.first.id);
  var repeat = PaymentRepeat.none;
  final saved = await showNexusSheet<bool>(
    context: context,
    builder: (context) {
      return StatefulBuilder(
        builder: (context, setSheet) {
          return SingleChildScrollView(
            child: Column(
              mainAxisAlignment: MainAxisAlignment.min,
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                const Text('支払予定を追加', style: TextStyle(fontSize: 18, fontWeight: FontWeight.w600)),
                TextField(
                  controller: title,
                  style: const TextStyle(color: NexusColors.text),
                  decoration: const InputDecoration(labelText: '支払い名'),
                ),
                TextField(
                  controller: amount,
                  keyboardType: TextInputType.number,
                  style: const TextStyle(color: NexusColors.text),
                  decoration: const InputDecoration(labelText: '金額'),
                ),
                OutlinedButton(
                  onPressed: () async {
                    final picked = await showDatePicker(
                      context: context,
                      initialDate: due,
                      firstDate: store.focusedDate,
                      lastDate: DateTime(store.focusedDate.year + 3),
                    );

```

```

        if (picked != null) setSheet(() => due = picked);
    },
    child: Text('支払予定日   ${jpDate(due)}'),
),
DropDownButton<String?>(
    value: boxId,
    dropdownColor: NexusColors.card,
    isExpanded: true,
    items: [
        const DropdownMenuItem(value: null, child: Text('支払い元なし')),
        for (final b in store.bboxes) DropdownMenuItem(value: b.id, child: Text('支払い元: ${b.name}')),
    ],
    onChanged: (v) => setSheet(() => boxId = v),
),
Wrap(
    spacing: 8,
    children: [
        for (final r in PaymentRepeat.values)
            ChoiceChip(
                label: Text(switch (r) {
                    PaymentRepeat.none => 'なし',
                    PaymentRepeat.monthly => '毎月',
                    PaymentRepeat.yearly => '毎年',
                }),
                selected: repeat == r,
                onSelected: (_) => setSheet(() => repeat = r),
            ),
    ],
),
TextField(
    controller: memo,
    style: const TextStyle(color: NexusColors.text),
    decoration: const InputDecoration(labelText: 'メモ'),
),
const SizedBox(height: 12),
FilledButton(onPressed: () => Navigator.pop(context, true), child: const Text('保存')),
],
),
);
},
);
};
};
);
final value = int.tryParse(amount.text.replaceAll(',', ''));
if (saved == true && title.text.trim().isNotEmpty && value != null && value > 0) {
    store.addPaymentPlan(
        title: title.text.trim(),
        amount: value,
        dueAt: due,
        boxId: boxId,
        repeat: repeat,
        memo: memo.text.trim(),
    );
    if (context.mounted) showNexusToast(context, store.lastToast);
}
title.dispose();
amount.dispose();
memo.dispose();
}

```

ファイル: lib/screens/money/box_detail_page.dart

行数: 237

```
import 'package:flutter/material.dart';

import '../..../app/theme.dart';
import '../..../core/format.dart';
import '../..../data/app_store.dart';
import '../..../data/models.dart';
import '../..../widgets/glass_card.dart';
import '../..../widgets/ui_bits.dart';
import 'money_forms.dart';

enum _CardSort { date, amount, tag }

class BoxDetailPage extends StatefulWidget {
  const BoxDetailPage({super.key, required this.boxId});

  final String boxId;

  @override
  State<BoxDetailPage> createState() => _BoxDetailPageState();
}

class _BoxDetailPageState extends State<BoxDetailPage> {
  DateTime _month = dateOnly(DateTime.now());
  _CardSort _sort = _CardSort.date;

  @override
  Widget build(BuildContext context) {
    final store = AppScope.of(context);
    final box = store.boxById(widget.boxId);
    if (box == null) {
      return const Scaffold(body: Center(child: Text('ボックスが見つかりません')));
    }

    var list = store.cardsForBox(box.id, month: box.isSavings ? null : _month);
    list = [...list];
    switch (_sort) {
      case _CardSort.date:
        list.sort((a, b) => b.at.compareTo(a.at));
      case _CardSort.amount:
        list.sort((a, b) => b.amount.compareTo(a.amount));
      case _CardSort.tag:
        list.sort((a, b) => a.tag.compareTo(b.tag));
    }

    return Scaffold(
      backgroundColor: NexusColors.background,
      body: PageScaffold(
        child: ListView(
          padding: const EdgeInsets.fromLTRB(16, 8, 16, 24),
          children: [
            Row(
              children: [
                IconButton(
                  onPressed: () => Navigator.pop(context),
                  icon: const Icon(Icons.close_rounded, color: NexusColors.text),
                ),
                Icon(box.icon, color: box.color),
                const SizedBox(width: 8),
                Expanded(
                  child: Text(
                    box.name,
                    style: const TextStyle(fontSize: 22, fontWeight: FontWeight.w600),
                  ),
                ),
              ],
            ),
            AddChip(
              label: 'カードを追加',
              onTap: () => openAddCard(context, store, boxId: box.id),
            ),
          ],
        ),
      ),
    );
  }
}
```

```

    ),
    const SizedBox(height: 8),
    if (!box.isSavings)
      Row(
        children: [
          IconButton(
            onPressed: () => setState(() => _month = DateTime(_month.year, _month.month - 1, 1)),
            icon: const Icon(Icons.chevron_left, color: NexusColors.cyan),
          ),
          Text(jpMonth(_month), style: const TextStyle(fontWeight: FontWeight.w700)),
          IconButton(
            onPressed: () => setState(() => _month = DateTime(_month.year, _month.month + 1, 1)),
            icon: const Icon(Icons.chevron_right, color: NexusColors.cyan),
          ),
        ],
      ),
    if (box.isSavings) _SavingsHero(store: store, box: box) else _BudgetHero(store: store, box: box, month: _month),
    const SizedBox(height: 12),
    Wrap(
      spacing: 8,
      children: [
        for (final sort in _CardSort.values)
          ChoiceChip(
            label: Text(switch (sort) {
              _CardSort.date => '日付順',
              _CardSort.amount => '金額順',
              _CardSort.tag => 'タグ別',
            })),
            selected: _sort == sort,
            onSelect: (_) => setState(() => _sort = sort),
          ),
      ],
    ),
    const SizedBox(height: 12),
    if (list.isEmpty)
      Padding(
        padding: const EdgeInsets.symmetric(vertical: 24),
        child: Text(
          box.isSavings ? 'まだ貯蓄履歴はありません' : 'この月のカードはありません',
          style: const TextStyle(color: NexusColors.textMuted),
        ),
      ),
    else
      for (final card in list)
        Padding(
          padding: const EdgeInsets.only(bottom: 8),
          child: InkWell(
            onTap: () => openEditCard(context, store, card),
            borderRadius: BorderRadius.circular(16),
            child: GlassCard(
              fill: card.kind == MoneyCardKind.saveIn
                ? NexusColors.sage.withValues(alpha: 0.7)
                : NexusColors.peach.withValues(alpha: 0.7),
              child: Row(
                children: [
                  Expanded(
                    child: Column(
                      crossAxisAlignment: CrossAxisAlignment.start,
                      children: [
                        Text(
                          '${card.at.month}/${card.at.day} ${card.title}',
                          style: const TextStyle(fontWeight: FontWeight.w700),
                        ),
                        if (card.tag.isNotEmpty)
                          Text(card.tag, style: const TextStyle(color: NexusColors.textMuted, fontSize: 12)),
                      ],
                    ),
                  ),
                  Text(
                    _amountLabel(card),
                    style: TextStyle(
                      fontWeight: FontWeight.w600,
                      color: card.kind == MoneyCardKind.saveIn
                        ? NexusColors.income
                        : NexusColors.expense,
                    ),
                  ),
                ],
              ),
            ),
          ),
        ),
      ),
    ),
  ),

```



```

        ],
      ),
    ),
  ),
],
),
);
}

String _amountLabel(MoneyCard card) {
  if (card.kind == MoneyCardKind.saveIn) return '+${yen(card.amount)}';
  if (card.kind == MoneyCardKind.saveOut) return '-${yen(card.amount)}';
  return yen(-card.amount);
}

}

class _BudgetHero extends StatelessWidget {
  const _BudgetHero({required this.store, required this.box, required this.month});

  final AppStore store;
  final BudgetBox box;
  final DateTime month;

  @override
  Widget build(BuildContext context) {
    final spent = store.spentOfBox(box.id, month: month);
    final remain = box.monthlyBudget - spent;
    return GlassCard(
      fill: box.color.withValues(alpha: 0.12),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          const Text('残り', style: TextStyle(color: NexusColors.textSecondary)),
          Text(yen(remain), style: const TextStyle(fontSize: 36, fontWeight: FontWeight.w800)),
          const SizedBox(height: 8),
          Row(
            children: [
              Text('予算 ${yen(box.monthlyBudget)}'),
              const Spacer(),
              Text('使用済み ${yen(spent)}', style: const TextStyle(color: NexusColors.expense)),
            ],
          ),
          const SizedBox(height: 8),
          SoftProgress(
            value: box.monthlyBudget == 0 ? 0 : spent / box.monthlyBudget,
            color: box.color,
          ),
          if (box.memo.isNotEmpty) ...[
            const SizedBox(height: 8),
            Text(box.memo, style: const TextStyle(color: NexusColors.textMuted, fontSize: 12)),
          ],
        ],
      ),
    );
  }
}

class _SavingsHero extends StatelessWidget {
  const _SavingsHero({required this.store, required this.box});

  final AppStore store;
  final BudgetBox box;

  @override
  Widget build(BuildContext context) {
    final current = store.savingsBalance(box);
    final ratio = box.targetAmount == 0 ? 0.0 : (current / box.targetAmount).clamp(0.0, 1.0);
    return GlassCard(
      fill: NexusColors.cream,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          const Text('目標金額', style: TextStyle(color: NexusColors.textSecondary)),
          Text(yen(box.targetAmount), style: const TextStyle(fontSize: 28, fontWeight: FontWeight.w800)),
          const SizedBox(height: 10),

```

```
const Text('現在金額', style: TextStyle(color: NexusColors.textSecondary)),
Text(yen(current), style: const TextStyle(fontSize: 36, fontWeight: FontWeight.w800, color: NexusColors.text)),
if (box.targetDate != null)
  Text('目標日 ${jpDate(box.targetDate!)}', style: const TextStyle(color: NexusColors.textMuted, fontSize: 12)),
const SizedBox(height: 8),
SoftProgress(value: ratio, color: box.color),
const SizedBox(height: 6),
Text('達成率 ${((ratio * 100).round())}%', style: const TextStyle(fontSize: 18, fontWeight: FontWeight.w600)),
],
),
);
}
}
```

ファイル: lib/screens/settings/settings_screen.dart

行数: 220

```
import 'package:flutter/material.dart';

import '../..../app/theme.dart';
import '../..../data/app_store.dart';
import '../..../widgets/glass_card.dart';
import '../..../widgets/nexus_logo.dart';
import '../..../widgets/ui_bits.dart';

class SettingsScreen extends StatelessWidget {
  const SettingsScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final store = AppScope.of(context);
    final s = store.settings;

    return PageScaffold(
      child: ListView(
        padding: const EdgeInsets.fromLTRB(16, 8, 16, 24),
        children: [
          const GradientTitle('設定'),
          const SizedBox(height: 4),
          const Text('Nexusを、自分らしく。', style: TextStyle(color: NexusColors.textSecondary)),
          const SizedBox(height: 14),
          GlassCard(
            fill: NexusColors.peach,
            child: Row(
              children: [
                const NexusLogo(size: 52),
                const SizedBox(width: 12),
                Expanded(
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Text(store.userName, style: const TextStyle(fontWeight: FontWeight.w600, fontSize: 16)),
                      const SizedBox(height: 2),
                      Text('Lv.${store.level}', style: const TextStyle(color: NexusColors.gold, fontSize: 12, letterSpacing:
0.4)),
                      const SizedBox(height: 6),
                      SoftProgress(value: store.levelProgress, color: NexusColors.gold, height: 4),
                    ],
                  ),
                ),
                OutlinedButton(
                  onPressed: () => _editProfile(context, store),
                  child: const Text('プロフィールを編集'),
                ),
              ],
            ),
          ),
          const SizedBox(height: 16),
          const Text('設定項目', style: TextStyle(color: NexusColors.textMuted, fontSize: 12)),
          const SizedBox(height: 8),
          GlassCard(
            padding: EdgeInsets.zero,
            child: Column(
              children: [
                _Item(
                  icon: Icons.grid_view_rounded,
                  title: 'ホームをカスタマイズ',
                  subtitle: 'ウィジェットの順とピン留め',
                  onTap: () => _open(context, 'ホームをカスタマイズ', 'Homeのウィジェットは今日の目標・今月の残高・今週の学習時間で
す。'),
                ),
                _ToggleItem(
                  icon: Icons.notifications_rounded,
                  title: '通知',
                  subtitle: '課題・予定・復習のリマインド',
                  value: s.notifyTasks && s.notifySchedule && s.notifyReview && s.notifyNegumo,
```

```

        onChanged: (v) => store.updateSettings(
          s.copyWith(
            notifyTasks: v,
            notifySchedule: v,
            notifyReview: v,
            notifyNegumo: v,
          ),
        ),
      ),
    ),
    _Item(
      icon: Icons.calendar_month,
      title: '連携',
      subtitle: 'カレンダー・時間割・ヘルスは明示許可',
      onTap: () => _open(context, '連携', '端末カレンダー、学校時間割、ヘルスデータはまだ接続していません。許可するまで読み取
りません。'),
    ),
    _ToggleItem(
      icon: Icons.palette_rounded,
      title: '外観',
      subtitle: 'Reduce Motion',
      value: s.reduceMotion,
      onChanged: (v) => store.updateSettings(s.copyWith(reduceMotion: v)),
    ),
    _Item(
      icon: Icons.lock_rounded,
      title: 'セキュリティ',
      subtitle: 'アプリロック設定',
      onTap: () => _open(context, 'セキュリティ', '端末認証とアプリロックは次のフェーズで接続します。通信は暗号化前提です。'),
    ),
    _Item(
      icon: Icons.help_outline,
      title: 'ヘルプ',
      subtitle: 'FAQ と問い合わせ',
      onTap: () => _open(context, 'ヘルプ', 'Nexus OS 0.1 のFAQです。投資・借入・購入の誘導はありません。'),
    ),
  ],
),
),
const SizedBox(height: 12),
const GlassCard(
  fill: NexusColors.sage,
  child: Row(
    children: [
      Icon(Icons.shield_outlined, color: NexusColors.cyan),
      SizedBox(width: 10),
      Expanded(
        child: Text(
          'あなたのデータは、あなたが管理します。何を保存するかをいつでも確認・変更できます。',
          style: TextStyle(height: 1.4, fontSize: 13),
        ),
      ),
    ],
  ),
),
const SizedBox(height: 12),
const Center(
  child: Text('Nexus OS 0.1', style: TextStyle(color: NexusColors.textMuted, fontSize: 12)),
),
],
),
);
}

```

```

Future<void> _editProfile(BuildContext context, AppStore store) async {
  final controller = TextEditingController(text: store.userName);
  final saved = await showNexusSheet<bool>(
    context: context,
    builder: (context) {
      return Column(
        mainAxisAlignment: MainAxisAlignment.min,
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          const Text('プロフィール', style: TextStyle(fontSize: 18, fontWeight: FontWeight.w700)),
          TextField(controller: controller, style: const TextStyle(color: NexusColors.text)),
          const SizedBox(height: 12),
          FilledButton(onPressed: () => Navigator.pop(context, true), child: const Text('保存')),
        ],
      );
    },
  );
}

```

```

    );
  },
);
if (saved == true && controller.text.trim().isEmpty) {
  store.setUserName(controller.text.trim());
}
controller.dispose();
}

Future<void> _open(BuildContext context, String title, String body) {
  return showNexusSheet<void>(
    context: context,
    builder: (_) => Column(
      mainAxisAlignment: MainAxisAlignment.start,
      children: [
        Text(title, style: const TextStyle(fontSize: 18, fontWeight: FontWeight.w700)),
        const SizedBox(height: 8),
        Text(body, style: const TextStyle(color: NexusColors.textSecondary, height: 1.4)),
      ],
    ),
  );
}

class _Item extends StatelessWidget {
  const _Item({
    required this.icon,
    required this.title,
    required this.subtitle,
    required this.onTap,
  });

  final IconData icon;
  final String title;
  final String subtitle;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    return ListTile(
      leading: AccentIcon(icon),
      title: Text(title),
      subtitle: Text(subtitle, style: const TextStyle(color: NexusColors.textMuted, fontSize: 12)),
      trailing: const Icon(Icons.chevron_right, color: NexusColors.textMuted),
      onTap: onTap,
    );
  }
}

class _ToggleItem extends StatelessWidget {
  const _ToggleItem({
    required this.icon,
    required this.title,
    required this.subtitle,
    required this.value,
    required this.onChanged,
  });

  final IconData icon;
  final String title;
  final String subtitle;
  final bool value;
  final ValueChanged<bool> onChanged;

  @override
  Widget build(BuildContext context) {
    return SwitchListTile(
      secondary: AccentIcon(icon),
      title: Text(title),
      subtitle: Text(subtitle, style: const TextStyle(color: NexusColors.textMuted, fontSize: 12)),
      value: value,
      onChanged: onChanged,
    );
  }
}

```


ファイル: lib/screens/ai/ai_screen.dart

行数: 255

```
import 'package:flutter/material.dart';

import '../..../app/theme.dart';
import '../..../data/app_store.dart';
import '../..../data/models.dart';
import '../..../widgets/glass_card.dart';
import '../..../widgets/ui_bits.dart';
import 'negumo_mascot.dart';

class AiScreen extends StatefulWidget {
  const AiScreen({super.key});

  @override
  State<AiScreen> createState() => _AiScreenState();
}

class _AiScreenState extends State<AiScreen> {
  final _input = TextEditingController();

  @override
  void dispose() {
    _input.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    final store = AppScope.of(context);
    final memory = [
      if (store.settings.memoryStudy) '学習',
      if (store.settings.memorySchedule) '予定',
      if (store.settings.memoryMoney) 'お金',
      if (store.settings.memoryLife) '生活',
    ].join('・');

    return PageScaffold(
      child: Column(
        children: [
          Expanded(
            child: ListView(
              padding: const EdgeInsets.fromLTRB(16, 8, 16, 12),
              children: [
                Row(
                  children: [
                    const GradientTitle('Negumo'),
                    const Spacer(),
                    Icon(Icons.lock, size: 14, color: NexusColors.green.withValues(alpha: 0.9)),
                    const SizedBox(width: 4),
                    const Text('セキュア接続中', style: TextStyle(color: NexusColors.green, fontSize: 11)),
                  ],
                ),
                const Text('次の一步を、一緒に考える。', style: TextStyle(color: NexusColors.textSecondary)),
                const SizedBox(height: 4),
                const Text(
                  'Negumo - powered by Nexus AI',
                  style: TextStyle(color: NexusColors.textMuted, fontSize: 11),
                ),
                const SizedBox(height: 12),
                const Center(child: NegumoMascot()),
                const SizedBox(height: 12),
                GlassCard(
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Text(store.messages.last.text, style: const TextStyle(height: 1.4)),
                      const SizedBox(height: 10),
                      Align(
                        alignment: Alignment.centerRight,
                        child: OutlinedButton(
```

```

        onPressed: () => _confirmPlan(context, store),
        child: const Text('計画を見直す'),
      ),
    ],
  ),
),
const SizedBox(height: 12),
Row(
  children: [
    _Quick(label: '今日何すればいい?', icon: Icons.today, onTap: () => _send(store, '今日何すればいい?')),
    _Quick(label: '学習計画を作る', icon: Icons.menu_book, onTap: () => _send(store, '学習計画を作る')),
    _Quick(label: '予定を整理', icon: Icons.checklist, onTap: () => _send(store, '予定を整理')),
    _Quick(label: '相談する', icon: Icons.chat_bubble_outline, onTap: () => _send(store, '相談する')),
  ],
),
const SizedBox(height: 12),
if (store.proposal != null)
  GlassCard(
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        const Text('ネグモの提案', style: TextStyle(color: NexusColors.cyan, fontWeight: FontWeight.w700)),
        const SizedBox(height: 8),
        Text(store.proposal!.summary, style: const TextStyle(height: 1.4)),
        const SizedBox(height: 6),
        Text(
          '根拠: ${store.proposal!.rationale}',
          style: const TextStyle(color: NexusColors.textMuted, fontSize: 12),
        ),
        const SizedBox(height: 8),
        Text(
          store.proposal!.status == ProposalStatus.approved
            ? '承認済み'
            : store.proposal!.status == ProposalStatus.rejected
              ? '未反映'
              : '未承認（データはまだ変わっていません）',
          style: const TextStyle(color: NexusColors.textSecondary, fontSize: 12),
        ),
        Align(
          alignment: Alignment.centerRight,
          child: TextButton(
            onPressed: store.proposal!.status == ProposalStatus.pending
              ? () => _confirmPlan(context, store)
              : null,
            child: const Text('プランを確認 >'),
          ),
        ),
      ],
    ),
  ),
const SizedBox(height: 8),
Row(
  children: [
    const Icon(Icons.shield_outlined, size: 14, color: NexusColors.cyan),
    const SizedBox(width: 6),
    Text('記憶: $memoryのみ', style: const TextStyle(color: NexusColors.textMuted, fontSize: 11)),
  ],
),
],
),
),
Padding(
  padding: const EdgeInsets.fromLTRB(16, 0, 16, 12),
  child: Row(
    children: [
      Expanded(
        child: TextField(
          controller: _input,
          style: const TextStyle(color: NexusColors.text),
          decoration: InputDecoration(
            hintText: 'ネグモに聞く...',
            hintStyle: const TextStyle(color: NexusColors.textMuted),
            filled: true,
            fillColor: NexusColors.surface,
            border: OutlineInputBorder(
              borderRadius: BorderRadius.circular(24),

```



```

        borderSide: BorderSide(color: NexusColors.border),
      ),
    ),
  ),
),
const SizedBox(width: 8),
IconButton.filled(
  onPressed: () {
    final text = _input.text.trim();
    if (text.isEmpty) return;
    _send(store, text);
    _input.clear();
  },
  icon: const Icon(Icons.send_rounded),
),
],
),
),
],
),
);
}

void _send(AppStore store, String text) {
  store.sendMessage(text);
  store.createDefaultProposal();
}

Future<void> _confirmPlan(BuildContext context, AppStore store) async {
  final p = store.proposal;
  if (p == null) return;
  final schedule = store.schedules.cast<ScheduleItem?>().firstWhere(
    (s) => s?.id == p.scheduleId,
    orElse: () => null,
  );
  final result = await showNexusSheet<String>(
    context: context,
    builder: (context) {
      return Column(
        mainAxisAlignment: MainAxisAlignment.min,
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          const Text('プランを確認', style: TextStyle(fontSize: 18, fontWeight: FontWeight.w700)),
          const SizedBox(height: 8),
          Text(p.summary),
          const SizedBox(height: 8),
          if (schedule != null)
            Text(
              '変更: ${schedule.title} ${schedule.startAt.hour}:${schedule.startAt.minute.toString().padLeft(2, '0')} →
              ${p.newStartAt.hour}:${p.newStartAt.minute.toString().padLeft(2, '0')}',
              style: const TextStyle(color: NexusColors.textSecondary),
            ),
          const SizedBox(height: 12),
          FilledButton(
            onPressed: () => Navigator.pop(context, 'ok'),
            child: const Text('承認して反映'),
          ),
          TextButton(
            onPressed: () => Navigator.pop(context, 'no'),
            child: const Text('今は反映しない'),
          ),
        ],
      );
    },
  );
  if (result == 'ok') {
    store.approveProposal();
    if (context.mounted) showNexusToast(context, store.lastToast);
  } else if (result == 'no') {
    store.rejectProposal();
    if (context.mounted) showNexusToast(context, store.lastToast);
  }
}
}

class _Quick extends StatelessWidget {
  const _Quick({required this.label, required this.icon, required this.onTap});

```

```
final String label;
final IconData icon;
final VoidCallback onTap;

@override
Widget build(BuildContext context) {
  return Expanded(
    child: Padding(
      padding: const EdgeInsets.symmetric(horizontal: 3),
      child: InkWell(
        onTap: onTap,
        borderRadius: BorderRadius.circular(12),
        child: GlassCard(
          padding: const EdgeInsets.symmetric(vertical: 12, horizontal: 6),
          child: Column(
            children: [
              Icon(icon, color: NexusColors.cyan, size: 18),
              const SizedBox(height: 6),
              Text(
                label,
                textAlign: TextAlign.center,
                style: const TextStyle(fontSize: 10, height: 1.2),
              ),
            ],
          ),
        ),
      ),
    ),
  );
}
```

ファイル: lib/screens/ai/negumo_mascot.dart

行数: 72

```
import 'package:flutter/material.dart';

import '../..../app/theme.dart';

class NegumoMascot extends StatelessWidget {
  const NegumoMascot({super.key});

  @override
  Widget build(BuildContext context) {
    return const SizedBox(
      width: 168,
      height: 168,
      child: CustomPaint(painter: _NegumoPainter()),
    );
  }
}

class _NegumoPainter extends CustomPainter {
  const _NegumoPainter();

  @override
  void paint(Canvas canvas, Size size) {
    final c = Offset(size.width / 2, size.height / 2 + 8);
    final body = Paint()
      ..shader = RadialGradient(
        colors: [
          const Color(0xFFEAF4FF).withValues(alpha: 0.95),
          const Color(0xFF9AD7FF).withValues(alpha: 0.55),
          const Color(0xFF3BA7FF).withValues(alpha: 0.18),
        ],
      ).createShader(Rect.fromCircle(center: c, radius: 70));

    canvas.drawOval(Rect.fromCenter(center: c, width: 118, height: 128), body);

    final antenna = Paint()
      ..color = const Color(0xFF7BE7FF)
      ..strokeWidth = 3
      ..strokeCap = StrokeCap.round
      ..style = PaintingStyle.stroke;
    canvas.drawLine(c.translate(-22, -58), c.translate(-34, -78), antenna);
    canvas.drawLine(c.translate(22, -58), c.translate(36, -76), antenna);
    canvas.drawCircle(c.translate(-34, -78), 4, Paint()..color = NexusColors.cyan);
    canvas.drawCircle(c.translate(36, -76), 4, Paint()..color = NexusColors.cyan);

    canvas.drawOval(
      Rect.fromCenter(center: c.translate(-16, -10), width: 12, height: 18),
      Paint()..color = const Color(0xFF101418),
    );
    canvas.drawOval(
      Rect.fromCenter(center: c.translate(16, -10), width: 12, height: 18),
      Paint()..color = const Color(0xFF101418),
    );
    canvas.drawCircle(c.translate(-22, 8), 8, Paint()..color = const Color(0xFFFF8AA0).withValues(alpha: 0.55));
    canvas.drawCircle(c.translate(22, 8), 8, Paint()..color = const Color(0xFFFF8AA0).withValues(alpha: 0.55));

    final starPaint = Paint()
      ..color = NexusColors.cyan
      ..style = PaintingStyle.fill;
    final starPath = Path()
      ..moveTo(c.dx, c.dy + 16)
      ..lineTo(c.dx + 11, c.dy + 28)
      ..lineTo(c.dx, c.dy + 40)
      ..lineTo(c.dx - 11, c.dy + 28)
      ..close();
    canvas.drawPath(starPath, starPaint);
    canvas.drawCircle(Offset(c.dx, c.dy + 28), 6, Paint()..color = Colors.white.withValues(alpha: 0.9));
  }
}

@override
```

```
bool shouldRepaint(covariant CustomPainter oldDelegate) => false;  
}
```

ファイル: lib/widgets/ui_bits.dart

行数: 480

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

import '../app/theme.dart';

void nexusHaptic() {
  HapticFeedback.lightImpact();
}

class PageScaffold extends StatelessWidget {
  const PageScaffold({super.key, required this.child});

  final Widget child;

  @override
  Widget build(BuildContext context) {
    return Stack(
      children: [
        const Positioned.fill(
          child: DecoratedBox(
            decoration: BoxDecoration(
              gradient: LinearGradient(
                begin: Alignment.topCenter,
                end: Alignment.bottomCenter,
                colors: [Color(0xFFF7F4EE), NexusColors.background],
              ),
            ),
          ),
        ),
        Positioned(
          top: -80,
          left: -30,
          child: IgnorePointer(
            child: Container(
              width: 220,
              height: 220,
              decoration: BoxDecoration(
                shape: BoxShape.circle,
                gradient: RadialGradient(
                  colors: [
                    NexusColors.peach.withValues(alpha: 0.7),
                    Colors.transparent,
                  ],
                ),
              ),
            ),
          ),
        ),
        Positioned(
          top: 220,
          right: -70,
          child: IgnorePointer(
            child: Container(
              width: 200,
              height: 200,
              decoration: BoxDecoration(
                shape: BoxShape.circle,
                gradient: RadialGradient(
                  colors: [
                    NexusColors.sky.withValues(alpha: 0.8),
                    Colors.transparent,
                  ],
                ),
              ),
            ),
          ),
        ),
        SafeArea(bottom: false, child: child),
      ],
    ),
  )
}
```

```

    );
  }
}

class GradientTitle extends StatelessWidget {
  const GradientTitle(this.text, {super.key, this.size = 32});

  final String text;
  final double size;

  @override
  Widget build(BuildContext context) {
    return Text(
      text,
      style: TextStyle(
        fontSize: size,
        fontWeight: FontWeight.w800,
        color: NexusColors.text,
        letterSpacing: -0.6,
        height: 1.1,
      ),
    );
  }
}

class AddChip extends StatelessWidget {
  const AddChip({super.key, required this.label, required this.onTap});

  final String label;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    return Material(
      color: Colors.transparent,
      child: InkWell(
        onTap: onTap,
        borderRadius: BorderRadius.circular(20),
        child: Container(
          padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 6),
          decoration: BoxDecoration(
            borderRadius: BorderRadius.circular(20),
            color: NexusColors.sky,
          ),
          child: Row(
            mainAxisAlignment: MainAxisAlignment.min,
            children: [
              const Icon(Icons.add_rounded, size: 14, color: NexusColors.cyan),
              const SizedBox(width: 4),
              Text(
                label,
                style: const TextStyle(
                  color: NexusColors.cyan,
                  fontSize: 11,
                  fontWeight: FontWeight.w700,
                  letterSpacing: 0.1,
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}

class SectionRow extends StatelessWidget {
  const SectionRow({
    super.key,
    required this.title,
    this.trailing,
  });

  final String title;
  final Widget? trailing;

  @override

```

```

Widget build(BuildContext context) {
  return Row(
    children: [
      Expanded(
        child: Text(
          title,
          style: const TextStyle(
            color: NexusColors.text,
            fontSize: 15,
            fontWeight: FontWeight.w600,
            letterSpacing: 0.3,
          ),
        ),
      ),
      ?trailing,
    ],
  );
}

class EmptyHint extends StatelessWidget {
  const EmptyHint({
    super.key,
    required this.text,
    this.icon = Icons.auto_awesome_outlined,
  });

  final String text;
  final IconData icon;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 8),
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          Icon(icon, size: 22, color: NexusColors.textMuted.withValues(alpha: 0.85)),
          const SizedBox(height: 8),
          Text(
            text,
            textAlign: TextAlign.center,
            style: const TextStyle(color: NexusColors.textMuted, fontSize: 13, height: 1.4),
          ),
        ],
      ),
    );
  }
}

class TimePill extends StatelessWidget {
  const TimePill(this.text, {super.key, this.color = NexusColors.cyan});

  final String text;
  final Color color;

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
      decoration: BoxDecoration(
        color: color.withValues(alpha: 0.12),
        borderRadius: BorderRadius.circular(8),
      ),
      child: Text(
        text,
        style: TextStyle(color: color, fontSize: 12, fontWeight: FontWeight.w800),
      ),
    );
  }
}

class SoftProgress extends StatelessWidget {
  const SoftProgress({
    super.key,
    required this.value,
    this.color = NexusColors.cyan,
  });
}

```

```

        this.height = 8,
    });

    final double value;
    final Color color;
    final double height;

    @override
    Widget build(BuildContext context) {
        return ClipRRect(
            borderRadius: BorderRadius.circular(99),
            child: LinearProgressIndicator(
                value: value.clamp(0, 1),
                minHeight: height,
                color: color,
                backgroundColor: NexusColors.border.withValues(alpha: 0.7),
            ),
        );
    }
}

class AccentIcon extends StatelessWidget {
    const AccentIcon(this.icon, {super.key, this.color = NexusColors.cyan});

    final IconData icon;
    final Color color;

    @override
    Widget build(BuildContext context) {
        return Container(
            width: 36,
            height: 36,
            alignment: Alignment.center,
            decoration: BoxDecoration(
                color: color.withValues(alpha: 0.12),
                borderRadius: BorderRadius.circular(11),
                border: Border.all(color: color.withValues(alpha: 0.22)),
            ),
            child: Icon(icon, size: 18, color: color),
        );
    }
}

class ThingsCheck extends StatelessWidget {
    const ThingsCheck({
        super.key,
        required this.checked,
        this.color = NexusColors.cyan,
        this.size = 22,
    });

    final bool checked;
    final Color color;
    final double size;

    @override
    Widget build(BuildContext context) {
        return AnimatedContainer(
            duration: const Duration(milliseconds: 180),
            curve: Curves.easeOutCubic,
            width: size,
            height: size,
            decoration: BoxDecoration(
                color: checked ? color : Colors.white,
                borderRadius: BorderRadius.circular(size * 0.28),
                border: Border.all(
                    color: checked ? color : const Color(0xFFD8D2C8),
                    width: checked ? 0 : 1.7,
                ),
            ),
            boxShadow: checked
                ? [
                    BoxShadow(
                        color: color.withValues(alpha: 0.28),
                        blurRadius: 7,
                        offset: const Offset(0, 2),
                    ),
                ]
            : []
        );
    }
}

```



```

        : null,
    ),
    alignment: Alignment.center,
    child: checked
      ? Icon(Icons.check_rounded, size: size * 0.7, color: Colors.white)
      : null,
  );
}
}

class SoftTile extends StatelessWidget {
  const SoftTile({
    super.key,
    required this.child,
    this.color,
    this.onTap,
    this.padding = const EdgeInsets.fromLTRB(10, 10, 10, 10),
  });

  final Widget child;
  final Color? color;
  final VoidCallback? onTap;
  final EdgeInsetsGeometry padding;

  @override
  Widget build(BuildContext context) {
    final radius = BorderRadius.circular(14);
    final body = Container(
      padding: padding,
      decoration: BoxDecoration(
        color: color ?? NexusColors.sky.withValues(alpha: 0.55),
        borderRadius: radius,
      ),
      child: child,
    );
    if (onTap == null) return body;
    return InkWell(onTap: onTap, borderRadius: radius, child: body);
  }
}

class StatChip extends StatelessWidget {
  const StatChip({super.key, required this.label, required this.value, required this.color});

  final String label;
  final String value;
  final Color color;

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 5),
      decoration: BoxDecoration(
        color: color.withValues(alpha: 0.12),
        borderRadius: BorderRadius.circular(10),
      ),
      child: Row(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          Text(label, style: TextStyle(color: color, fontSize: 10, fontWeight: FontWeight.w700)),
          const SizedBox(width: 6),
          Text(value, style: TextStyle(color: color, fontSize: 11, fontWeight: FontWeight.w800)),
        ],
      ),
    );
  }
}

class TrackBar extends StatelessWidget {
  const TrackBar({
    super.key,
    required this.value,
    this.color = NexusColors.cyan,
    this.width,
  });

  final double value;
  final Color color;

```

```

final double? width;

@override
Widget build(BuildContext context) {
  return LayoutBuilder(
    builder: (context, box) {
      final h = box.maxHeight * value.clamp(0.0, 1.0);
      return Align(
        alignment: Alignment.bottomCenter,
        child: SizedBox(
          width: width ?? double.infinity,
          child: Stack(
            alignment: Alignment.bottomCenter,
            children: [
              Container(
                height: box.maxHeight,
                decoration: BoxDecoration(
                  color: Colors.white.withValues(alpha: 0.62),
                  borderRadius: BorderRadius.circular(8),
                ),
              ),
              AnimatedContainer(
                duration: const Duration(milliseconds: 480),
                curve: Curves.easeOutCubic,
                height: h,
                decoration: BoxDecoration(
                  color: color,
                  borderRadius: BorderRadius.circular(8),
                ),
              ),
            ],
          ),
        ),
      );
    },
  );
}

Future<T?> showNexusSheet<T>({
  required BuildContext context,
  required WidgetBuilder builder,
  bool useRootNavigator = false,
}) {
  return showModalBottomSheet<T>({
    context: context,
    isScrollControlled: true,
    isDismissible: true,
    enableDrag: true,
    showDragHandle: true,
    useRootNavigator: useRootNavigator,
    backgroundColor: NexusColors.card,
    barrierColor: Colors.black.withValues(alpha: 0.28),
    clipBehavior: Clip.antiAlias,
    shape: const RoundedRectangleBorder(
      borderRadius: BorderRadius.vertical(top: Radius.circular(24)),
    ),
    builder: (sheetContext) {
      final media = MediaQuery.of(sheetContext);
      final maxBody = (media.size.height * 0.82 - media.viewInsets.bottom).clamp(160.0, 720.0);
      return Padding(
        padding: EdgeInsets.fromLTRB(16, 0, 8, media.viewInsets.bottom + 20),
        child: ConstrainedBox(
          constraints: BoxConstraints(maxHeight: maxBody),
          child: Stack(
            alignment: Alignment.topCenter,
            children: [
              ListView(
                shrinkWrap: true,
                primary: false,
                padding: const EdgeInsets.only(top: 4, right: 40, left: 0, bottom: 8),
                children: [builder(sheetContext)],
              ),
              Positioned(
                top: -8,
                right: 0,
                child: IconButton(

```

```
        visualDensity: VisualDensity.compact,  
        tooltip: '閉じる',  
        onPressed: () => Navigator.pop(sheetContext),  
        icon: const Icon(Icons.close_rounded, size: 20, color: NexusColors.textMuted),  
      ),  
    ),  
  ],  
),  
),  
);  
},  
);  
}  
}  
  
void showNexusToast(BuildContext context, String message) {  
  if (message.isEmpty) return;  
  ScaffoldMessenger.of(context).hideCurrentSnackBar();  
  ScaffoldMessenger.of(context).showSnackBar(  
    SnackBar(  
      content: Text(message, style: const TextStyle(color: Colors.white)),  
      behavior: SnackBarBehavior.floating,  
      backgroundColor: NexusColors.text,  
    ),  
  );  
}
```

ファイル: lib/widgets/glass_card.dart

行数: 81

```
import 'package:flutter/material.dart';

import '../app/theme.dart';

class GlassCard extends StatelessWidget {
  const GlassCard({
    super.key,
    required this.child,
    this.padding = const EdgeInsets.all(16),
    this.borderColor,
    this.glowColor,
    this.fill,
    this.height,
  });

  final Widget child;
  final EdgeInsetsGeometry padding;
  final Color? borderColor;
  final Color? glowColor;
  final Color? fill;
  final double? height;

  @override
  Widget build(BuildContext context) {
    final radius = BorderRadius.circular(NexusColors.cardRadius);
    return Container(
      decoration: BoxDecoration(
        borderRadius: radius,
        boxShadow: [
          BoxShadow(
            color: const Color(0x14000000),
            blurRadius: 22,
            offset: const Offset(0, 10),
          ),
          if (glowColor != null)
            BoxShadow(
              color: glowColor!.withValues(alpha: 0.16),
              blurRadius: 18,
              offset: const Offset(0, 6),
            ),
        ],
      ),
      child: Material(
        color: fill ?? NexusColors.card,
        elevation: 0,
        shadowColor: Colors.transparent,
        shape: RoundedRectangleBorder(
          borderRadius: radius,
          side: BorderSide(color: borderColor ?? NexusColors.border.withValues(alpha: 0.85)),
        ),
        clipBehavior: Clip.antiAlias,
        child: SizedBox(
          height: height,
          child: Stack(
            children: [
              const Positioned(
                top: 0,
                left: 0,
                right: 0,
                height: 40,
                child: IgnorePointer(
                  child: DecoratedBox(
                    decoration: BoxDecoration(
                      gradient: LinearGradient(
                        begin: Alignment.topCenter,
                        end: Alignment.bottomCenter,
                        colors: [Color(0xAFFFFFFF), Color(0x00FFFFFF)],
                      ),
                    ),
                  ),
                ),
            ],
          ),
        ),
      ),
    );
  }
}
```

```
        ),
      ),
    ),
    Padding(padding: padding, child: child),
  ],
),
),
),
);
}
```

ファイル: lib/widgets/nexus_nav_bar.dart

行数: 127

```
import 'package:flutter/material.dart';

import '../app/theme.dart';
import 'ui_bits.dart';

class NexusTab {
  NexusTab._();

  static const home = 0;
  static const study = 1;
  static const life = 2;
  static const money = 3;
  static const settings = 4;
}

class NexusNavBar extends StatelessWidget {
  const NexusNavBar({
    super.key,
    required this.currentIndex,
    required this.onTap,
  });

  final int currentIndex;
  final ValueChanged<int> onTap;

  static const _items = [
    (Icons.home_rounded, 'Home'),
    (Icons.menu_book_rounded, 'Study'),
    (Icons.favorite_rounded, 'Life'),
    (Icons.account_balance_wallet_rounded, 'Money'),
    (Icons.settings_rounded, '設定'),
  ];

  @override
  Widget build(BuildContext context) {
    final bottom = MediaQuery.paddingOf(context).bottom;

    return Padding(
      padding: EdgeInsets.fromLTRB(14, 0, 14, bottom > 0 ? bottom : 10),
      child: DecoratedBox(
        decoration: BoxDecoration(
          color: NexusColors.navBar,
          borderRadius: BorderRadius.circular(28),
          border: Border.all(color: NexusColors.border),
          boxShadow: const [
            BoxShadow(
              color: Color(0x1A2C2A28),
              blurRadius: 24,
              offset: Offset(0, 8),
            ),
          ],
        ),
        child: SizedBox(
          height: 62,
          child: Row(
            children: [
              for (var i = 0; i < _items.length; i++)
                Expanded(
                  child: _NavItem(
                    icon: _items[i].$1,
                    label: _items[i].$2,
                    selected: currentIndex == i,
                    onTap: () {
                      nexusHaptic();
                      onTap(i);
                    },
                  ),
                ),
            ],
          ),
        ),
      ),
    );
  }
}
```

```

    ),
  ),
),
);
}
}

class _NavItem extends StatelessWidget {
  const _NavItem({
    required this.icon,
    required this.label,
    required this.selected,
    required this.onTap,
  });

  final IconData icon;
  final String label;
  final bool selected;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    final color = selected ? NexusColors.cyan : NexusColors.textMuted;

    return GestureDetector(
      behavior: HitTestBehavior.opaque,
      onTap: onTap,
      child: Padding(
        padding: const EdgeInsets.symmetric(horizontal: 4, vertical: 6),
        child: AnimatedContainer(
          duration: const Duration(milliseconds: 200),
          curve: Curves.easeOutCubic,
          decoration: BoxDecoration(
            color: selected ? NexusColors.sky : Colors.transparent,
            borderRadius: BorderRadius.circular(18),
          ),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Icon(icon, size: 20, color: color),
              const SizedBox(height: 2),
              Text(
                label,
                style: TextStyle(
                  fontSize: 10,
                  color: color,
                  fontWeight: selected ? FontWeight.w700 : FontWeight.w500,
                  letterSpacing: 0.1,
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}

```

ファイル: lib/widgets/nexus_logo.dart

行数: 135

```
import 'dart:ui' as ui;

import 'package:flutter/material.dart';

class NexusLogo extends StatelessWidget {
  const NexusLogo({super.key, this.size = 44});

  final double size;

  @override
  Widget build(BuildContext context) {
    final radius = size * 0.24;
    return SizedBox(
      width: size,
      height: size,
      child: DecoratedBox(
        decoration: BoxDecoration(
          borderRadius: BorderRadius.circular(radius),
          boxShadow: [
            BoxShadow(
              color: const Color(0x1A2C2A28),
              blurRadius: size * 0.28,
              offset: Offset(0, size * 0.08),
            ),
          ],
        ),
        child: ClipRRect(
          borderRadius: BorderRadius.circular(radius),
          child: Image.asset(
            'assets/branding/nexus_mark.png',
            width: size,
            height: size,
            fit: BoxFit.cover,
            errorBuilder: (_, __, _) => CustomPaint(
              size: Size.square(size),
              painter: NexusMarkPainter(),
            ),
          ),
        ),
      ),
    );
  }
}

class NexusMarkPainter extends CustomPainter {
  @override
  void paint(Canvas canvas, Size size) {
    final r = size.width * 0.24;
    final rect = Offset.zero & size;
    canvas.drawRRect(
      RRect.fromRectAndRadius(rect, Radius.circular(r)),
      Paint()
        ..shader = ui.Gradient.linear(
          Offset(0, 0),
          Offset(size.width, size.height),
          const [Color(0xFF4C8DFF), Color(0xFF6FA4FF), Color(0xFF8B8FD9)],
        ),
    );

    final highlight = Paint()
      ..shader = ui.Gradient.linear(
        Offset(size.width * 0.15, 0),
        Offset(size.width * 0.85, size.height * 0.4),
        [
          Colors.white.withValues(alpha: 0.16),
          Colors.transparent,
        ],
      );
    canvas.drawRRect(
```



```

        RRect.fromRectAndRadius(rect.deflate(1), Radius.circular(r)),
        highlight,
    );

    final inset = size.width * 0.18;
    final thick = size.width * 0.17;
    final top = size.height * 0.2;
    final bottom = size.height * 0.82;
    final left = inset;
    final right = size.width - inset;

    final nShader = ui.Gradient.linear(
      Offset(size.width * 0.3, top),
      Offset(size.width * 0.7, bottom),
      const [
        Colors.white,
        Color(0xFFFF4F8FF),
        Colors.white,
        Color(0xFFEAF2FF),
      ],
      const [0, 0.35, 0.7, 1],
    );

    final paint = Paint()
      ..shader = nShader
      ..style = PaintingStyle.fill;

    final leftBar = Path()
      ..addRRect(
        RRect.fromLTRBR(left, top, left + thick, bottom, Radius.circular(thick * 0.28)),
      );
    canvas.drawPath(leftBar, paint);

    final rightBar = Path()
      ..addRRect(
        RRect.fromLTRBR(right - thick, top, right, bottom, Radius.circular(thick * 0.28)),
      );
    canvas.drawPath(rightBar, paint);

    final diagonal = Path()
      ..moveTo(left + thick * 0.15, top + thick * 0.2)
      ..lineTo(left + thick * 1.05, top + thick * 0.05)
      ..lineTo(right - thick * 0.12, bottom - thick * 0.15)
      ..lineTo(right - thick * 1.05, bottom - thick * 0.02)
      ..close();
    canvas.drawPath(diagonal, paint);

    final shine = Paint()
      ..shader = ui.Gradient.linear(
        Offset(left, top),
        Offset(right, top + size.height * 0.25),
        [
          Colors.white.withValues(alpha: 0.55),
          Colors.white.withValues(alpha: 0.0),
        ],
      );
    canvas.saveLayer(rect, Paint());
    canvas.drawPath(leftBar, shine);
    canvas.drawPath(diagonal, shine);
    canvas.restore();
  }

  @override
  bool shouldRepaint(covariant CustomPainter oldDelegate) => false;
}

```

ファイル: lib/widgets/progress_ring.dart

行数: 89

```
import 'dart:math' as math;

import 'package:flutter/material.dart';

import '../app/theme.dart';

class ProgressRing extends StatelessWidget {
  const ProgressRing({
    super.key,
    required this.progress,
    this.size = 96,
    this.stroke = 8,
    this.colors = const [NexusColors.cyan, Color(0xFF7EB3FF), NexusColors.cyan],
    this.child,
    this.animate = true,
  });

  final double progress;
  final double size;
  final double stroke;
  final List<Color> colors;
  final Widget? child;
  final bool animate;

  @override
  Widget build(BuildContext context) {
    final target = progress.clamp(0.0, 1.0);
    Widget ring(double value) {
      return SizedBox(
        width: size,
        height: size,
        child: CustomPaint(
          painter: _RingPainter(progress: value, stroke: stroke, colors: colors),
          child: Center(child: child),
        ),
      );
    }

    if (!animate) return ring(target);
    return TweenAnimationBuilder<double>({
      tween: Tween(begin: 0, end: target),
      duration: Duration(milliseconds: target >= 0.999 ? 720 : 480),
      curve: Curves.easeOutCubic,
      builder: (context, value, _) => ring(value),
    });
  }
}

class _RingPainter extends CustomPainter {
  _RingPainter({required this.progress, required this.stroke, required this.colors});

  final double progress;
  final double stroke;
  final List<Color> colors;

  @override
  void paint(Canvas canvas, Size size) {
    final center = Offset(size.width / 2, size.height / 2);
    final radius = size.width / 2 - stroke / 2;

    final track = Paint()
      ..color = const Color(0xFFE6E1D8)
      ..style = PaintingStyle.stroke
      ..strokeWidth = stroke
      ..strokeCap = StrokeCap.round;
    canvas.drawCircle(center, radius, track);

    final rect = Rect.fromCircle(center: center, radius: radius);
    final sweep = Paint()
```

```
    ..shader = SweepGradient(  
        startAngle: -math.pi / 2,  
        endAngle: math.pi * 1.5,  
        colors: colors,  
    ).createShader(rect)  
    ..style = PaintingStyle.stroke  
    ..strokeWidth = stroke  
    ..strokeCap = StrokeCap.round;  
  
    canvas.drawArc(rect, -math.pi / 2, progress * math.pi * 2, false, sweep);  
}  
  
@override  
bool shouldRepaint(covariant _RingPainter oldDelegate) {  
    return oldDelegate.progress != progress ||  
        oldDelegate.stroke != stroke ||  
        oldDelegate.colors != colors;  
}  
}
```

ファイル: lib/widgets/schedule_sheet.dart

行数: 140

```
import 'package:flutter/material.dart';

import '../app/theme.dart';
import '../core/format.dart';
import '../data/models.dart';
import '../data/app_store.dart';
import 'ui_bits.dart';

class ScheduleEditSheet extends StatefulWidget {
  const ScheduleEditSheet({super.key, this.initial});

  final ScheduleItem? initial;

  @override
  State<ScheduleEditSheet> createState() => _ScheduleEditSheetState();
}

class _ScheduleEditSheetState extends State<ScheduleEditSheet> {
  late final TextEditingController _title;
  late TimeOfDay _time;

  @override
  void initState() {
    super.initState();
    final item = widget.initial;
    _title = TextEditingController(text: item?.title ?? '');
    final start = item?.startAt;
    _time = start != null
      ? TimeOfDay(hour: start.hour, minute: start.minute)
      : const TimeOfDay(hour: 18, minute: 0);
  }

  @override
  void dispose() {
    _title.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      mainAxisAlignment: MainAxisAlignment.min,
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        Text(
          widget.initial == null ? '予定を追加' : '予定を編集',
          style: const TextStyle(
            color: NexusColors.text,
            fontSize: 18,
            fontWeight: FontWeight.w700,
          ),
        ),
        const SizedBox(height: 12),
        TextField(
          controller: _title,
          style: const TextStyle(color: NexusColors.text),
          decoration: _input('タイトル'),
        ),
        const SizedBox(height: 10),
        ListTile(
          contentPadding: EdgeInsets.zero,
          title: const Text('時刻', style: TextStyle(color: NexusColors.textSecondary)),
          trailing: Text(
            '${two(_time.hour)}:${two(_time.minute)}',
            style: const TextStyle(color: NexusColors.cyan, fontWeight: FontWeight.w700),
          ),
        ),
        onTap: () async {
          final next = await showTimePicker(context: context, initialTime: _time);
          if (next != null) setState(() => _time = next);
        },
      ],
    );
  }
}
```

```

    },
  ),
  const SizedBox(height: 8),
  FilledButton(
    onPressed: () {
      final title = _title.text.trim();
      if (title.isEmpty) return;
      Navigator.pop(context, (title, _time));
    },
    child: const Text('保存'),
  ),
  if (widget.initial != null) ...[
    const SizedBox(height: 8),
    TextButton(
      onPressed: () => Navigator.pop(context, 'delete'),
      child: const Text('削除', style: TextStyle(color: NexusColors.expense)),
    ),
  ],
],
);
}
}

InputDecoration _input(String label) {
  return InputDecoration(
    labelText: label,
    labelStyle: const TextStyle(color: NexusColors.textMuted),
    enabledBorder: OutlineInputBorder(
      borderSide: BorderSide(color: NexusColors.border),
      borderRadius: BorderRadius.circular(12),
    ),
    focusedBorder: OutlineInputBorder(
      borderSide: const BorderSide(color: NexusColors.cyan),
      borderRadius: BorderRadius.circular(12),
    ),
  );
}

Future<void> openScheduleEditor(
  BuildContext context, {
  ScheduleItem? item,
  DateTime? day,
}) async {
  final store = AppScope.of(context);
  final result = await showNexusSheet<Object>(
    context: context,
    builder: (_) => ScheduleEditSheet(initial: item),
  );
  if (result == 'delete' && item != null) {
    store.deleteSchedule(item.id);
    return;
  }
  if (result is (String, TimeOfDay)) {
    final base = item != null
      ? dateOnly(item.startAt)
      : dateOnly(day ?? store.focusedDate);
    final start = DateTime(
      base.year,
      base.month,
      base.day,
      result.$2.hour,
      result.$2.minute,
    );
    if (item == null) {
      store.addSchedule(title: result.$1, startAt: start);
    } else {
      store.updateSchedule(item.copyWith(title: result.$1, startAt: start));
    }
  }
}
}

```

ファイル: lib/widgets/duration_picker.dart

行数: 228

```
import 'package:flutter/material.dart';

import '../app/theme.dart';
import '../core/format.dart';

const kMinStudyDurationMinutes = 1;
const kMaxStudyDurationMinutes = 12 * 60;

class DurationMinutesPicker extends StatefulWidget {
  const DurationMinutesPicker({
    super.key,
    required this.minutes,
    required this.onChanged,
    this.enabled = true,
    this.minMinutes = kMinStudyDurationMinutes,
    this.maxMinutes = kMaxStudyDurationMinutes,
  });

  final int minutes;
  final ValueChanged<int> onChanged;
  final bool enabled;
  final int minMinutes;
  final int maxMinutes;

  @override
  State<DurationMinutesPicker> createState() => _DurationMinutesPickerState();
}

class _DurationMinutesPickerState extends State<DurationMinutesPicker> {
  late FixedExtentScrollController _hours;
  late FixedExtentScrollController _mins;
  var _syncing = false;

  int get _maxHours => widget.maxMinutes ~/ 60;

  @override
  void initState() {
    super.initState();
    final total = widget.minutes.clamp(widget.minMinutes, widget.maxMinutes);
    _hours = FixedExtentScrollController(initialItem: total ~/ 60);
    _mins = FixedExtentScrollController(initialItem: total % 60);
  }

  @override
  void didUpdateWidget(covariant DurationMinutesPicker oldWidget) {
    super.didUpdateWidget(oldWidget);
    if (_syncing || widget.minutes == oldWidget.minutes) return;
    final total = widget.minutes.clamp(widget.minMinutes, widget.maxMinutes);
    _jump(_hours, total ~/ 60);
    _jump(_mins, total % 60);
  }

  @override
  void dispose() {
    _hours.dispose();
    _mins.dispose();
    super.dispose();
  }

  void _jump(FixedExtentScrollController controller, int item) {
    if (!controller.hasClients || controller.selectedItem == item) return;
    controller.jumpToItem(item);
  }

  void _emit({int? hours, int? minutes}) {
    if (!widget.enabled) return;
    final nextHours = (hours ?? (_hours.hasClients ? _hours.selectedItem : widget.minutes ~/ 60))
      .clamp(0, _maxHours);
    final nextMins = (minutes ?? (_mins.hasClients ? _mins.selectedItem : widget.minutes % 60))
```

```

        .clamp(0, 59);
final total = (nextHours * 60 + nextMins).clamp(widget.minMinutes, widget.maxMinutes);
if (total == widget.minutes) return;
_syncing = true;
widget.onChanged(total);
_syncing = false;
}

@override
Widget build(BuildContext context) {
  return Opacity(
    opacity: widget.enabled ? 1 : 0.45,
    child: IgnorePointer(
      ignoring: !widget.enabled,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          Text(
            '時間 ${studyGoalLabel(widget.minutes)}',
            textAlign: TextAlign.center,
            style: const TextStyle(
              color: NexusColors.cyan,
              fontSize: 16,
              fontWeight: FontWeight.w600,
            ),
          ),
          const SizedBox(height: 8),
          SizedBox(
            height: 168,
            child: Stack(
              alignment: Alignment.center,
              children: [
                IgnorePointer(
                  child: Container(
                    height: 44,
                    margin: const EdgeInsets.symmetric(horizontal: 8),
                    decoration: BoxDecoration(
                      color: NexusColors.cyan.withValues(alpha: 0.12),
                      borderRadius: BorderRadius.circular(12),
                      border: Border.all(color: NexusColors.cyan.withValues(alpha: 0.28)),
                    ),
                  ),
                ),
                Row(
                  children: [
                    Expanded(
                      child: _DurationWheel(
                        controller: _hours,
                        itemCount: _maxHours + 1,
                        onChanged: (value) => _emit(hours: value),
                      ),
                    ),
                    const Text(
                      ':',
                      style: TextStyle(
                        fontSize: 26,
                        height: 1,
                        fontWeight: FontWeight.w600,
                        color: NexusColors.textMuted,
                      ),
                    ),
                    Expanded(
                      child: _DurationWheel(
                        controller: _mins,
                        itemCount: 60,
                        pad: true,
                        onChanged: (value) => _emit(minutes: value),
                      ),
                    ),
                  ],
                ),
              ],
            ),
          ),
          const Row(
            children: [
              Expanded(

```

```

        child: Text(
          '時間',
          textAlign: TextAlign.center,
          style: TextStyle(color: NexusColors.textMuted, fontSize: 11),
        ),
      ),
      SizedBox(width: 12),
      Expanded(
        child: Text(
          '分',
          textAlign: TextAlign.center,
          style: TextStyle(color: NexusColors.textMuted, fontSize: 11),
        ),
      ),
    ],
  ),
],
),
),
);
}
}

class _DurationWheel extends StatefulWidget {
  const _DurationWheel({
    required this.controller,
    required this.itemCount,
    required this.onChanged,
    this.pad = false,
  });

  final FixedExtentScrollController controller;
  final int itemCount;
  final ValueChanged<int> onChanged;
  final bool pad;

  @override
  State<_DurationWheel> createState() => _DurationWheelState();
}

class _DurationWheelState extends State<_DurationWheel> {
  @override
  Widget build(BuildContext context) {
    return NotificationListener<ScrollNotification>(
      onNotification: (notification) => notification.depth == 0,
      child: ListWheelScrollView.useDelegate(
        controller: widget.controller,
        itemExtent: 44,
        perspective: 0.003,
        diameterRatio: 1.2,
        physics: const FixedExtentScrollPhysics(),
        onSelectedItemChanged: (index) {
          setState(() {});
          widget.onChanged(index);
        },
        childDelegate: ListWheelChildBuilderDelegate(
          childCount: widget.itemCount,
          builder: (context, index) {
            final selected =
              widget.controller.hasClients && widget.controller.selectedItem == index;
            final label = widget.pad ? two(index) : '$index';
            return Align(
              alignment: Alignment.center,
              child: Text(
                label,
                textAlign: TextAlign.center,
                style: TextStyle(
                  fontSize: selected ? 26 : 18,
                  height: 1,
                  leadingDistribution: TextLeadingDistribution.even,
                  fontWeight: selected ? FontWeight.w600 : FontWeight.w500,
                  color: selected ? NexusColors.text : NexusColors.textMuted,
                ),
              ),
            );
          },
        ),
      ),
    );
  }
}

```



```
    },  
    );  
}  
}
```

ファイル: lib/widgets/count_up_yen.dart

行数: 46

```
import 'package:flutter/material.dart';

import '../core/format.dart';

class CountUpYen extends StatefulWidget {
  const CountUpYen({
    super.key,
    required this.value,
    this.style,
  });

  final int value;
  final TextStyle? style;

  @override
  State<CountUpYen> createState() => _CountUpYenState();
}

class _CountUpYenState extends State<CountUpYen> {
  late int _from;

  @override
  void initState() {
    super.initState();
    _from = widget.value;
  }

  @override
  void didUpdateWidget(covariant CountUpYen oldWidget) {
    super.didUpdateWidget(oldWidget);
    if (oldWidget.value != widget.value) _from = oldWidget.value;
  }

  @override
  Widget build(BuildContext context) {
    return TweenAnimationBuilder<double>({
      tween: Tween(begin: _from.toDouble(), end: widget.value.toDouble()),
      duration: const Duration(milliseconds: 420),
      curve: Curves.easeOutCubic,
      builder: (context, value, _) {
        return Text(yen(value.round()), style: widget.style);
      },
    });
  }
}
```

ファイル: test/widget_test.dart

行数: 113

```
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:google_fonts/google_fonts.dart';

import 'package:nexus/app/app.dart';
import 'package:nexus/data/app_store.dart';
import 'package:nexus/data/models.dart';
import 'package:nexus/domain/money_calc.dart';
import 'package:nexus/domain/review_scheduler.dart';
import 'package:nexus/widgets/nexus_nav_bar.dart';

void main() {
  setUpAll(() {
    GoogleFonts.config.allowRuntimeFetching = false;
  });

  testWidgets('5タブを切り替えられる', (tester) async {
    tester.view.physicalSize = const Size(390, 844);
    tester.view.devicePixelRatio = 1;
    addTearDown(tester.view.resetPhysicalSize);
    addTearDown(tester.view.resetDevicePixelRatio);

    await tester.pumpWidget(const NexusApp());

    expect(find.text('蒼井 ユウ'), findsWidgets);
    expect(find.text('今日の予定'), findsOneWidget);

    Future<void> tapTab(String label) async {
      await tester.tap(find.descendant(of: find.byType(NexusNavBar), matching: find.text(label)));
      await tester.pumpAndSettle();
    }

    await tapTab('Study');
    expect(find.text('総勉強時間'), findsOneWidget);

    await tapTab('Life');
    expect(find.text('毎日を、整える。'), findsOneWidget);

    await tapTab('Money');
    expect(find.text('未来のために、今日を知る。'), findsOneWidget);
    expect(find.text('収入を追加'), findsOneWidget);
    expect(find.text('ボックスを追加'), findsOneWidget);
    expect(find.text('カードを追加'), findsOneWidget);
    expect(find.text('支払予定を追加'), findsOneWidget);

    await tapTab('設定');
    expect(find.text('Nexusを、自分らしく。'), findsOneWidget);
  });

  test('問題記録で1・5日の復習カードが作られる', () {
    final store = AppStore.seed();
    final math = store.addSubject(name: '数学');
    final before = store.reviewCards.length;
    final problem = store.addProblem(subjectId: math.id, title: '確認問題');
    final created = store.reviewCards.where((c) => c.problemId == problem.id).toList();
    expect(store.reviewCards.length, before + 2);
    expect(created.map((c) => c.intervalStep), [1, 5]);
    expect(
      created.map((c) => c.dueAt.difference(problem.learnedAt).inDays),
      [1, 5],
    );
  });

  test('ネグモの提案は承認するまで予定を変えない', () {
    final store = AppStore.seed();
    store.addSchedule(title: 'テレビ', startAt: DateTime(2026, 8, 14, 20));
    store.createDefaultProposal();
    final before = store.schedules.firstWhere((s) => s.title == 'テレビ').startAt;
    expect(store.proposal?.status, ProposalStatus.pending);
  });
}
```

```
expect(store.schedules.firstWhere((s) => s.title == 'テレビ').startAt, before);
store.approveProposal();
expect(store.proposal?.status, ProposalStatus.approved);
expect(
  store.schedules.firstWhere((s) => s.title == 'テレビ').startAt,
  before.add(const Duration(minutes: 30)),
);
});

test('Moneyの残高は収入-支出、使える額は残日数で割る', () {
  final boxes = [
    const BudgetBox(
      id: 'a',
      name: '食費',
      monthlyBudget: 10000,
      spent: 3600,
      color: Color(0xFF000000),
      icon: Icons.restaurant,
    ),
  ];
  final payments = [
    PaymentPlan(id: 'p', title: 'x', amount: 1000, dueAt: DateTime(2026, 8, 25)),
  ];
  final snap = MoneyCalc.compute(
    income: 40000,
    boxes: boxes,
    payments: payments,
    today: DateTime(2026, 8, 14),
  );
  expect(snap.expense, 3600);
  expect(snap.balance, 36400);
  expect(snap.spendableToday >= 0, isTrue);
});

test('ReviewScheduler は learnedAt から 1/5 日', () {
  final cards = ReviewScheduler.cardsFor(
    problemId: 'p',
    learnedAt: DateTime(2026, 8, 14),
    nextId: () => 'id',
  );
  expect(cards.map((c) => c.intervalStep).toList(), [1, 5]);
});
}
```

ファイル: test/home_content_test.dart

行数: 312

```
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';

import 'package:nexus/core/format.dart';
import 'package:nexus/data/app_store.dart';
import 'package:nexus/data/models.dart';
import 'package:nexus/domain/daily_quotes.dart';
import 'package:nexus/domain/day_occasions.dart';

void main() {
  test('8月18日は米の日で、漢字の由来が説明される', () {
    final day = occasionFor(DateTime(2026, 8, 18));
    expect(day.name, '米の日');
    expect(day.reason.contains('八'), isTrue);
    expect(day.alsoKnownAs, contains('高校野球記念日'));
  });

  test('国民の祝日は記念日より優先される', () {
    expect(occasionFor(DateTime(2026, 1, 1)).name, '元日');
    expect(occasionFor(DateTime(2026, 5, 5)).name, 'こどもの日');
    expect(occasionFor(DateTime(2026, 8, 11)).name, '山の日');
    expect(occasionFor(DateTime(2026, 5, 6)).name, '振替休日');
  });

  test('日付が変わると記念日も名言も変わる', () {
    final a = occasionFor(DateTime(2026, 8, 18));
    final b = occasionFor(DateTime(2026, 8, 15));
    expect(a.name, isNot(b.name));

    final q1 = quoteFor(DateTime(2026, 8, 18));
    final q2 = quoteFor(DateTime(2026, 1, 1));
    expect(q1.text, isNot(q2.text));
    expect(q1.author, isNotEmpty);
  });

  test('残りの勉強時間は60分以上なら時間、未満なら分数', () {
    expect(remainingStudyLabel(0), '達成');
    expect(remainingStudyLabel(25), '25分');
    expect(remainingStudyLabel(60), '1時間');
    expect(remainingStudyLabel(80), '1時間20分');
  });

  test('1時間以上のタイマー表示は時:分:秒になる', () {
    expect(mmss(25 * 60), '25:00');
    expect(mmss(90 * 60), '1:30:00');
    expect(mmss(12 * 60 * 60), '12:00:00');
  });

  test('今日の勉強目標を変えると残り時間が変わる', () {
    final store = AppStore.seed();
    store.setDailyStudyGoalMinutes(60);
    expect(store.dailyStudyGoalMinutes, 60);
    expect(store.remainingStudyMinutes(), 60);

    store.todayStudyLoggedSeconds = 20 * 60;
    expect(store.remainingStudyMinutes(), 40);
    expect(remainingStudyLabel(store.remainingStudyMinutes()), '40分');
  });

  test('提出物は期限までの日数、目標はサブゴール4つを持てる', () {
    final store = AppStore.seed();
    final math = store.addSubject(name: '数学');
    final today = store.focusedDate;
    store.addAssignment(
      subjectId: math.id,
      title: '小テスト',
      dueAt: today.add(const Duration(days: 3)),
    );
    expect(daysLeftLabel(today.add(const Duration(days: 3)), today), 'あと3日');
```

```

expect(
  store.assignments.any((a) => a.title == '小テスト'),
  isTrue,
);

store.addGoal(
  title: '英検準1級',
  current: 10,
  target: 100,
  dueAt: today.add(const Duration(days: 60)),
  subGoalTitles: ['単語', '長文', 'リスニング', 'スピーキング'],
);
expect(store.goals.last.subGoals.length, 4);
expect(store.goals.last.filledSubGoals.length, 4);
});

test('学習記録は積層グラフの科目時間に積まれる', () {
  final store = AppStore.seed();
  final math = store.addSubject(name: '数学');
  final before = store.weekStudyHours;
  store.addStudySession(subjectId: math.id, minutes: 60, focus: StudyFocus.peak);
  expect(store.weekStudyHours, greaterThan(before));
  expect(store.sessions.first.focus, StudyFocus.peak);
});

test('新しい教科は一覧にすぐ追加される', () {
  final store = AppStore.seed();
  final added = store.addSubject(name: '物理');
  expect(store.subjects.any((s) => s.id == added.id && s.name == '物理'), isTrue);
});

test('収入は入金月ではなく使用月で計算する', () {
  final store = AppStore.seed();
  store.incomes.clear();
  store.addIncome(
    name: '給料',
    amount: 140000,
    depositedAt: DateTime(2026, 8, 30),
    useYear: 2026,
    useMonth: 9,
  );
  expect(store.incomeForMonth(DateTime(2026, 8, 18)), 0);
  expect(store.incomeForMonth(DateTime(2026, 9, 1)), 140000);
});

test('カードを別ボックスへ移すと二重計上されない', () {
  final store = AppStore.seed();
  final food = store.addBudgetBox(
    name: '食費',
    icon: Icons.restaurant_rounded,
    color: const Color(0xFF3DA9FC),
    monthlyBudget: 30000,
    renewalDay: 1,
    tags: const ['外食'],
  );
  final train = store.addBudgetBox(
    name: '交通費',
    icon: Icons.train_rounded,
    color: const Color(0xFF9B6BFF),
    monthlyBudget: 5000,
    renewalDay: 1,
    tags: const ['電車'],
  );
  final card = store.addMoneyCard(
    boxId: food.id,
    title: 'サイゼリヤ追加',
    amount: 1480,
    at: store.focusedDate,
    tag: '外食',
  );
  final foodBefore = store.spentOfBox(food.id, month: store.focusedDate);
  final trainBefore = store.spentOfBox(train.id, month: store.focusedDate);
  store.updateMoneyCard(card.copyWith(boxId: train.id));
  expect(store.spentOfBox(food.id, month: store.focusedDate), foodBefore - 1480);
  expect(store.spentOfBox(train.id, month: store.focusedDate), trainBefore + 1480);
});

```

```

test('予算ボックスは指定した更新日から次の更新日前日までを一期間とする', () {
  final store = AppStore.seed();
  final created = store.addBudgetBox(
    name: '食費',
    icon: Icons.restaurant_rounded,
    color: const Color(0xFF3DA9FC),
    monthlyBudget: 30000,
    renewalDay: 10,
    tags: const ['外食'],
  );
  final box = created.copyWith(renewalDay: 10);
  final mid = store.budgetPeriod(box, DateTime(2026, 8, 18));
  expect(mid.start, DateTime(2026, 8, 10));
  expect(mid.end, DateTime(2026, 9, 9));
  final early = store.budgetPeriod(box, DateTime(2026, 8, 5));
  expect(early.start, DateTime(2026, 7, 10));
  expect(early.end, DateTime(2026, 8, 9));
});

test('貯蓄ボックスは月が変わっても残高を持ち越し、カード移動で残高が入れ替わる', () {
  final store = AppStore.seed();
  final korea = store.addSavingsBox(
    name: '韓国旅行',
    icon: Icons.flight_rounded,
    color: const Color(0xFFFF8AD2),
    targetAmount: 150000,
    openingAmount: 72000,
    tags: const ['ホテル'],
  );
  expect(store.savingsBalance(korea), 72000);
  expect(korea.isSavings, isTrue);

  store.addBudgetBox(
    name: '友達',
    icon: Icons.people_alt_rounded,
    color: const Color(0xFFFF8AD2),
    monthlyBudget: 10000,
    renewalDay: 25,
    tags: const ['ご飯', 'その他'],
  );
  final friend = store.bboxes.last;
  final card = store.addMoneyCard(
    boxId: friend.id,
    title: 'サイゼリヤ',
    amount: 1480,
    at: DateTime(2026, 8, 18),
    tag: 'ご飯',
  );
  expect(store.spentOfBox(friend.id, month: DateTime(2026, 8, 1)), 1480);
  store.updateMoneyCard(card.copyWith(boxId: korea.id, kind: MoneyCardKind.saveOut));
  expect(store.spentOfBox(friend.id, month: DateTime(2026, 8, 1)), 0);
  expect(store.savingsBalance(korea), 72000 - 1480);
});

test('教科の追加はJSON経由でも復元できる', () {
  final store = AppStore.seed();
  final added = store.addSubject(name: '物理');
  final restored = StudySubject.fromJson(added.toJson());
  expect(restored.name, '物理');
  expect(restored.id, added.id);
  expect(restored.icon.codePoint, added.icon.codePoint);
});

test('習慣は名前を付けて追加できる', () {
  final store = AppStore.seed();
  final added = store.addHabit(
    name: '読書',
    icon: Icons.menu_book_rounded,
    color: const Color(0xFF00D4FF),
  );
  expect(store.habits.any((h) => h.id == added.id && h.name == '読書'), isTrue);
});

test('習慣チェーンは連続日数で数える', () {
  final store = AppStore.seed();
  final today = store.focusedDate;
  store.habits.add(

```

```

    Habit(
      id: 'h-wake',
      name: '起床',
      icon: Icons.wb_sunny_rounded,
      color: const Color(0xFFFFC857),
      doneDays: {
        for (var i = 0; i < 5; i++) dateKey(today.subtract(Duration(days: i))),
      },
    ),
  );
});
final wake = store.habits.firstWhere((h) => h.name == '起床');
expect(wake.currentStreak(store.focusedDate), 5);
store.toggleHabit(wake.id, store.focusedDate);
final after = store.habits.firstWhere((h) => h.name == '起床');
expect(after.doneOn(store.focusedDate), isFalse);
expect(after.currentStreak(store.focusedDate), 4);
});

test('睡眠は就寝と起床から時間を記録する', () {
  final store = AppStore.seed();
  store.sleepLogs.clear();
  store.logSleep(
    bedAt: DateTime(2026, 8, 17, 23, 30),
    wakeAt: DateTime(2026, 8, 18, 7, 0),
    quality: 4,
  );
  expect(store.sleepHours, 7.5);
  expect(store.sleepLogOn(DateTime(2026, 8, 18))?.qualityLabel, 'よい');

  store.startSleep(at: DateTime(2026, 8, 18, 23, 0));
  expect(store.isSleeping, isTrue);
  store.wakeUp(at: DateTime(2026, 8, 19, 7, 0), quality: 5);
  expect(store.isSleeping, isFalse);
  expect(store.sleepLogOn(DateTime(2026, 8, 19))?.hours, 8.0);
});

test('Lifeの日付を変えても他画面の日付は変わらない', () {
  final store = AppStore.seed();
  final today = store.focusedDate;
  store.setLifeDate(DateTime(today.year, today.month, 19));
  expect(store.lifeDate.day, 19);
  expect(store.focusedDate, today);
  store.setLifeDate(DateTime(today.year, today.month, 27));
  expect(store.lifeDate.day, 27);
  expect(store.focusedDate, today);
});

test('ボックスと支払予定は削除でき、カードは未振り分けへ移る', () {
  final store = AppStore.seed();
  final unassigned = store.addBudgetBox(
    name: '未振り分け',
    icon: Icons.inbox_rounded,
    color: const Color(0xFF8B9BB4),
    monthlyBudget: 0,
    renewalDay: 1,
    tags: const ['その他'],
  );
  final food = store.addBudgetBox(
    name: '食費',
    icon: Icons.restaurant_rounded,
    color: const Color(0xFF3DA9FC),
    monthlyBudget: 30000,
    renewalDay: 1,
    tags: const ['外食'],
  );
  store.addMoneyCard(
    boxId: food.id,
    title: 'サイゼリヤ',
    amount: 1480,
    at: store.focusedDate,
    tag: '外食',
  );
  expect(store.cards.any((c) => c.boxId == food.id), isTrue);
  store.deleteBox(food.id);
  expect(store.boxById(food.id), isNull);
  expect(store.cards.any((c) => c.boxId == food.id), isFalse);
  expect(store.cards.any((c) => c.title == 'サイゼリヤ' && c.boxId == unassigned.id), isTrue);
});

```



```
store.addPaymentPlan(  
  title: 'スマホ代',  
  amount: 3000,  
  dueAt: store.focusedDate.add(const Duration(days: 7)),  
);  
final payId = store.payments.last.id;  
store.deletePayment(payId);  
expect(store.payments.any((p) => p.id == payId), isFalse);  
});  
}
```